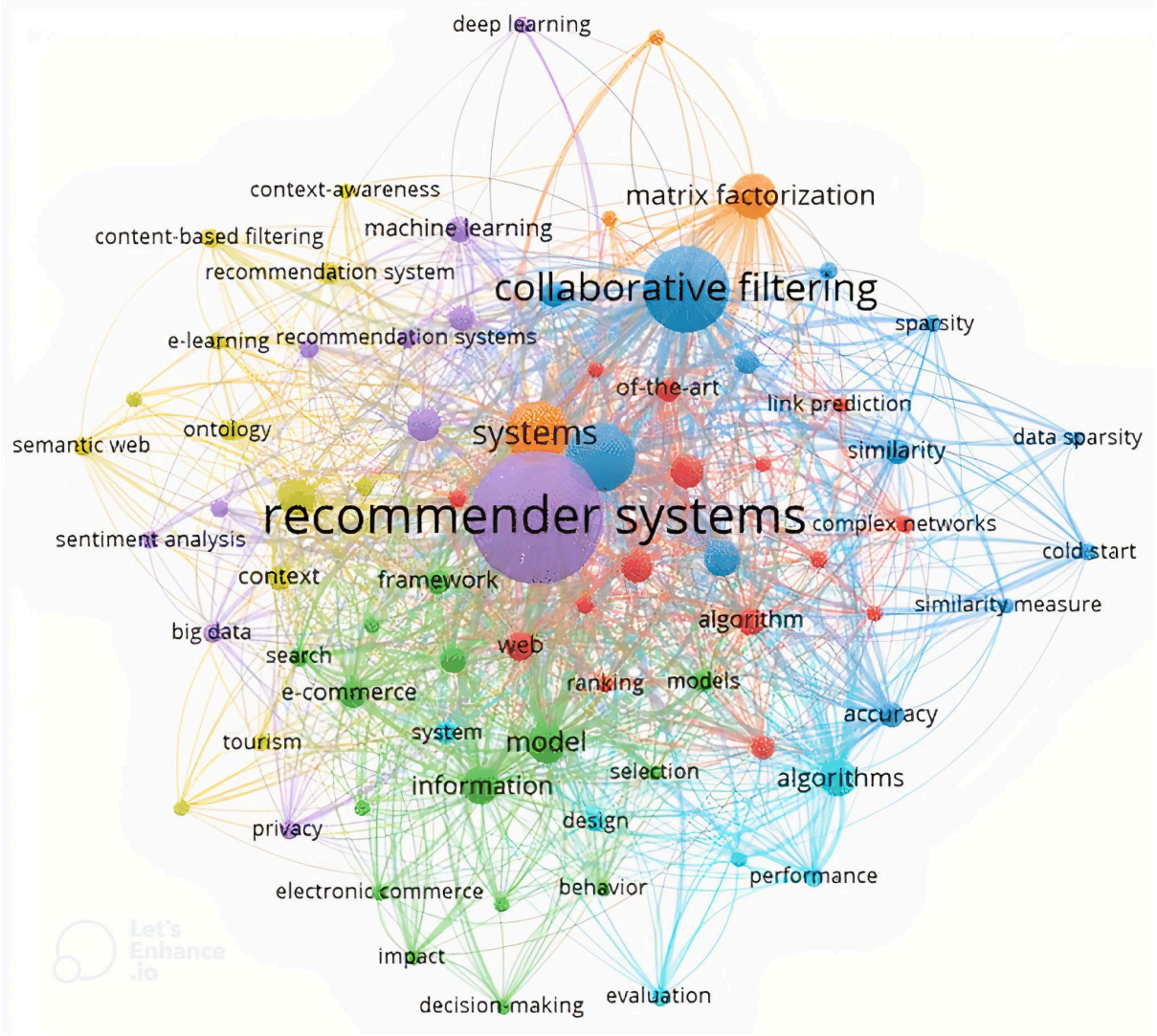
Final Year Project - Final Report

Template: CM3005 Data Science – Project 1.1



My Final Project GitHub Repository Link:
https://github.com/maryamzaman30/Data-Driven-Personalized-Educational-Content-Recommendation-System/tree/main

My Project Title:
Data-Driven Personalized Educational Content Recommendation System for TOEIC Preparation

---

Table Of Contents

List Of Figures

List Of Tables

Chapter 1: Introduction (766/1000 words)

1.1 Project Overview

This project tackles a significant challenge in online education: **content overload**. With an ever-expanding pool of educational resources available on platforms such as **Coursera**, **edX** and **YouTube**, learners are increasingly overwhelmed by the volume of choices. Selecting content that matches their **learning preferences, pace and academic goals** can become an obstacle to engagement and progression. To address this, this project proposes a **personalized educational content recommendation system** that intelligently recommends resources (**question bundles or lecture materials**) aligned with a learner's needs, recommends the most suitable content (questions, lessons) and addresses content overload via hybrid personalized recommendation from **EdNet.** EdNet is a **real-world**, large-scale educational dataset designed specifically for evaluating learning patterns and performance.

Hybrid recommender systems are **commonly** used in **real-world** applications like **Netflix, Spotify and e-commerce platforms**. Hybrid recommendation systems are widely used in industry [1] due to their ability to combine the strengths of multiple models, content-based methods excel at leveraging metadata and content semantics, while collaborative filtering captures behavioral patterns. This approach not only improves accuracy but also offers a foundation for producing **academically valuable** results, which is why I chose this approach. My project's implementation will mimic practical systems (e.g., combining behavioral patterns and metadata) and later on expand to deep learning.

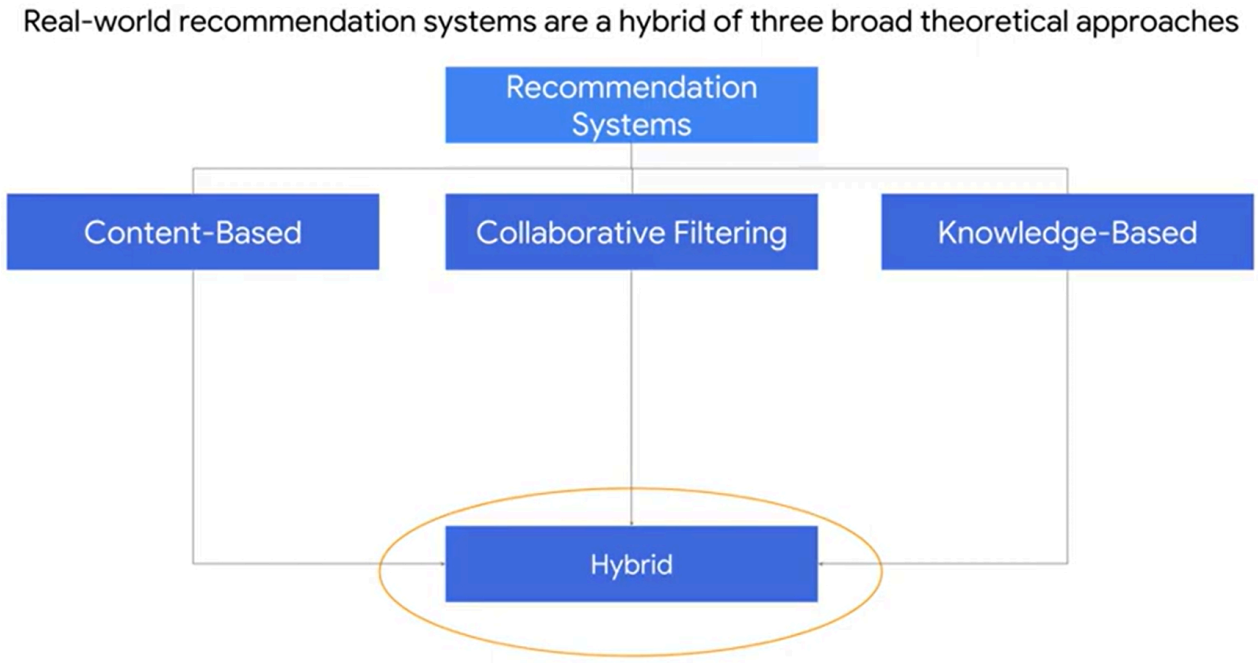


Figure 1: Showing how hybrid recommendation system is implemented in real world applications

1.2 Template Used

This project is based on **CM3005 Data Science – Project Idea 1.1: Data-Driven Personalised Educational Content Recommendation**. This template emphasizes advanced data science approaches to deliver personalized recommendations by analyzing user interactions, content features and learning outcomes. It aligns well with current research trends in **Educational Data Mining (EDM)** and **Learning Analytics**.

The title/name of my project is **Data-Driven Personalized Educational Content Recommendation System for TOEIC Preparation**. This reflects both the technical foundation of the system, leveraging real-world learner interaction data and its educational focus, which is specifically tailored to the TOEIC exam. It emphasizes personalization, data-driven modeling and the integration of multiple recommendation techniques to support adaptive learning in English language assessment. **This part is covered in detail in the next chapters.**

This project will use EdNet Datasets https://github.com/riiid/ednet

1.3 Motivation

Online education and self-paced courses have grown quickly because of new technology and the need caused by global events like the COVID-19 pandemic. While online learning offers flexibility and accessibility, it also presents unique challenges, particularly concerning student retention. [2]

With rising dropout rates and low engagement in self-paced courses [2], one of these factors influencing student persistence and dropout in online education include personal motivation, self-regulation and prior academic experiences [2]. The need for **intelligent personalization** is clear. Learners face **decision fatigue** when choosing from overwhelming amounts of content. By delivering tailored, explainable recommendations, this system aims to; **Reduce cognitive overload, Improve retention and motivation and enhance course completion rates.** The use of interpretable scores (success rate, tag metadata) and multiple recommendation types will address this.

Unlike generic recommenders such as **Google Recommendations AI** or **Deep Knowledge Tracing (DKT) [13]**, which are typically black-box systems with limited educational transparency, my project explicitly integrates interpretable **content-based features** (e.g. part names, subject categories, success rate, tags) and **learner engagement signals** (e.g. interaction count, correctness rate) to enhance **trust and adaptability**. Engagement signals are captured through correctness-weighted interaction scores. Additionally, the **meta-learner ensemble model** learns to weigh interpretable features like `part_id` and `success_rate`, enabling both transparency and tunable educational relevance in recommendations (e.g. students who perform well on intermediate grammar receive bundles with `success rates ~0.5` and topics like `vocabulary_business`). This is covered in detail in **Chapter 2: Literature Review**

Leveraging curated datasets such as **EdNet**, the project contributes to **Educational Data Mining** and supports applications in **adaptive learning platforms, curriculum design** and **future extensions** via deep or reinforcement learning techniques.

Why Recommendation Beats Search

A recommendation system **assists** users in **discovering engaging content within vast collections**. For instance, platforms like the Google Play Store offer millions of apps and YouTube hosts billions of videos. With new apps and videos being added regularly, how can users uncover fresh and interesting content? While searching or filtering is one way to find specific items, a recommendation engine goes a step further by suggesting content that users might not have considered searching for themselves [3]. This leads to better **engagement, discovery and satisfaction**.

Moreover, users **themselves** prefer to see a **short list** of likely items rather than **struggling** with the **full catalog**. They try to find the best items, but sometimes they don't even know what they really want (e.g., Best sweater for staying warm in winter). In a physical store, an expert sales associate would assist by offering helpful recommendations. So, why not in an online store? [4]

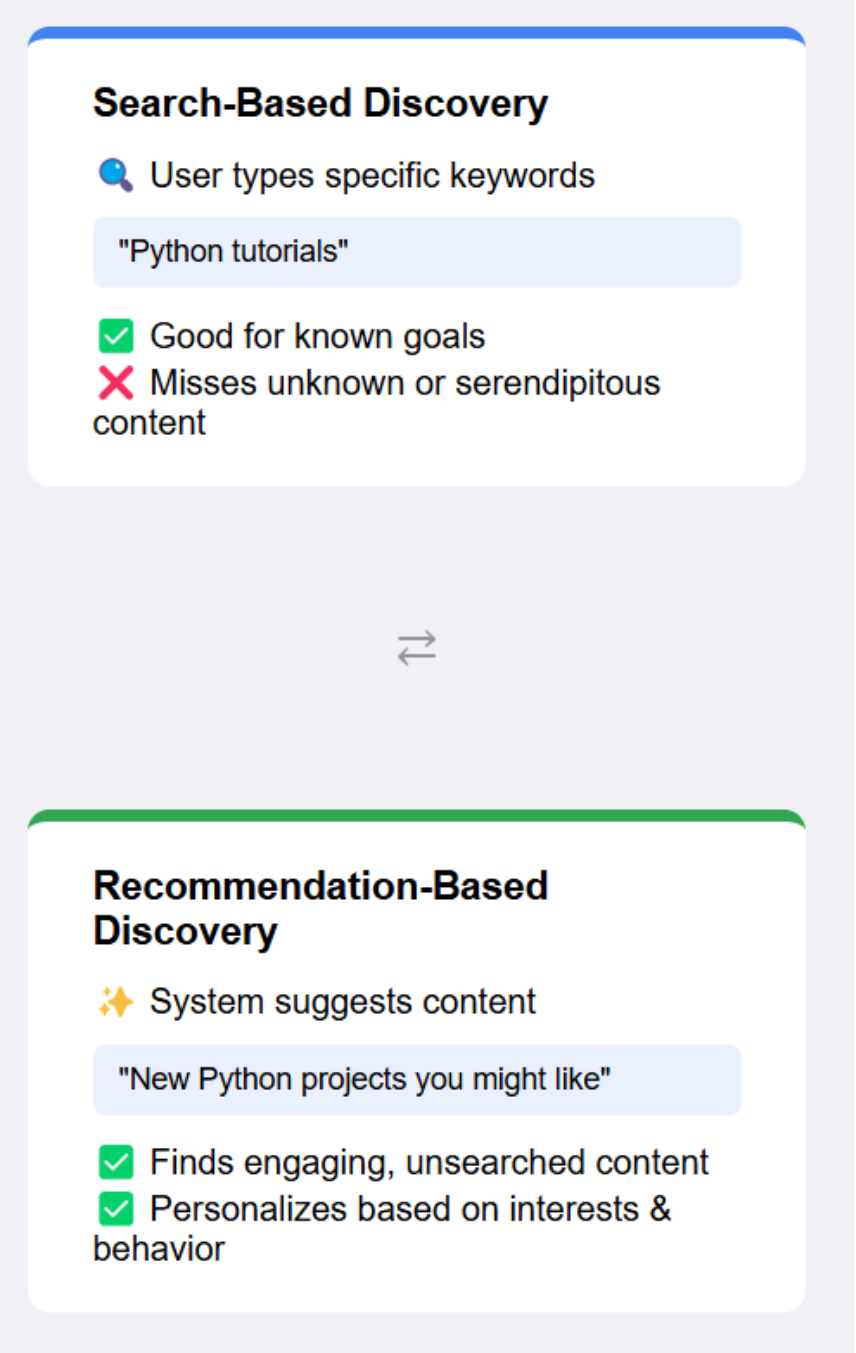


Figure 2: Comparison between Search-Based & Recommendation-Based Content Discovery

---

Chapter 2: Literature Review (2415/2500 words)

Recent surveys confirm that *hybrid* recommendation models dominate educational settings [5]. EdTech platforms increasingly seek to personalize content by matching learning resources to individual student profiles. For example, Edmodo's partnership with IBM/Watson aims to deliver "personalized recommendations for learning resources" tailored

to each student's grade and interests [6]. This real-world initiative underscores the industry demand for adaptive content filtering in education. However, such systems often lack transparency in algorithmic design and studies point out that most evaluations focus on accuracy rather than learning outcomes [5]. This gap motivates comprehensive models that combine effective algorithms with pedagogical impact.

2.1 Similar Projects

Hybrid Content/CF Models (TF-IDF + SVD)

Several projects combine content-based TF-IDF with collaborative SVD methods. **Mediani (2022)** presents a hybrid recommender for pedagogical resources that uses TF-IDF to extract keywords and SVD for matrix factorization based on collaborative signals, ‘’*improving accuracy*” over single-technique baselines. However, the authors note that **deep learning** is still necessary for richer semantic understanding and this work lacked scalability testing on very large datasets [7].

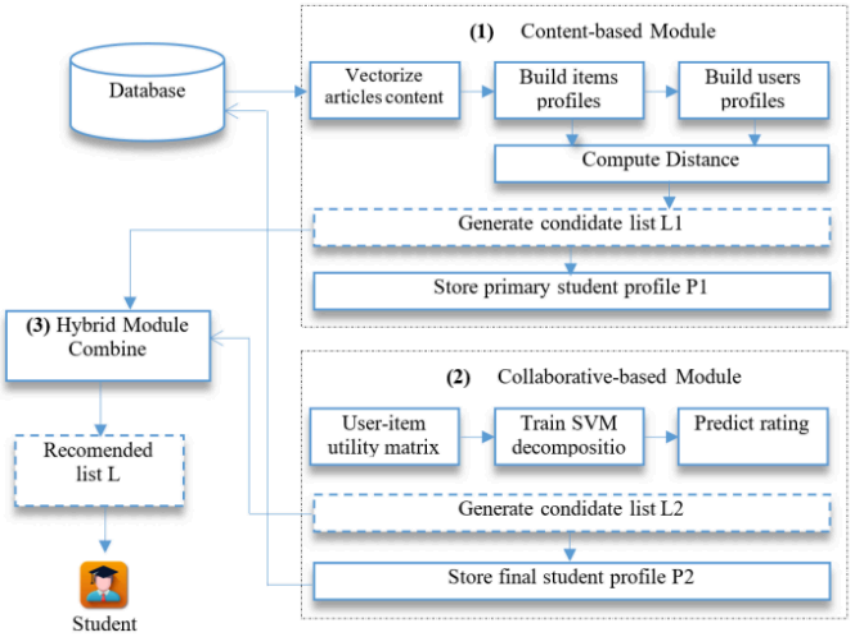


Figure 3: **Mediani (2022)’s** system detailed architecture

Similarly, the **MoodleREC** plugin for Moodle LMS (De Medio et al., 2019) offers a hybrid search: teachers query keywords to retrieve resources, with two ranked lists, one by keyword relevance and another by social usage. While it supports personalization with both content-based and collaborative recommendations, it presents them separately, rather than integrating the models. Additionally, MoodleREC **does not use advanced language models** like **Sentence-BERT**, relying on keyword matching and ratings, and lacks a unified hybrid weighting, leaving the teacher to balance the signals. This plugin is limited by their reliance on interaction histories [8]

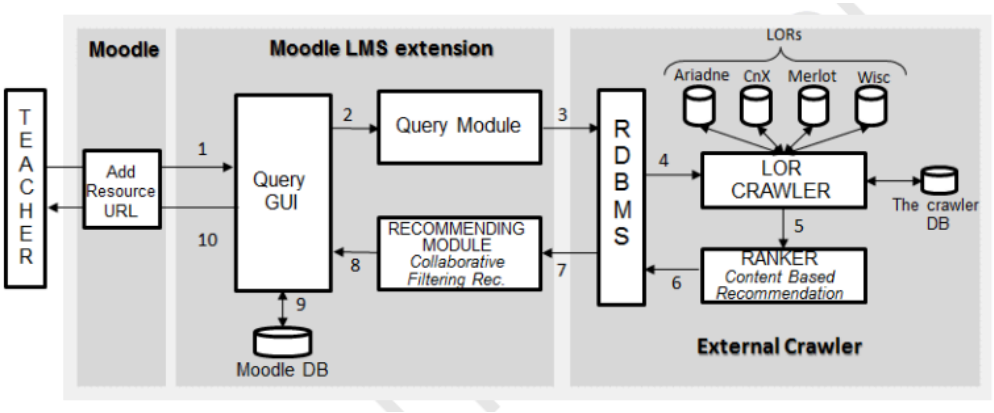


Figure 4: The functional schema of the Moodle system

This aligns with broader limitations in **educational recommendation systems**, where **deep semantic understanding** and **adaptive personalization** are often missing

These limitations directly informed my choice to explore hybrid methods that balance scalability with inclusivity and support my approach of fusing content and CF: TF-IDF profiles help with textual content filtering, while SVD (matrix factorization) captures user-rating patterns. However, they also reveal limitations: none apply advanced language models to understand content deeply and none use a unified hybrid weighting (MoodleREC leaves integration to the user) [8]. My project addresses these gaps by first implementing basic content-based and collaborative models (02_tf_idf.ipynb, 03_svd_hybrid.ipynb) as building blocks, then developing a weighted hybrid model with unified weighting (04_model_tuning.ipynb) and further enriching it with neural embeddings (05_advanced_recommendations.ipynb).

Educational content now includes a wide spectrum of formats; from MOOCs and videos to interactive simulations. For example, informal whiteboard notes on e-learning mention “MOOCs”, “Flipped Classroom”, “Web 2.0” and learning platforms. In practice, platforms like Edmodo use AI to match this rich content to learners [6]. Hybrid models that combine content-based TF-IDF and collaborative SVD have been shown to improve recommendation accuracy in education [7]. These approaches motivate my use of personalized profiling, though I extend them by adding deep semantic understanding **(next sections)**.

Neural Collaborative Filtering (NCF) in Education

Deep neural CF models have been explored for personalized course and content recommendations. **Laroibafi and Rumaisa (2024)** design a “Course Learning Recommendation System” using *neural collaborative filtering*. They fuse matrix-factorization embeddings with a multi-layer perceptron to model complex user-course interactions.

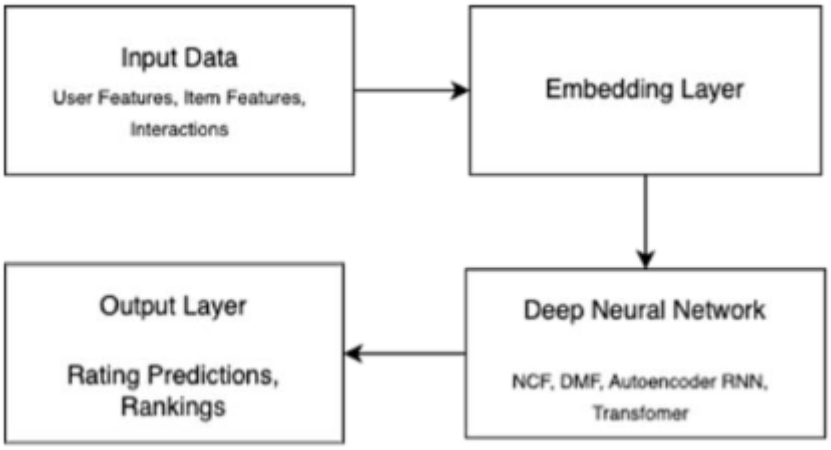


Figure 5: Research Design by **Laroibafi and Rumaisa (2024)**

This approach focuses on **diverse learner preferences** by using **Neural Collaborative Filtering (NCF)**, which helps capture **non-linear interactions**, something traditional **Collaborative Filtering (CF)** often struggles with.

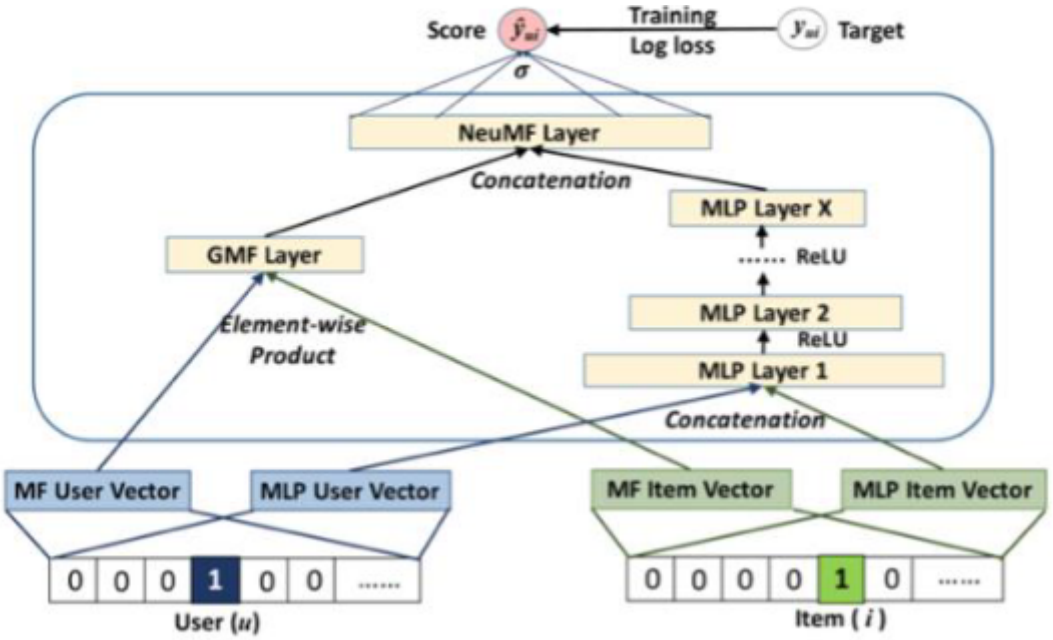


Figure 6: Neural Collaborative Filtering Framework

However, the paper mainly **describes system design** [9] and **does not provide quantitative evaluation results**, such as performance metrics like **Precision@K, Recall@K, or AUC**. Additionally, while **NCF improves collaborative filtering**, the model **does not incorporate content features**, meaning it may still **struggle with cold-start problems** (i.e., recommending content to new users with little history) or **content-rich scenarios** where metadata plays a crucial role.

Likewise, **Ullah et al. (2020)** propose *Deep Edu*, a deep neural CF model for recommending books and educational services. Deep Edu embeds users and items into latent factors and passes them through deep layers. The authors confirm that **Deep Edu outperforms**

traditional educational recommendation methods, particularly by **mitigating data sparsity and cold-start issues [10]**. In fact, they report that Deep Edu yields higher accuracy than a standard NCF baseline when the data are very sparse [10]. These findings confirm that neural CF can enhance recommendations in education, but they also highlight a gap: Deep Edu is purely collaborative (no item content is used) and its evaluation **focuses on rating prediction** rather than **broader recommendation quality** or **content-based filtering**.

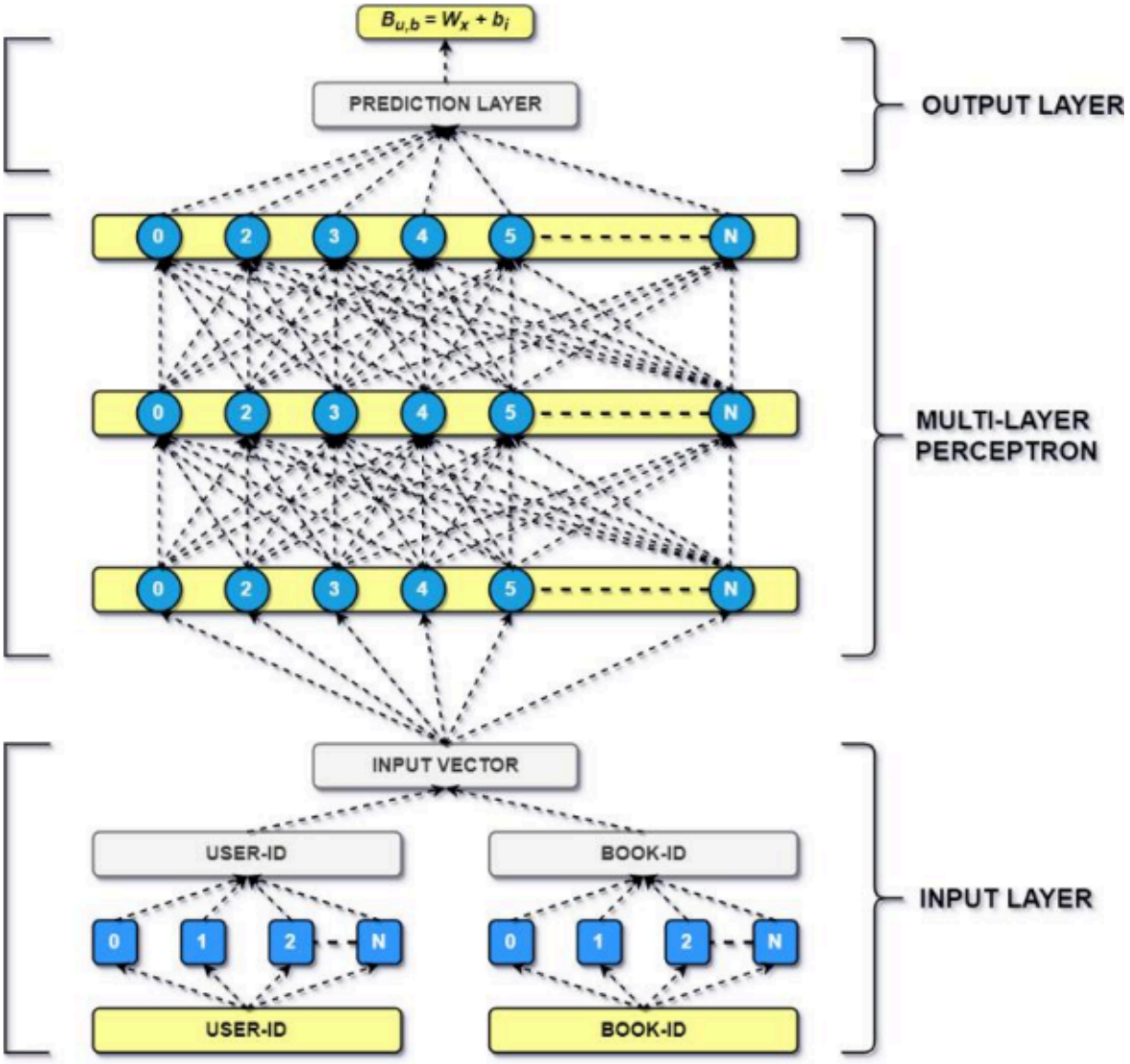


Figure 7: Deep Edu Architecture

Overall, these neural-CF projects validate the use of personalization and deep learning in educational recommenders. They suggest that multi-layer networks (as in Neural Collaborative Filtering) are effective when profile data are limited [10] Nonetheless, their reliance on user-item matrices means they do not address content understanding (such as book metadata or descriptions). My work will incorporate NCF (05_advanced_recommendations.ipynb) while also injecting content signals (TF-IDF, BERT) to fill this gap. These neural models outperform linear factorization in sparse settings. However, these models often act as **black boxes**, limiting interpretability, an important factor

in educational contexts. I addressed this gap by introducing a meta-learner ensemble, which not only leveraged the predictive power of deep learning but also allowed recommendations to be broken down into contributions from different algorithms, making the system more transparent for educators and learners.

Transformer-Based (BERT) and Content Embedding Models

Recent work applies large language models to capture content semantics in recommendations. **Sangeetha et al. (2025)** introduce *FindMate*, a hybrid recommender for academic networking. They explicitly combine TF-IDF (for surface text relevance) with BERT embeddings (for context). In their evaluation on custom user-profile data, adding BERT *dramatically improves* clustering quality and NDCG relative to TF-IDF alone [11]. For example, they report that incorporating BERT's "contextual understanding" noticeably boosts recommendation metrics [11]. This confirms that transformer-level features can capture nuanced meaning that TF-IDF misses. However, FindMate's domain is peer matching (connecting scholars), not educational content, and it uses these techniques for profile vectors rather than for course materials. This points to my project's novelty: applying BERT embeddings specifically to learning content (TOEIC resources) within a course recommendation framework.

(a) TF-IDF Recommendations

Name	Domain/Skillset	Similarity score	Cluster
Joan Evans	ai ml, C, C++, Python, Java, HTML, CSS	0.9803	101
Gregory Williamson	Data Mining, C, C++, Python, Java, SQL	0.9714	101
Alexis Moore	Java, HTML, CSS, SQL, AWS, FlutterFlow	0.9524	101
Joshua Hughes	Marketing,Game design, C, C++, Pytho	0.9439	101
Jacob Harper	SQL, AWS, Figma, Canva, Adobe XD	0.8797	101

(b) BERT Recommendations

Name	Domain/Skillset	Similarity score	Cluster
Gregory Williamson	Data Mining, C, C++, Python, Java, SQL	0.9895	97
Alexis Moore	Java, HTML, CSS, SQL, AWS, FlutterFlow	0.9894	97
Joan Evans	ai ml, C, C++, Python, Java, HTML, CSS	0.9863	97
Joshua Hughes	Marketing,Game design, C, C++, Python	0.9846	97
Paula Russell	Cyber security, python, HTML, ReactJS	0.9221	83

(c) Hybrid Recommendations

Name	Domain/Skillset	Similarity score	Cluster
Joan Evans	ai ml, C, C++, Python, Java, HTML, CSS	0.9833	102
Gregory Williamson	Data Mining, C, C++, Python, Java, SQL	0.9804	102
Alexis Moore	Java, HTML, CSS, SQL, AWS, FlutterFlow	0.9709	102
Joshua Hughes	Marketing,Game design, C, C++, Pytho	0.9643	102
Paula Russell	Cyber security, python, HTML, ReactJS	0.8943	102

Figure 8: Illustrating that FindMate connect scholars

Similarly, **Moon et al. (2024)** develop a BERT-based recommender for online courses. Their system embeds each Udemy course via BERT and then applies content-based filtering to match courses to learners' past enrollments [12]. They find that leveraging BERT "revolutionizes" course selection by capturing deep course semantics. However, their approach remains content-only: they generate course vectors and use cosine similarity for personalization [12]. They do not integrate collaborative filtering, which means their model could still suffer from cold-start or lack social proof. Notably, Moon *points out* that traditional CF struggles with cold-start and CB has trouble capturing complex preferences [12]. This observation motivates my hybrid design: I will use BERT (05_advanced_recommendations.ipynb) to enrich TF-IDF content vectors, *and* I combine them with user-course interaction models (SVD/NCF) to cover both sides of the problem.

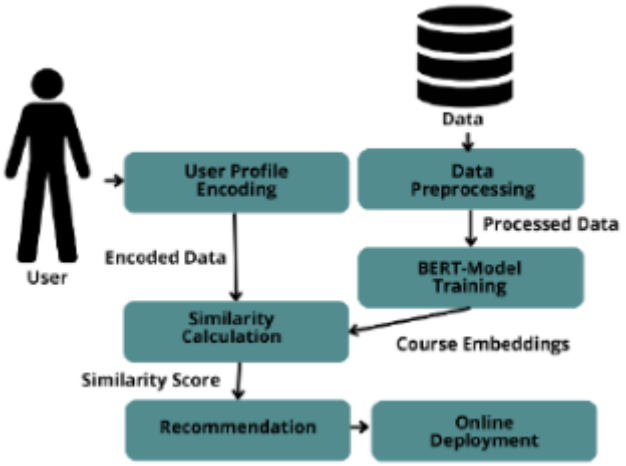


Figure 9: Schematic diagram of **Moon et al. (2024)**

Overall, these studies demonstrate the value of semantic embeddings in capturing domain-specific meaning, but they often assume access to fine-tuned language models. My project adapted these insights by applying pre-trained Sentence-BERT embeddings to TOEIC content, striking a balance between computational feasibility and semantic precision.

EdNet Dataset in Action

Dongmin Shin. (2021)'s SAINT+ is a Transformer-based knowledge tracing model developed to predict student correctness in the EdNet dataset by integrating temporal features such as elapsed time (the time taken to answer a question) and lag time (the duration between learning activities) [13]. While SAINT+ improves accuracy compared to previous models and I was **considering it for my project** at first, it still has several shortcomings that limit its effectiveness in educational recommendation systems

SAINT+ is **not a recommendation system** but a **knowledge tracing model**. Knowledge tracing focuses on **predicting student performance** rather than **suggesting learning materials**. Specifically, SAINT+ aims to determine **whether a student will answer a**

question correctly based on their past interactions, integrating temporal features like **elapsed time and lag time** to enhance prediction accuracy. [13]

Unlike **recommendation systems**, which **suggest learning resources or exercises** based on user behavior, SAINT+ primarily works as a **predictive tool for assessing mastery**. Its future directions mention expanding to **learning activities such as watching lectures**, which could make it more relevant for **adaptive learning systems**. However, the paper does not explicitly propose a recommendation mechanism that **guides students to suitable educational content**, it remains centered on **response prediction**.

My project, on the other hand, **is a recommendation system**. While it also utilizes EdNet data, my approach **personalizes content selection** rather than just predicting correctness. By combining **TF-IDF, SVD, NCF and Sentence-BERT**, I ensure that students **receive relevant study materials**, addressing both **knowledge gaps and personalized learning preferences**, something SAINT+ does not inherently do

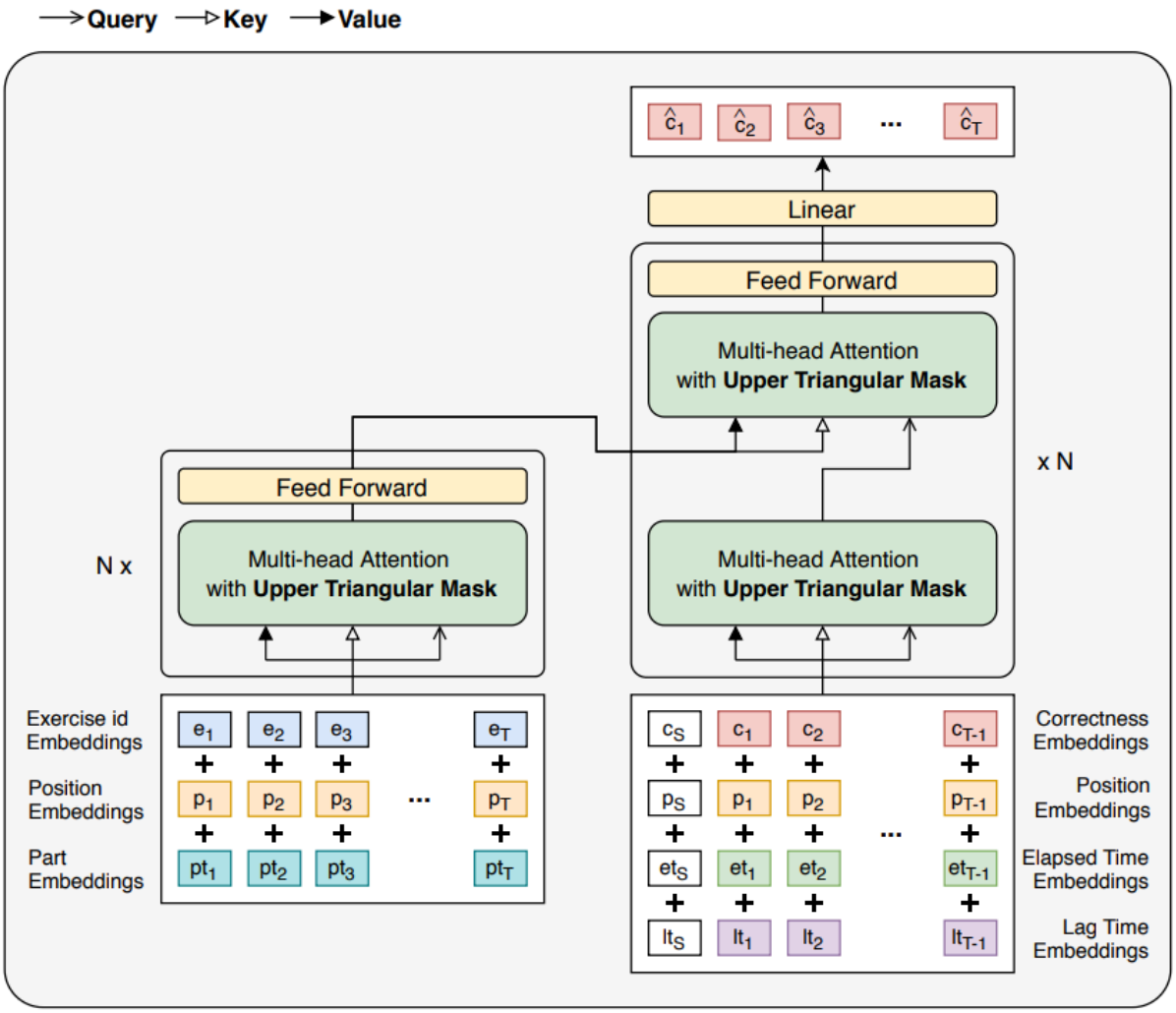


Figure 10: Model architecture of SAINT+

Modern e-learning integrates various modes, physical lectures, online videos, mobile apps, etc. A below photograph of lecture notes lists tools like **Moodle**, **Blended Learning**, **MOOC** and **Video Lectures**. **Effective recommendations** across these mediums require **deep semantic understanding**, where **transformer models** like BERT excel. Studies show

BERT's embeddings outperform TF-IDF for content similarity, leading to **more accurate** suggestions [11]. Hybrid systems combining content-based and collaborative filters yield the best accuracy [5]. My project combines TF-IDF, SVD/NCF and BERT to address limitations in prior work, such as lack of deep content features, personalization and isolated hybrid components.

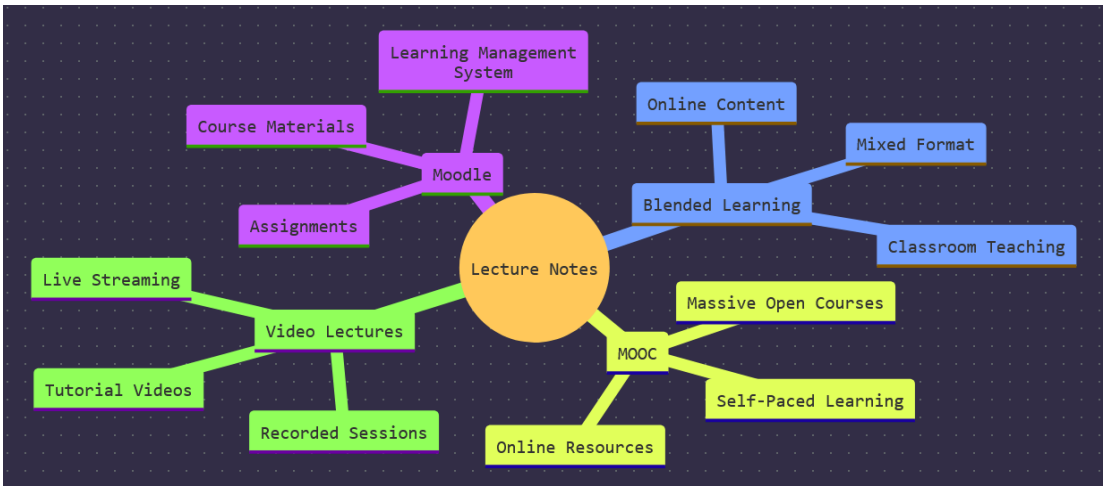


Figure 11: Diagram illustrating blended learning components; including Moodle, MOOCs and video lectures

2.2 Industry Applications and Gaps

While academic work informs technique, industry showcases real needs. For instance, Edmodo's personalized recommendation engine, powered by IBM Watson, suggests grade-appropriate resources [6]. This shows AI-driven recommendations are key to addressing learning gaps. However, without published details, such implementations leave us uncertain of their methods. Many platforms (e.g., LMS plugins, MOOCs) use simple content tagging or popularity-based recommendations. Literature suggests several unmet needs: deeper content understanding (beyond TF-IDF), fully integrated hybrids and rigorous evaluation on learning outcomes. My project bridges these gaps using advanced models (BERT/NCF) alongside traditional techniques like TF-IDF and SVD, focusing on TOEIC content personalization. In summary, these sources highlight the importance of personalization and hybrid methods while identifying gaps that my project aims to fill.

2.3 Reasons for Credibility of References

[5]

Published in a Peer-Reviewed Journal: The article is published in *Education and Information Technologies*, a respected peer-reviewed journal, which means it has undergone rigorous review by experts in the field.

[6]

Authoritative Source: This is a press release from NetDragon Websoft Holdings, a reputable company known for developing games, mobile VR/AR and educational

technologies. The partnership with IBM Watson Education adds further weight to the credibility of this information, as IBM is a well-known leader in AI and educational technology.

Company Publication: Press releases issued by established companies are often credible sources of information about technological developments, partnerships and new initiatives, as they reflect the company's official position and updates. According to a 2024 State of the Media Report, ‘’89% of journalists’’ consider official press releases their most trusted source for organizational news. Additionally, companies that regularly issue press releases are ’three times more likely’ to be quoted as industry experts

[7]

Published in a Peer-Reviewed Journal: *Ingénierie des Systèmes d'Information* is a peer-reviewed academic journal. Articles published in such journals are scrutinized for their validity, methodology and relevance to the field.

[8]

Will be published in a Peer-Reviewed Journal: This is not the final version; it has been provided to give early visibility of the article. The PDF states that the final version will undergo additional copyediting, typesetting and review before publication in its final form. It will be published in *Computers in Human Behavior*, a peer-reviewed academic journal .

[9]

Published in a Peer-Reviewed Journal: *Brilliance: Research of Artificial Intelligence* is a peer-reviewed journal, ensuring that the research methodology and findings are reviewed for scientific accuracy and validity.

[10]

Published in IEEE Access: IEEE Access is an open-access, peer-reviewed journal published by IEEE, one of the largest and most reputable organizations for electrical and electronics engineers. Articles in this journal are highly regarded for their scientific rigor.

[11]

Preprint on arXiv: arXiv is a widely respected open-access repository for scientific preprints. Although the paper has not yet gone through formal peer review, arXiv submissions often undergo scrutiny by experts in the field. Preprints from credible authors can be considered valuable, especially if they are from reputable institutions like Vellore Institute of Technology.

[12]

Conference Proceedings: published in the ’2024 6th International Conference on Electrical Engineering and Information & Communication Technology (ICEEICT)’. Since it was presented at an IEEE conference, it is ’peer-reviewed’, making it a credible source for

academic research. Conference papers undergo a review process before acceptance, ensuring the validity of their methodology and findings.

[13]

Published at a Top-Tier Venue: It was presented at the LAK 2021 conference (ACM Learning at Scale), which is a respected venue for learning analytics and educational data mining research. ACM (Association for Computing Machinery) is one of the most reputable publishers in computer science.

Peer-Reviewed: Papers accepted at LAK undergo rigorous peer review, ensuring that the methodology, results and contributions meet high academic standards.

2.4 Research Studies on Effectiveness

A study found that **76% of teachers believe personalized learning enhances engagement** and students in **personalized programs score 30% higher on ‘’standardized tests’’,** They also do better in math and ‘’reading’’, this is relevant to my project which is about English Test and Reading [14]

English remains the global business language and the **TOEIC score is a formal requirement in sectors such as aviation, hospitality and international business**. For learners from non-English-speaking or developing countries, personalized recommendation can reduce test anxiety, improve learning focus and increase their TOEIC score, directly impacting their **career mobility or visa eligibility** [15] [16]

For Dataset

Educational Domain-Specific: EdNet directly focuses on learning activities, fitting perfectly with the project’s aim of improving educational outcomes [17].

Research Alignment: EdNet has been extensively used in academic papers (e.g., Deep Knowledge Tracing) and machine learning benchmarks, ensuring that the project aligns with current research practices. [18] [19]

One such example is **SAINT+**, which is already discussed in **2.1 Similar Projects** [13]

Large Size: Due to the large size of the dataset, I will utilize deep learning and make small adjustments to the loss function and optimizer. For the optimizer, I will use Adam, which validates the strategy in [20], where this approach drastically improved the results.

State of the art build: Several datasets, such as ASSISTments, Junyi Academy, Synthetic and STATICS, are publicly available and widely used, they are not large enough to leverage the full potential of state-of-the-art data-driven models and limits the recorded behaviors to question-solving activities. To this end, I use EdNet [21]

Samplings: Many datasets are too large, so sampling is a common technique used to make analysis more manageable. Sampling allows us to work with a representative subset of the data rather than the full dataset, reducing computational costs and improving efficiency. This validated my dataset’s sampling [22]

Chapter 3: System Design (1910/2000 words)

3.1 Domain and Target Users

Table 1: Domain and Target Users

Aspect	Details	Value Delivered
Domain	**Personalized Educational Technology – TOEIC Preparation** The project operates at the intersection of *educational recommender systems, language assessment, and adaptive learning*. Its core aim is to reduce content overload and provide learners with tailored question bundles and lecture materials aligned with their performance, goals and progress. By leveraging hybrid recommendation techniques (content-based filtering, collaborative filtering and deep learning), the system personalizes study pathways in the specific context of **TOEIC (Test of English for International Communication)**, a globally recognized English proficiency test for non-native speakers.	**General Value** • Bridges gap between content abundance and learner needs. • Increases efficiency and relevance of TOEIC preparation. • Supports mastery-based and adaptive learning.
Primary Target Users	**TOEIC Learners** - **Profile:** Non-native English speakers preparing for the TOEIC exam. - **Subgroups:** • *College students* seeking to improve language proficiency for academic and career opportunities. • *Working professionals and job-seekers* aiming for higher TOEIC scores to meet industry requirements or advance careers. • *Learners at multiple proficiency levels* (beginner, intermediate, advanced), requiring different learning trajectories.	• Tailored recommendations reduce time spent searching for suitable materials. • Adaptive learning paths align with learner goals and weaknesses. • Increased engagement and motivation through relevant study resources.

	- **Needs Addressed:** Personalized practice materials, adaptive progression, reduced decision fatigue, mastery-based learning support.	
Secondary Target Users	**Educators and Curriculum Designers** - **Profile:** Teachers, trainers and content creators in TOEIC preparation programs. - **Use Case:** • Access to learner performance analytics (strengths, weaknesses, progress). • Data-driven insights to adapt teaching strategies. • Alignment of practice questions and lectures with TOEIC exam structure.	• Better understanding of learner needs through data insights. • Ability to personalize teaching interventions. • More effective and targeted curriculum development.
Tertiary Target Users	**EdTech Platforms & Developers** - **Profile:** Companies, startups and researchers building intelligent tutoring systems, digital learning apps, or language training solutions. - **Use Case:** • Integration of the recommender system into existing TOEIC preparation platforms. • Leveraging the hybrid approach (TF-IDF, SVD, NCF, Sentence-BERT) for improved personalization. • Enhancing learner engagement and outcomes through adaptive recommendations.	• Competitive advantage by offering personalized TOEIC prep solutions. • Increased user retention through adaptive recommendation features. • Scalable system adaptable to other educational domains.

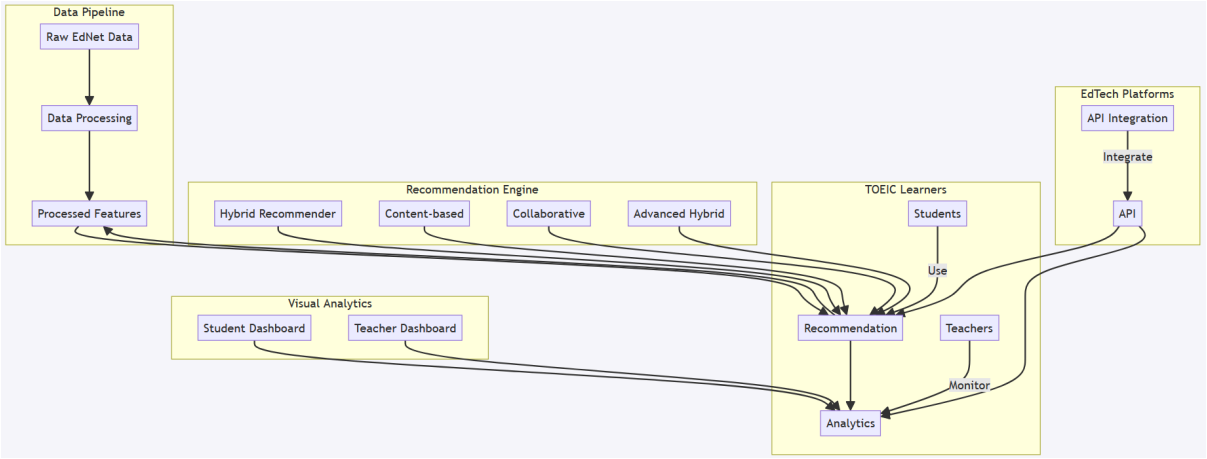


Figure 12: Domain & Target Users Flow Diagram

3.2 Design Justification

1. Hybrid Recommender Architecture (SVD + Content-Based)

User Need: Learners require recommendations that reflect their individual skill gaps while remaining robust in situations where there is limited interaction data.

Design Decision: I began with basic implementations of TF-IDF (for content similarity) and SVD (for collaborative filtering (02_tf_idf.ipynb, 03_svd_hybrid.ipynb) then combined them into a **hybrid model** using an α-weighted blending approach. The collaborative filtering captured latent user-item patterns, while content-based similarity ensured recommendations could still be generated for new bundles or users with sparse histories.

Justification: This design balanced personalization and coverage. SVD efficiently modeled behavioral interactions, but alone it struggled with cold-start learners. TF-IDF, while simpler, provided a fallback mechanism based on content overlap. The hybrid model allowed me to tune the contribution of each method, resulting in stronger overall performance and a solid MVP for later extensions like SBERT and NCF.

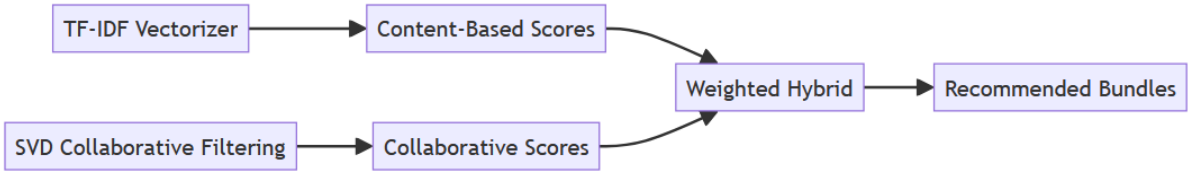


Figure 13: Hybrid Recommender Architecture Flow Diagram

2. Advanced Embedding Models (Sentence-BERT)

User Need: Learners require study materials that are not only similar in keywords but also semantically relevant and aligned with their context.

Design Decision: I first implemented TF-IDF as a baseline for content-based filtering, but later enhanced it with Sentence-BERT embeddings applied to features such as part name, subject category, and tags..

Justification: TF-IDF was useful for an MVP, but it could not capture deeper semantic similarity. SBERT improved contextual understanding, allowing the system to distinguish subtle domain-specific differences (e.g., “Grammar Intermediate” vs. “Vocabulary General”) and provide richer, more accurate recommendations.

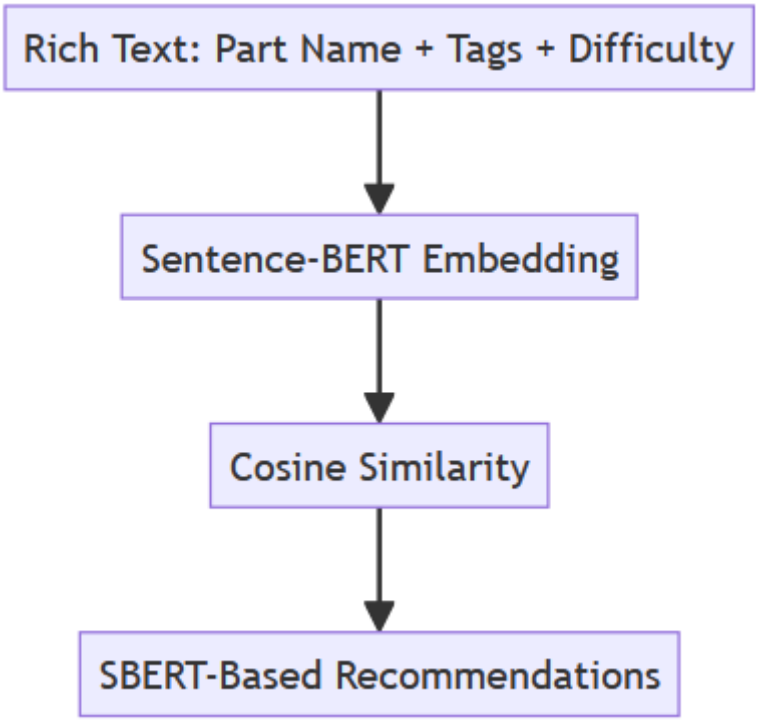


Figure 14: Sentence-BERT Semantic Matching Flow Diagram

3. Meta-Learning Ensemble for Advanced Hybrid Recommender

User Need: Learners benefit from recommendations that adapt to multiple signals, such as performance, difficulty and content similarity.

Design Decision: I started with weighted hybrid (03_svd_hybrid.ipynb) then implemented Neural Collaborative Filtering (NCF) with SBERT embeddings and combined their scores through a **meta-learner (linear regression)**.

Justification: Weighted hybrid model serves as initial implementation (MVP). Captures both deep user-item interaction (via NCF) and semantic matching (via SBERT). Neural meta-learner allows flexible scoring based on real interaction signals like success rate, part type, etc. The neural architecture can learn complex, non-linear relationships between collaborative and content signals, leading to more nuanced recommendations.

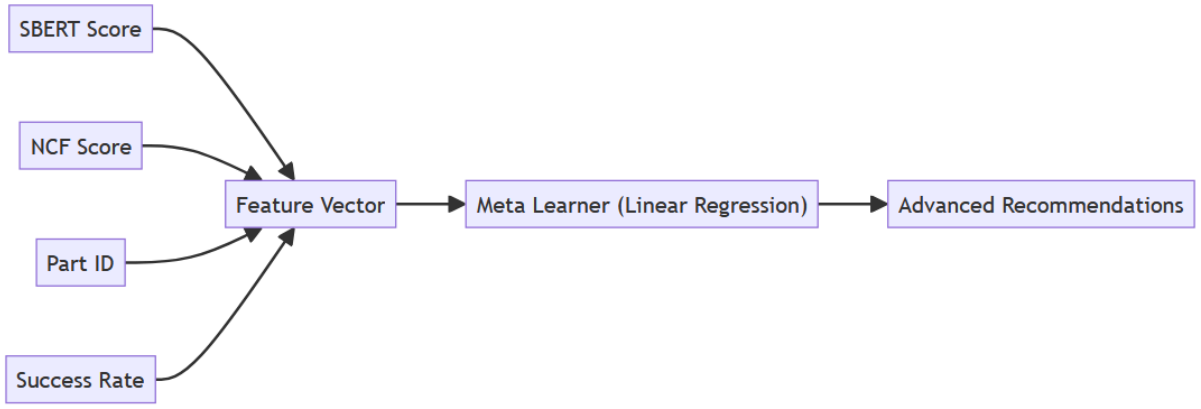


Figure 15: Meta-Learner Ensemble Model Flow Diagram

For EdNet Dataset
There are four types of EdNet datasets named KT1, KT2, KT3 and KT4 with different extents. There are five types of contents: questions, lectures, payment items, coupons and scores. Below are the datasets and contents that I will use

Table 2: Datasets used

Dataset	Purpose
EdNet-KT1 (dataset)	User interaction & performance logs
questions.csv (Content)	Metadata for content-based filtering
lectures.csv (Content)	Supplement for richer recommendations (optional)

Design justification and Curation of a high-quality dataset

1. Merged Dataset (User, Lecture, Bundle Interactions)

User Need: Tracking progress and strengths across TOEIC sections.

Design Decision: EdNet-KT1 is very large, with over 784,309 files. I couldn't include and use all the files, so I will write a script named ednet_kt1_sampler.py to randomly select 1000 files and combine them from the EdNet-KT1 dataset directory into a single DataFrame. Then I will merge this file with questions.csv as questions are directly linked to user interactions and responses, making them a natural pairing for personalized recommendations, but I will not combine lectures.csv as lectures are learning resources, not interaction-based like questions. Keeping them separate allows content-based models (TF-IDF, SBERT) to process them independently, ensuring better lecture recommendations. Overall this is about Merge logs of user_answer, correct_answer, elapsed_time and tags with bundle metadata. After sampling, 2 datasets are used throughout the project, named merged_cleaned_data.csv and cleaned_lectures.csv

Justification: Enables computation of correctness_rate, interaction_count and success_rate for bundles. Supports ‘’performance-based personalization’’; key to meaningful learning.

2. Bundle Feature Engineering

User Need: Meaningful feedback and recommendations based on question types and difficulty.

Design Decision: I engineered bundle features like part_name, subject_category and difficulty_level from raw tags and parts using functions like get_subject_category_mapping.

Justification: These features aligned the system with TOEIC pedagogy and improved explainability. For example, the system could highlight weaknesses in *"Part 5: Reading – Incomplete Sentences,"* making recommendations actionable and transparent.

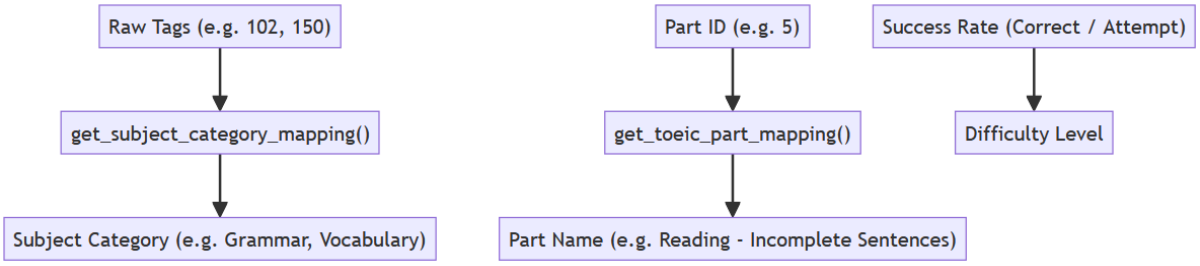


Figure 16: Bundle Feature Engineering Flow Diagram

3. User-Item Interaction Matrix with Ratings Derived from Learning Performance

User Need: Recommendations should reflect both engagement and learning quality, (not just clicks).

Design Decision: Construct ratings as:
interaction_count × (1 + 0.4 × correctness_rate)

Justification: This rewarded learners who interacted frequently *and* performed well, ensuring collaborative filtering emphasized high-quality learning signals. It promoted accuracy-driven personalization, aligning with TOEIC preparation goals.

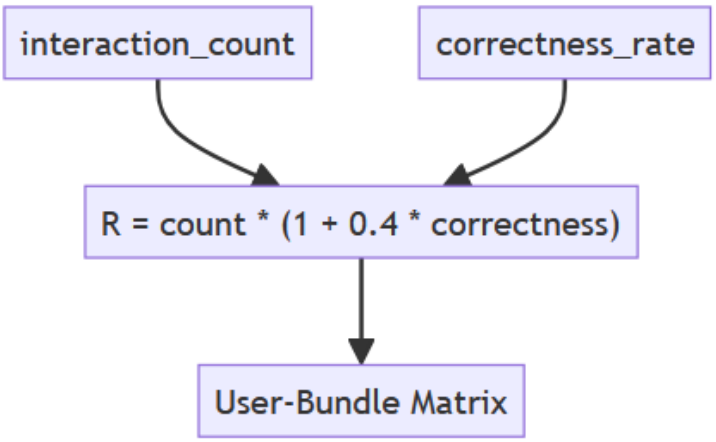


Figure 17: Interaction Matrix Design Flow Diagram

4. Mappings

User Need: Personalized learning paths for English learners

Design Decision: The dataset uses numeric IDs for users, questions and bundles to protect student privacy, so I mapped some of these identifiers to human-readable labels. Since this dataset is based on the English TOEIC exam [23], EdNet-KT1's raw identifiers, such as part_id = 2, bundle_id = b5712, and question_id = 11300, provide no insight into their actual content. To address this, I mapped each numeric part ID to its respective TOEIC section [16]. By associating part numbers with descriptions like "Part 2: Listening - Question-Response" or "Part 5: Reading - Incomplete Sentences," users can now understand which areas they are engaging with. Additionally, I categorized question tags into skill domains, such as Grammar, Vocabulary and Reading Comprehension, ensuring that each question was linked to a relevant educational focus.

Mapping was also critical in optimizing data processing. converting raw IDs into mapped indices improved retrieval efficiency. I created dictionaries for user mappings (user_map), item mappings (item_map), and reverse mappings (reverse_item_map). These transformations allowed recommendation models to process interactions with speed and accuracy, ensuring that learning paths are effectively personalized.

Justification: Mapping part IDs and tag categories serves two critical goals: (1) making recommendations interpretable for learners and educators, and (2) structuring the data for efficient matrix operations. For instance, "Part 5" maps to "Reading - Incomplete Sentences," helping users understand their weaknesses, while user_map, item_map, and reverse_item_map ensure consistent, optimized indexing during training and prediction.

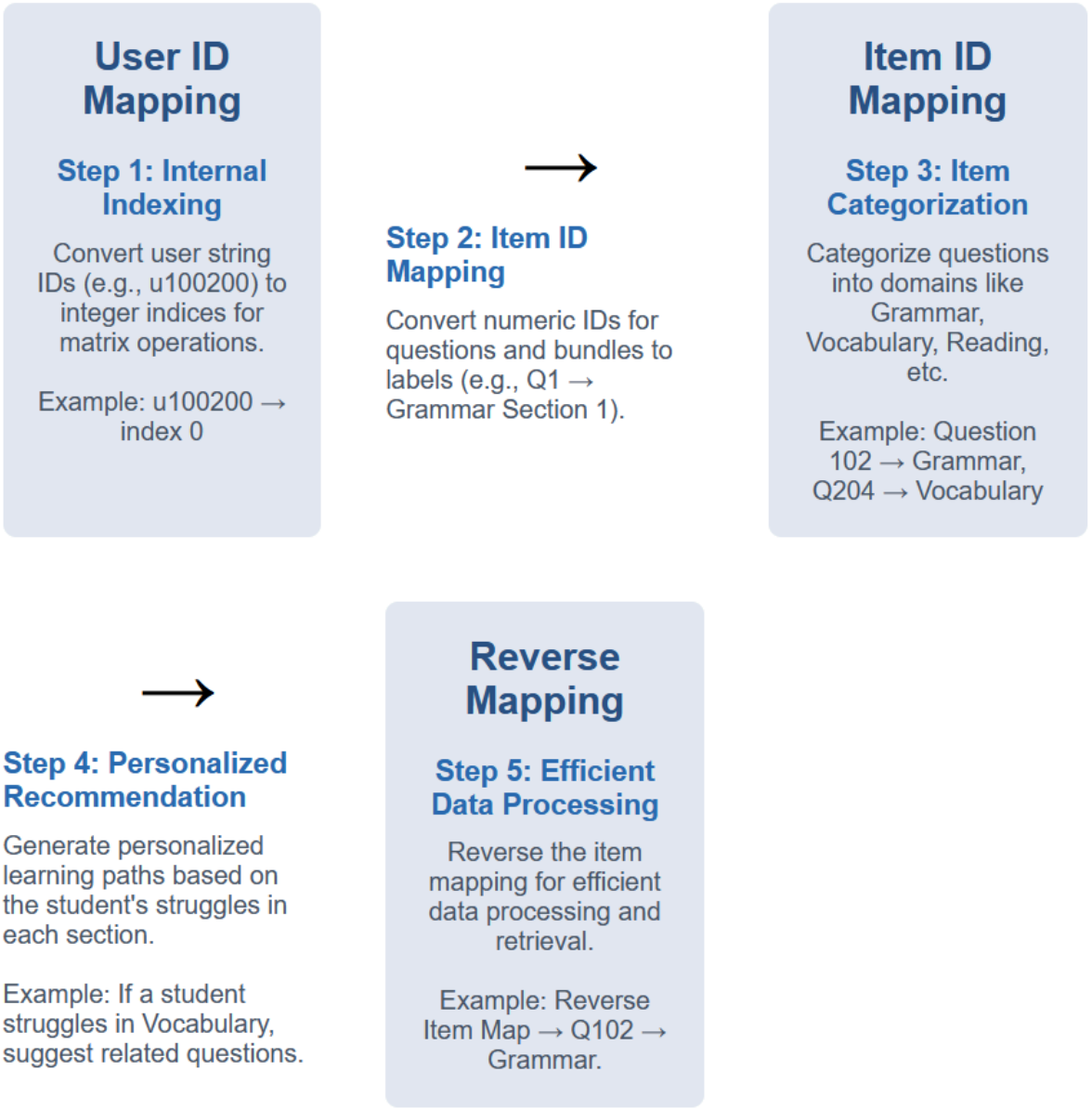


Figure 18: Design Decision Diagram of Mappings

English Test-Specific: Although, the features of EdNet are domain-agnostic [24]. Still I decided to focus on English since EdNet is based on the English TOEIC exam [16] [23]. I aim to use the existing dataset effectively, to create a unique recommendation system and to create a niche market for the English TOEIC exam. This strengthen it slightly by adding:

Specific technical challenges unique to TOEIC: The mapping system is designed to align with the TOEIC test structure by organizing content across its seven parts. It facilitates skill progression from basic to advanced grammar levels while ensuring a well-structured arrangement based on both TOEIC sections and subject categories.

How my approach will improve upon existing TOEIC preparation methods: The mappings provide a structured learning path through TOEIC sections, ensuring clear progression. It offers better content organization compared to traditional methods that rely on random practice tests, while also enabling effective progress tracking across different sections.

With this I can still meet the academic requirements and demonstrate data science techniques

3.3 Overall structure of the project

The system follows a modular architecture with clear separation of concerns:

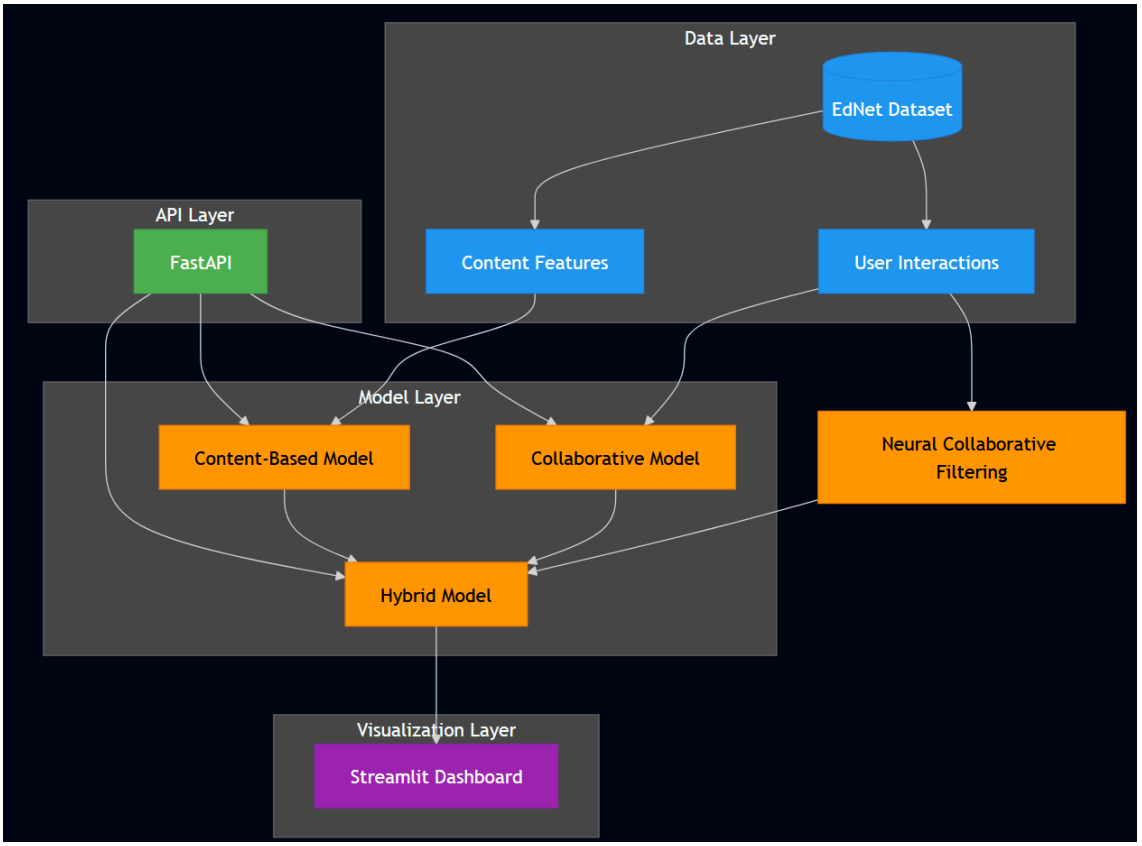


Figure 19: Architecture diagram illustrates the four main layers of the system

The system follows a modular pipeline architecture, with distinct components for data preprocessing, feature engineering, model training, recommendation generation, and frontend deployment. This separation of concerns allowed for iterative development and testing of individual modules, ensuring robustness and reproducibility.

- **Data Layer**: The EdNet-KT1 dataset was sampled and merged using a custom script (ednet_kt1_sampler.py) to create a manageable subset of user interaction logs. These were joined with questions.csv to form a rich user-item interaction matrix, while lectures.csv was processed separately for content-based recommendations. Datasets were also then preprocessed.
- **Feature Engineering**: Raw tags and metadata were mapped to TOEIC-specific categories using functions like get_subject_category_mapping, enabling explainable feedback such as “You’re weak in Part 5: Reading – Incomplete Sentences.” Ratings were derived from performance-weighted formulas to reflect meaningful engagement.
- **Model Layer**: The system integrates multiple recommendation algorithms:
    - **TF-IDF** for content similarity
    - **SVD** for collaborative filtering based on latent factors

- **Sentence-BERT** for semantic embedding and contextual matching
- **Neural Collaborative Filtering (NCF)** for non-linear user-item modeling
- **Meta-Learner** (linear regression) to ensemble the above models based on validation performance

Each model was implemented in a dedicated notebook (02_tf_idf.ipynb, 03_svd_hybrid.ipynb, 04_sbert_embeddings.ipynb, 05_advanced_recommendations.ipynb) and evaluated using metrics such as RMSE, Precision@10 and Recall@10.

Figure 20: Directory's structure of Project

The **project codebase** is organized following good software engineering practices. It separates concerns across modules as seen in Figure 20. The project is designed with

reproducibility and modularity in mind. Every model, whether TF-IDF, SVD hybrid, or NCF will be saved into the models/ directory and versioned via pickle files.

The code in the project is also organized with comments, but I've kept them to a minimum as excessive comments can make the code look messy.

For EdNet Dataset

Table 3: Ednet Content files

File	Content Type	Key Fields
questions.csv	Questions	question_id, bundle_id, tags, part, correct_answer
lectures.csv	Lectures	lecture_id, tags, video_length, deployed_at

Table 4: Interactions (KT1)

Field	Type	Purpose
timestamp	int (ms)	Order of learning events
solving_id	int	Session tracking (pseudonymous grouping)
question_id	str	Links to content
user_answer	char (a–d)	Learner response
elapsed_time	int (ms)	Time spent per question

Entity Relationships

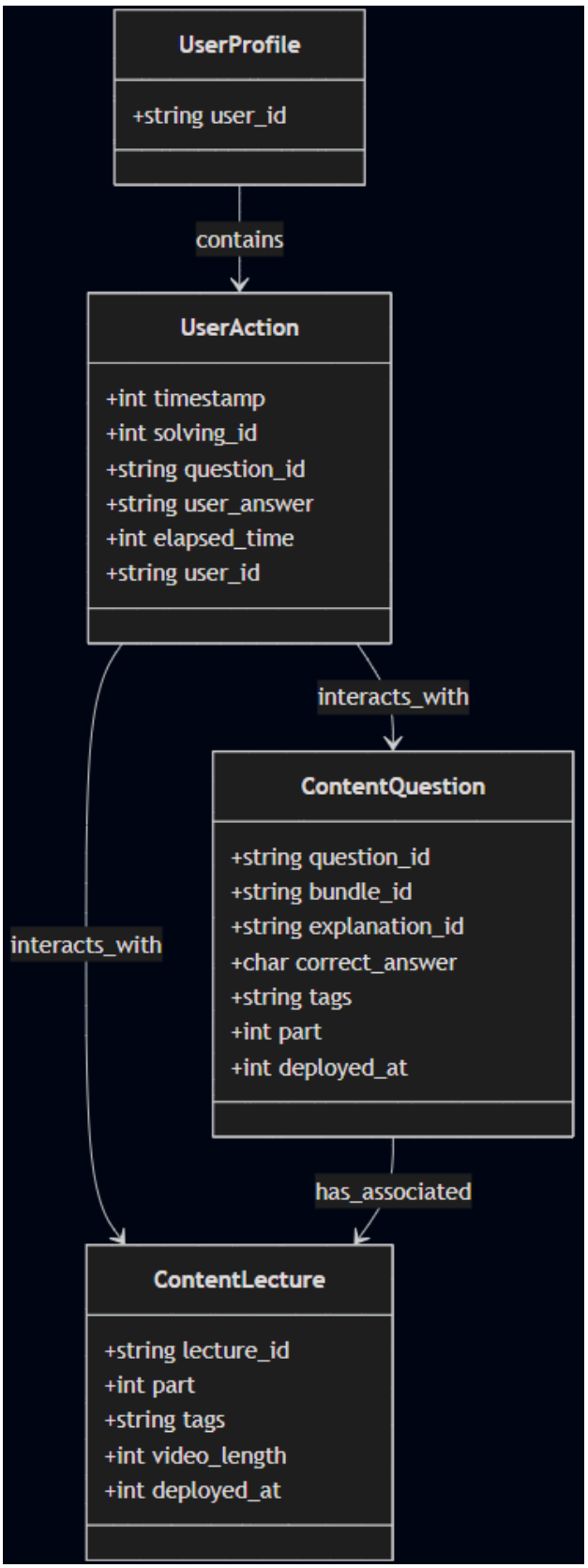


Figure 21: Schema Diagram

Key Points in the above Schema

UserProfile: This entity contains basic information about the user, specifically their unique identifier (user_id). It is linked with UserAction, which represents individual actions performed by the user.

UserAction: Tracks all actions performed by a user, including interactions with questions and lectures. Actions are timestamp and include user responses, question identifiers and time spent (elapsed_time).

ContentQuestion: Represents all the content related to questions the student interacts with. Each question is linked to a specific bundle and has tags, which help in understanding the educational context of the question. Questions also include their correct answers and deployment information.

ContentLecture: Contains all the lectures available to students. Each lecture is associated with a specific TOEIC part and includes tags and video length information for content analysis. Lectures also have deployment timestamps for tracking availability.

Table 5: Logical Entities Summary

Entity	Primary Key	Relationships
Questions	question_id	bundle_id, tags
Lectures	lecture_id	tags
Interactions	(none)	question_id, bundle_id
Tags	tag_id	Shared across questions and lectures

3.4 Key Technologies and Methods

Table 6: Python Version Used [25]

Python Version	Status	Reason
Python 3.10	Recommended	Stable support for RecSys libraries like surprise, lightfm, implicit, a lot of support and tutorials available
(Why not use the latest?) Python 3.13	Not Recommended	Most libraries not yet compatible; breaks installation. Ideal only for exploration and hobby projects

Table 7: Why I Used Virtual Environment (`.venv`) with Jupyter [26]

Feature	Anaconda Base Env	Linked `.venv` Kernel
Same Python version as my project	Not guaranteed	Yes same version
Uses same pip-installed packages	No	Yes
Avoid dependency conflicts	Risky	Isolated
Portable + reproducible	Hard to share	Easy with `requirements.txt`

Key Libraries

Table 8: Machine Learning & Recommender Algorithms

Library	Purpose
scikit-learn (sklearn)	TF-IDF vectorization, cosine similarity, model selection, evaluation metrics
scipy.sparse.linalg	SVD matrix factorization (collaborative filtering)
sentence-transformers	BERT-based sentence embeddings for semantic similarity
torch (PyTorch)	Building and training Neural Collaborative Filtering (NCF) models
tensorflow / tf-keras	Included for compatibility in deep learning workflows (not central to pipeline)

Table 9: Data Processing & Manipulation

Library	Purpose

pandas	Data manipulation, aggregation and I/O
numpy	Numerical computations and matrix operations
tqdm	Progress bars for iterative processes

Table 10: Evaluation Metrics & Visualization

Library	Purpose
sklearn.metrics	Evaluation metrics: RMSE, precision, recall, AUC
matplotlib.pyplot	Visualization of model tuning results and performance comparisons
seaborn	Enhanced styling for plots and statistical graphics
plotly	Interactive visualizations and diagnostic dashboards

Table 11: Persistence & File I/O

Library	Purpose
pickle	Saving/loading trained models and precomputed matrices
os / sys	File path handling and system path management

Table 12: Text Processing

Library	Purpose
re	Regular expressions for text preprocessing
sklearn.feature_extraction.text.TfidfVectorizer	Vectorization of text content using TF-IDF

Table 13: User Interface

Library	Purpose
streamlit	Web-based interface for learner interaction and dashboards

Table 14: API Frameworks

Library	Purpose
fastapi	REST API framework for serving recommendations
uvicorn	ASGI server for running the FastAPI backend

Table 15: Testing & Quality Assurance

Library	Purpose
pytest	Unit testing framework
pytest-cov	Test coverage measurement

pytest-mock	Mocking external dependencies
coverage	Generating detailed coverage reports

Table 16: Development & Documentation Tools

Library	Purpose
black	Code formatting
flake8	Linting for code style and errors
mypy	Static type checking
isort	Consistent import ordering
pre-commit	Git hooks for automated code quality checks
sphinx	Project documentation generation
sphinx-rtd-theme	ReadTheDocs theme for documentation

3.5 Work Plan and Gantt Chart

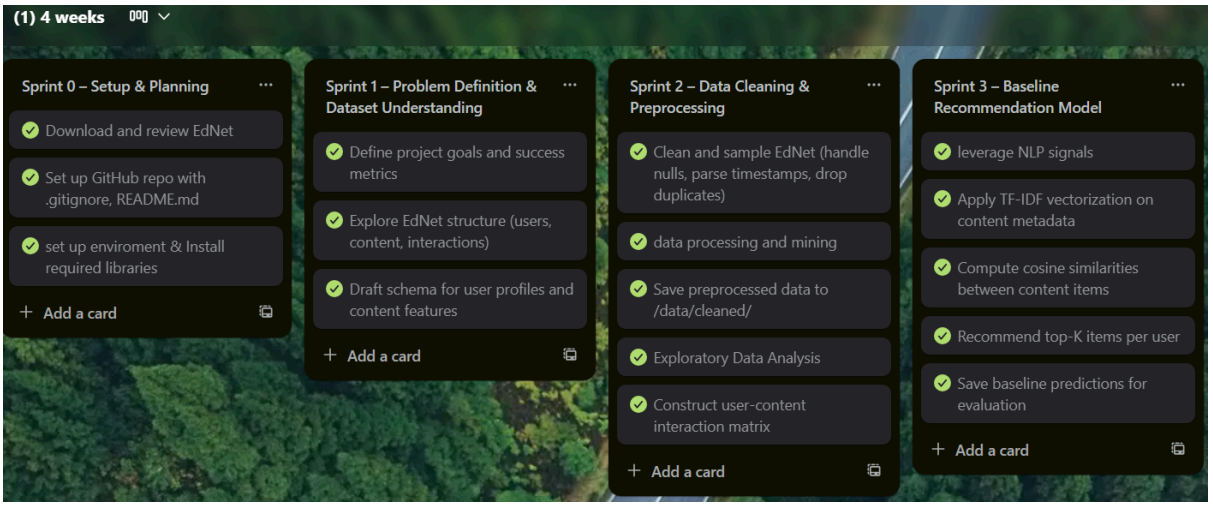


Figure 22: Trello Board 1

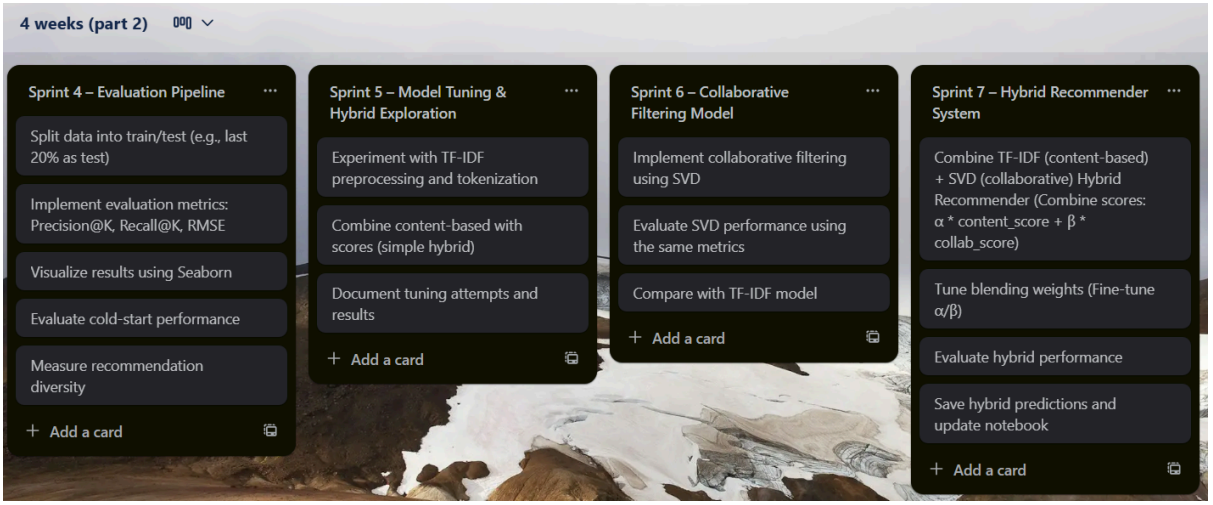


Figure 23: Trello Board 2

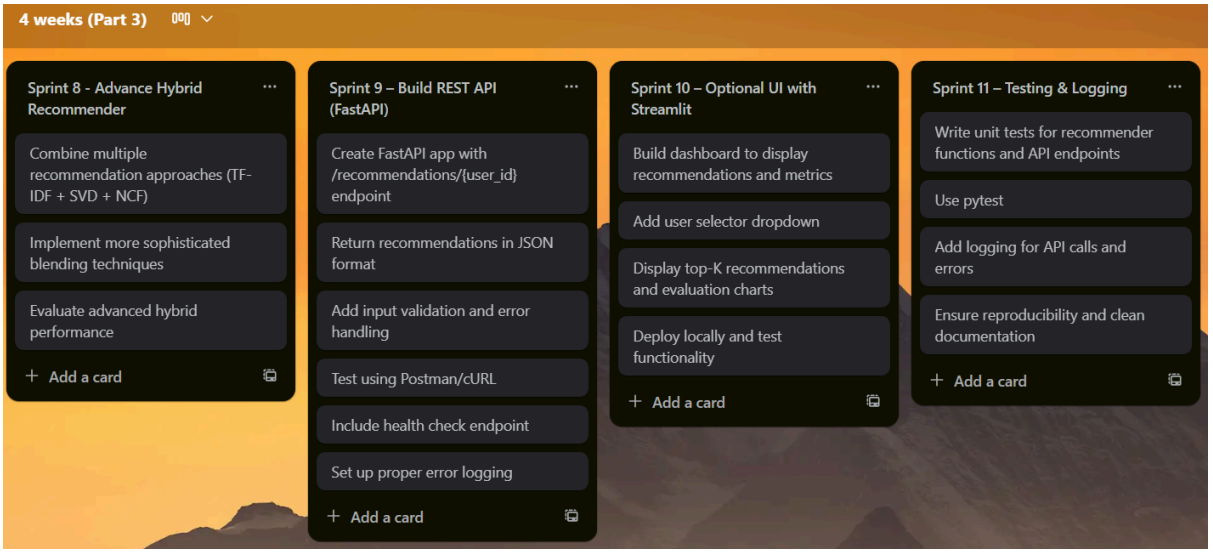


Figure 24: Trello Board 3

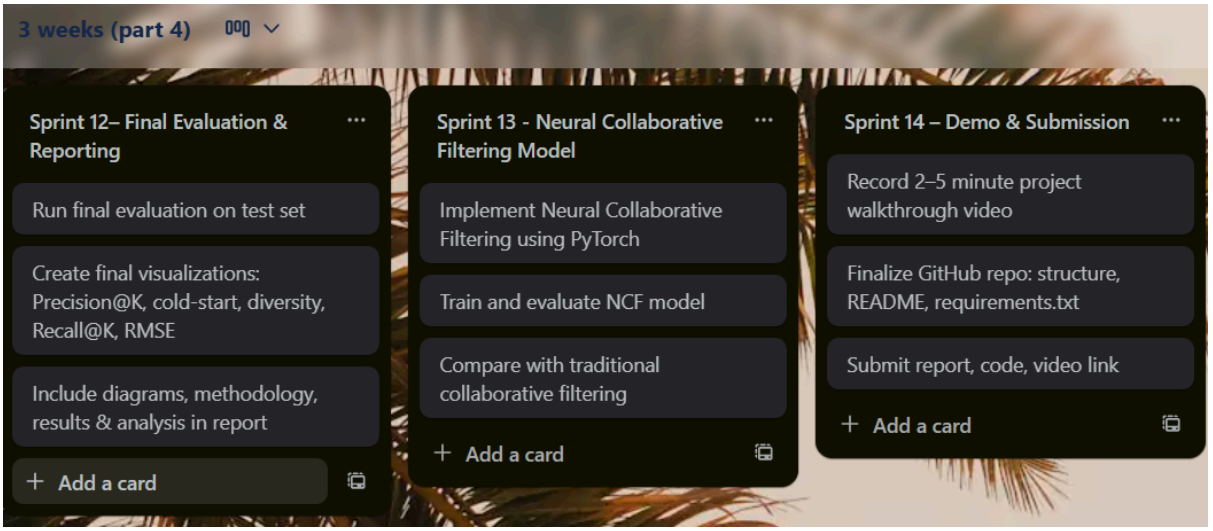


Figure 25: Trello Board 4

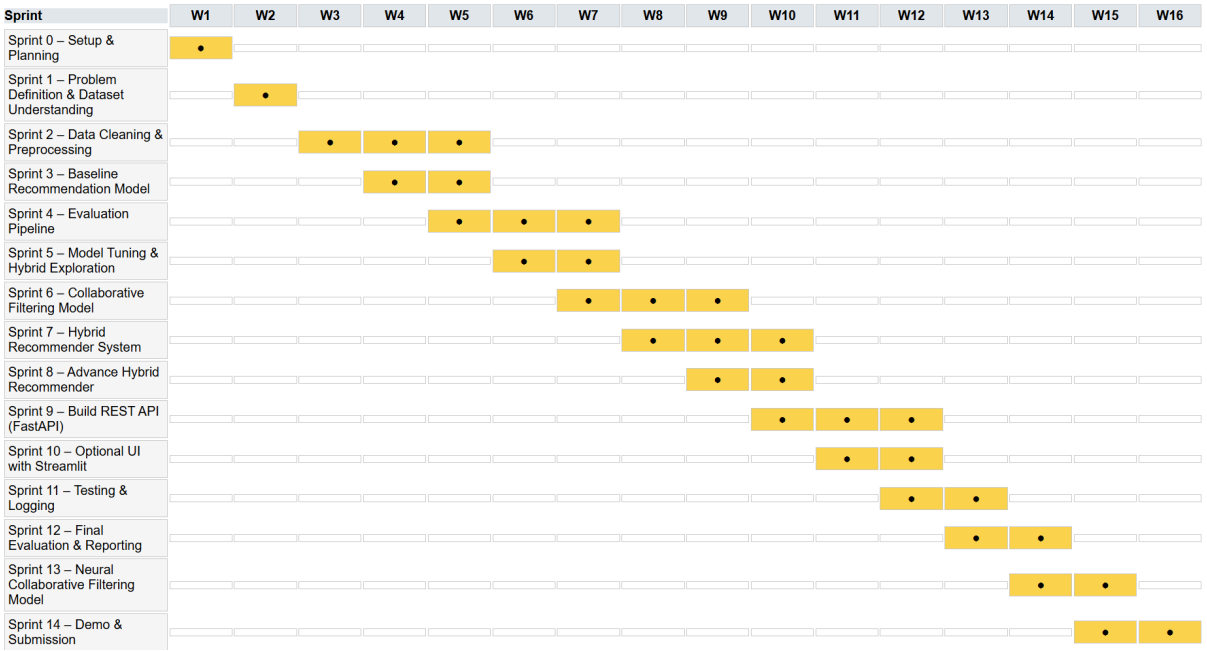


Figure 26: Gantt Chart of 15-Sprints Project Plan

Project Feasibility and Resources

As you can see in the figures, I have 4 Trello boards and each Trello board has sprints numbered from 0 to 14. Each sprint contains tasks broken down into smaller chunks, which I can easily dedicate time to. My original plan was to complete each sprint in one week (which would have taken 15 weeks in total). However, after completing half of Trello board 1, I realized that this approach was unrealistic. This is why I created a Gantt chart, where I allocated more weeks to sprints that seemed harder or contained longer tasks, such as sprint 4 and sprint 9. I also added one more week to the chart, making it a total of 16 weeks.

Dividing the plan into Trello boards, sprints and small chunks, and then laying out the sprints on the Gantt chart, allowed me to work more feasibly. As a result, I was able to finish the remainder of Trello board 1 easily. Completing this project in 16 weeks, rather than the full 22 weeks allocated in the final year module, will give me extra time to check for bugs and improvements.

For resources, I have access to Slack project channels, online resources, the teaching center, and guidance from others. I have mostly made a plan for the project, with only a little planning for the report. For the report, I am following along with the 22-week final year project module, watching the videos and participating in peer reviews while also receiving feedback from others.

I am still learning about this project, as you can see in this report. I have used a lot of visual aids like diagrams, tables and graphs etc. They may not be perfect or completely accurate, but they are part of my learning process. As a visual learner, I believe that visual aids and pictures can capture what words cannot. I find visual aids from online resources and some I create myself using tools like Mermaid, Graphviz, Jupyter Notebook and Figma, which I am familiar with. Additionally, there are some aspects of my project design and feature prototype that I have explained in great detail in paragraphs because I understand them well. For

topics that I am still learning or don't fully understand, I explain them briefly, either in small paragraphs or even bullet points.

The reason I am sharing my journey in detail is to show how our learning and experiences influence the plan and the project. A plan is not always feasible, as I experienced with Trello board 1, but learning and experience are always feasible (achievable).

3.6 Testing and Evaluation Strategy

Table 17: Success Metrics

Metric	Use Case
Precision@K / Recall@K	Measure recommendation relevance (used when recommending next questions) [27]
AUC (ROC Curve)	For binary predictions (e.g., will the user answer correctly?) [28]
RMSE / MAE	For predicting scores or probabilities of success [29]
Engagement Metrics	CTR, time spent and completion rate on recommended content [30]
Qualitative Review	Top-N bundle recommendations reviewed for semantic relevance

Table 18: Evaluation Plan

Evaluation Aspect	Details
Cross-validation	Perform cross-validation on user-bundle interactions.
Model Comparison	Compare models: TF-IDF, SVD, SBERT, NCF, Advanced Hybrid.
Success Criteria	If hybrid or NCF models show > 0.02 precision@10 on sparse data, the project is considered successful.
Deployment Readiness	Models saved in /models/. Recommenders callable from the API layer.

Additional Testing	A/B testing, user feedback or user studies in a deployed setting. Or implement unit test using pytest or tox

Modular design ensures reusability and extensibility to new educational domains or test formats.

Chapter 4: Implementation (2494/2500 words)

4.0 Before Prototype

Table 19: Streamlit App Development Phases

Phase	Description
Early Internal Version	During development, I created an early internal version of the Streamlit app (see Figure 27). This version was not intended as the prototype but rather as a proof-of-concept to validate the end-to-end pipeline: • verifying that the API endpoints correctly returned recommendations • checking that the models integrated smoothly • ensuring that recommendation scores were passed through to the interface.
Debugging View	At this EARLY stage, recommended items were displayed only as bundle_ids with raw scores. While this view was functional for debugging, it was not user-friendly and was never meant to be presented as the prototype.
Actual Prototype & Implementation	The actual prototype and implementation (Read through this Chapter) was designed from the start to include: • mappings of bundle_ids to user-friendly lecture and bundle names, • friendly visualizations (can be understood by anyone), • clear recommendation labels (e.g., lecture title, part number), • and insights for educators and learners.
Project Alignment	This aligns with my original plan, where the project had to demonstrate not just technical correctness but also usability & friendly visualizations for non-technical stakeholders.

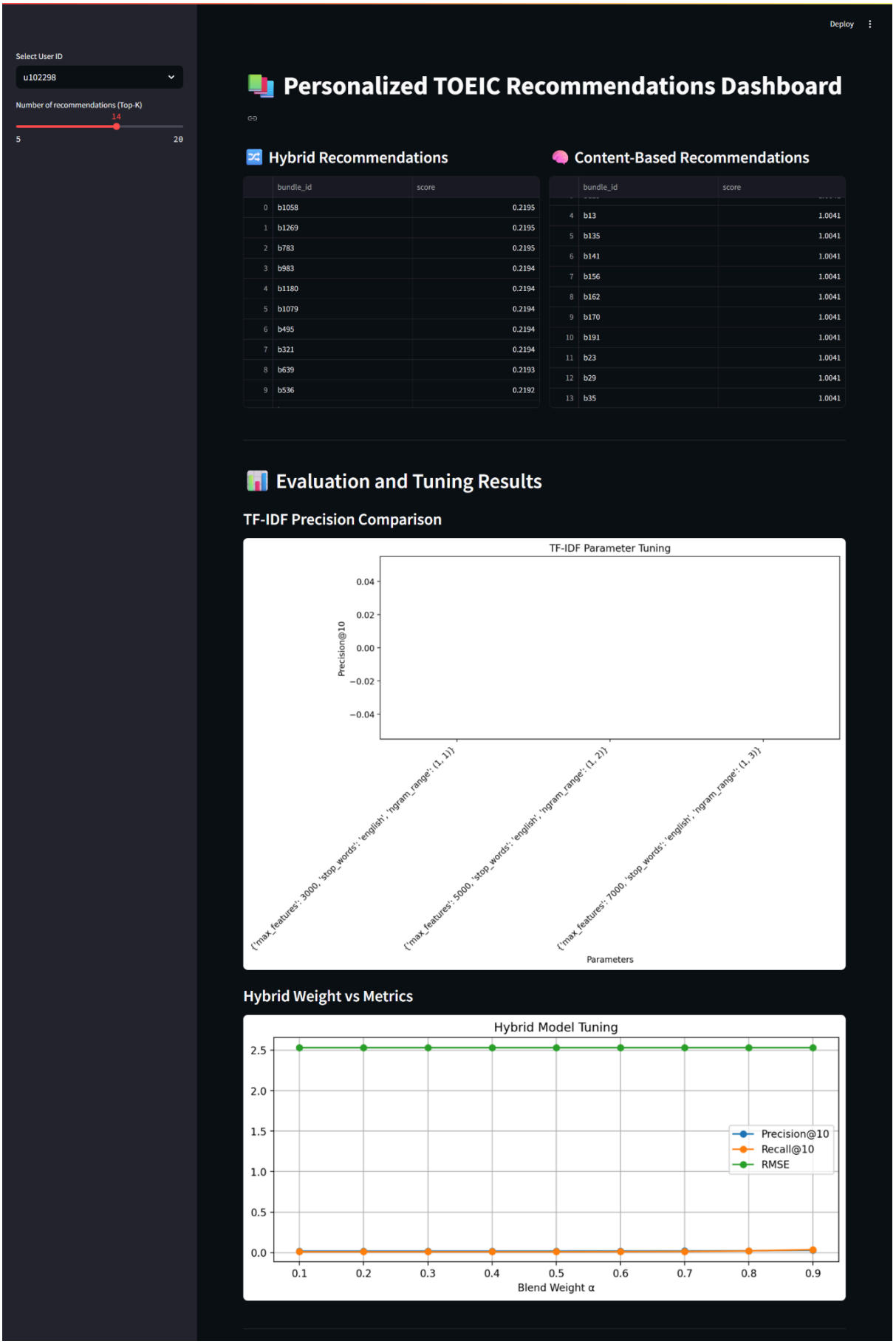


Figure 27: Early internal version of the app

PART 1: Prototype

4.1 Introduction

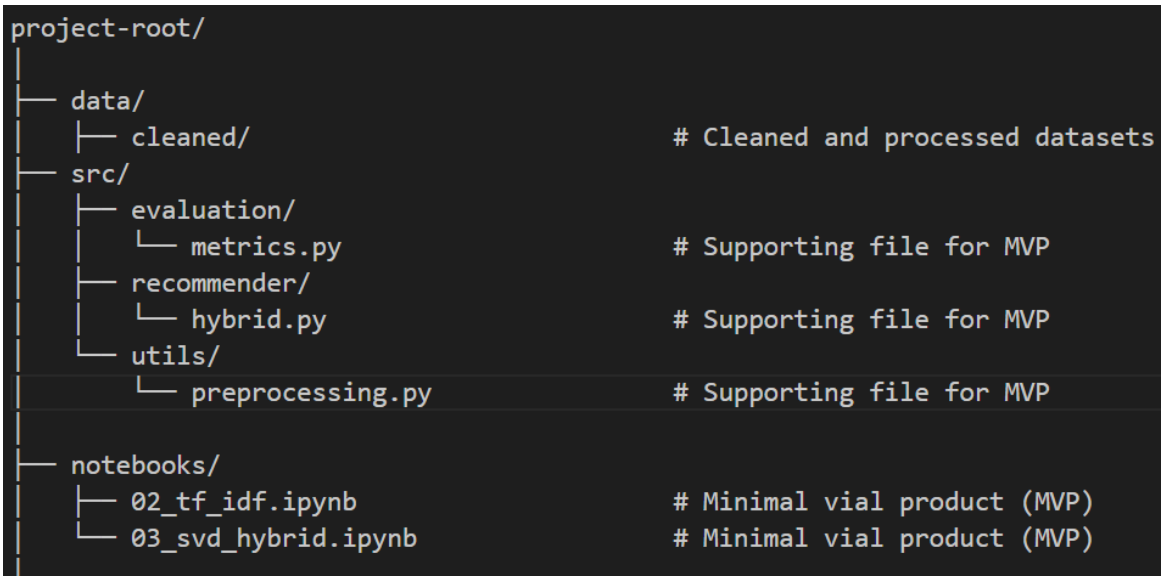


Figure 28: Directory Structure for Prototype

The feature prototype, primarily implemented in 03_svd_hybrid.ipynb, tests this hybrid model, representing the project's core innovation. Initially, 02_tf_idf.ipynb, a content-based TF-IDF model, was considered but deemed a weak prototype due to its poor performance, serving instead as a baseline. Supported by hybrid.py, metrics.py and preprocessing.py, the prototype is evaluated using standard machine learning metrics. This part 1 details the prototype's functionality, performance and proposed improvements, aligned with recommender system best practices. My main goal is to test the mappings, recommendations on the chosen datasets and ensure the models work as expected.

Before implementing the prototype, sampling the EdNet dataset was essential due to its large size. The implementation plan has already been discussed in the **previous chapter**.

```python
import pandas as pd # For working with structured data (DataFrames)
import random # For selecting files randomly
from pathlib import Path # For handling file paths in a platform-independent way

def sample_kt1_logs(folder_path: str, sample_size: int = 1000) -> pd.DataFrame:
    """
    Randomly sample user logs from the EdNet-KT1 directory and combine them into one DataFrame.

    Args:
        folder_path (str): Path to the folder containing KT1 user .csv logs.
        sample_size (int): Number of user logs to sample.

    Returns:
        pd.DataFrame: Combined DataFrame of sampled logs.
    """

    # Convert the folder path to a Path object for easy file handling
    path = Path(folder_path)
    # Find all CSV files that match the pattern 'u*.csv' (e.g., user logs)
    all_files = list(path.glob('u*.csv'))

    # Check if the requested sample size exceeds available files
    if sample_size > len(all_files):
        # Raise an error if sample size is too large
        raise ValueError("Sample size exceeds available files")

    # Randomly select 'sample_size' number of files from the available logs
    sampled_files = random.sample(all_files, sample_size)

    # Initialize an empty list to store individual DataFrames
    dataframes = []

    for file in sampled_files:
        # Read each sampled CSV file into a DataFrame
        df = pd.read_csv(file)
        # Extract the user ID from the filename (e.g., u123456) and add as a new column
        df['user_id'] = file.stem
        # Append the processed DataFrame to the list
        dataframes.append(df)

    # Merge all sampled DataFrames into a single DataFrame
    combined_df = pd.concat(dataframes, ignore_index=True)
    return combined_df  # Return the final combined DataFrame


# Sample KT1 dataset & save as 'sampled_kt1_logs.csv'
# Call the function to sample logs
sampled_df = sample_kt1_logs("./KT1", sample_size=1000)
# Save the sampled data to a CSV file without index
sampled_df.to_csv("sampled_kt1_logs.csv", index=False)
```

Figure 29: Implementation for Sampling Dataset in ednet_kt1_sampler.py

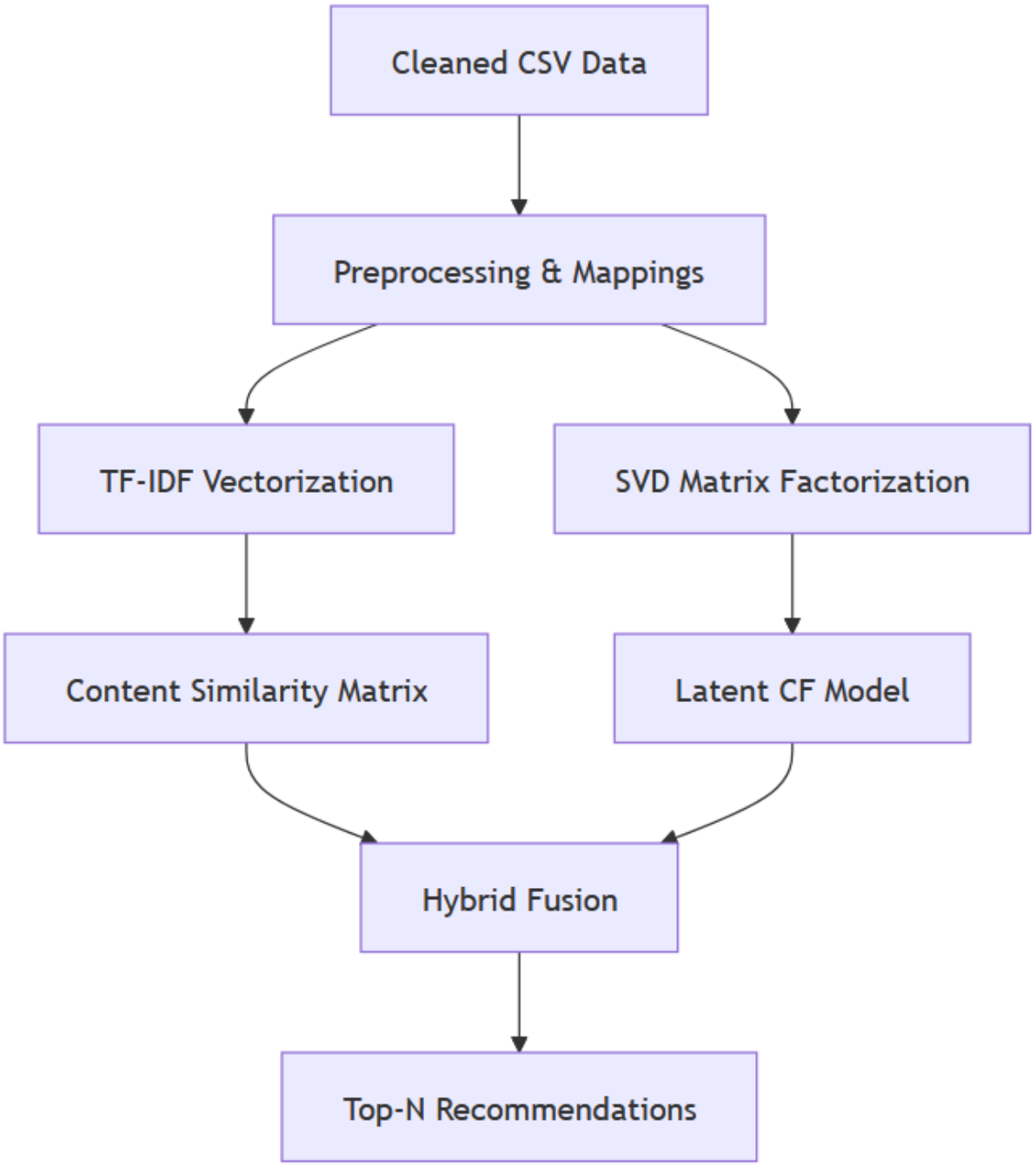


Figure 30: Shows system Architecture of Notebooks

4.2 Background

Recommender systems are pivotal in educational platforms, personalizing content to improve user engagement. Content-based filtering uses item metadata (e.g., tags, categories) to suggest similar items, while collaborative filtering infers preferences from user interactions [31]. Hybrid systems mitigate limitations like cold-start and data sparsity by combining both approaches [32].

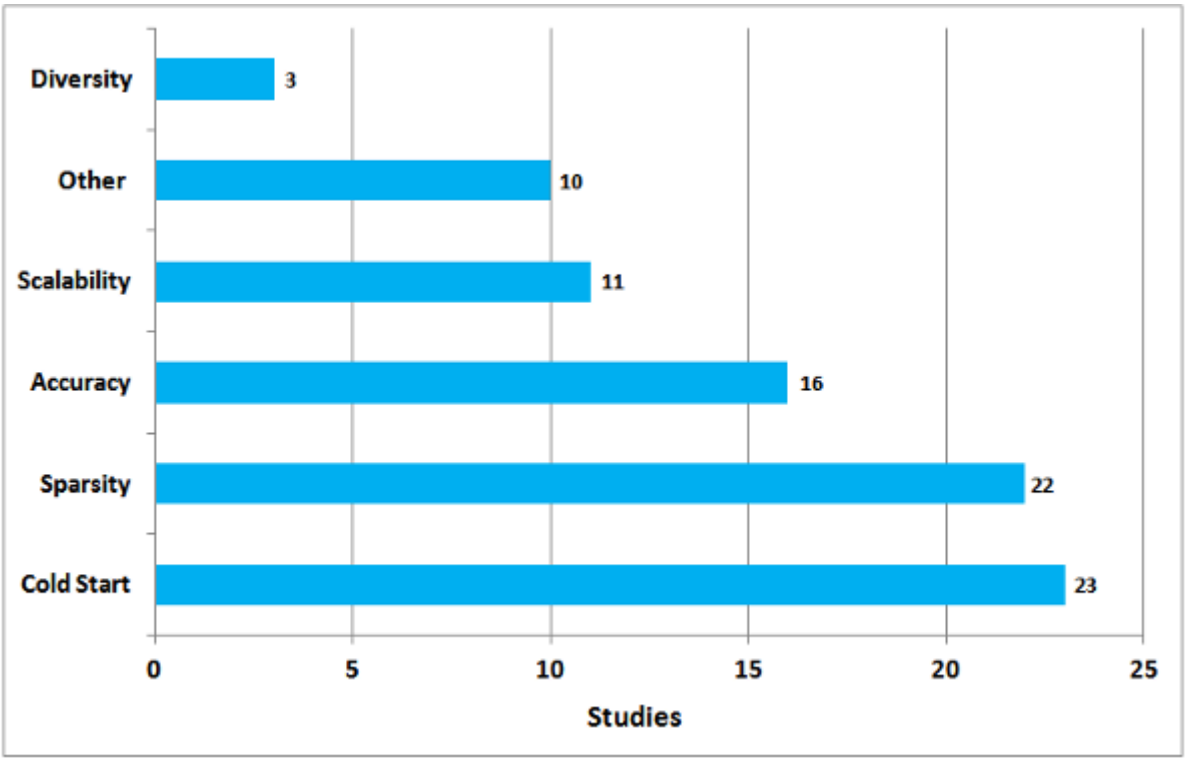


Figure 31: Problems addressed in Recommendation System

For TOEIC preparation, where users interact with bundles across listening, reading and grammar skills, a hybrid model captures both content relevance and behavioral patterns. The prototype implements TF-IDF for content similarity and SVD for user-item interactions, evaluated with precision@k, recall@k and RMSE to align with industry standards.

4.3 Methodology

The project utilizes datasets of user interactions (merged_cleaned_data.csv) and lecture metadata (cleaned_lectures.csv), preprocessed into user-item matrices and content feature vectors.

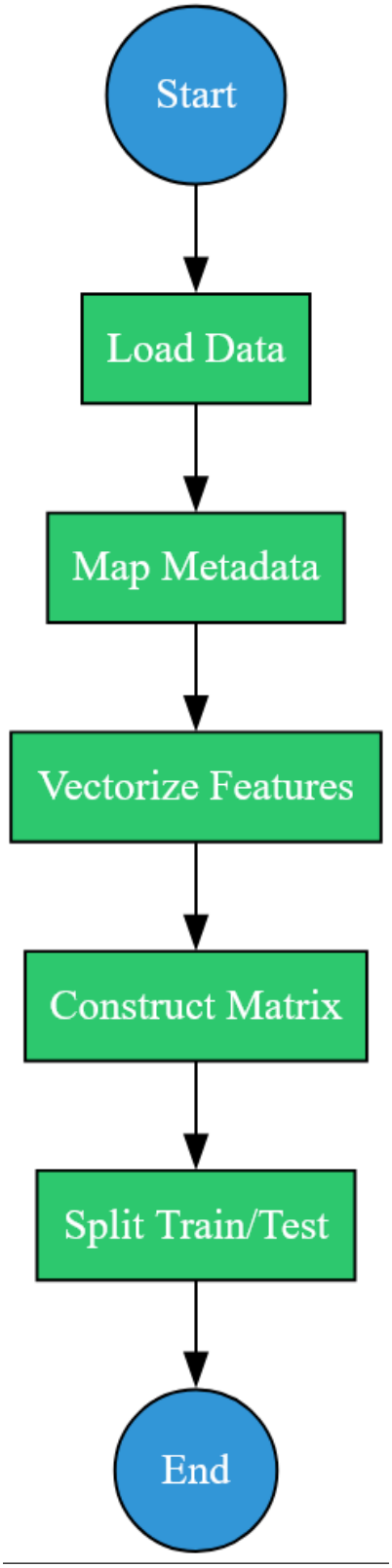


Figure 32: Flowchart depicting the data preprocessing pipeline: starting from loading raw data to splitting it for model training and testing.

The 02_tf_idf.ipynb notebook builds a content-based model, vectorizing bundle features (part, tags, subject categories) with TF-IDF and using cosine similarity for recommendations.

The 03_svd_hybrid.ipynb notebook implements a hybrid model, blending SVD-based collaborative filtering with TF-IDF content similarity via a tunable weight.

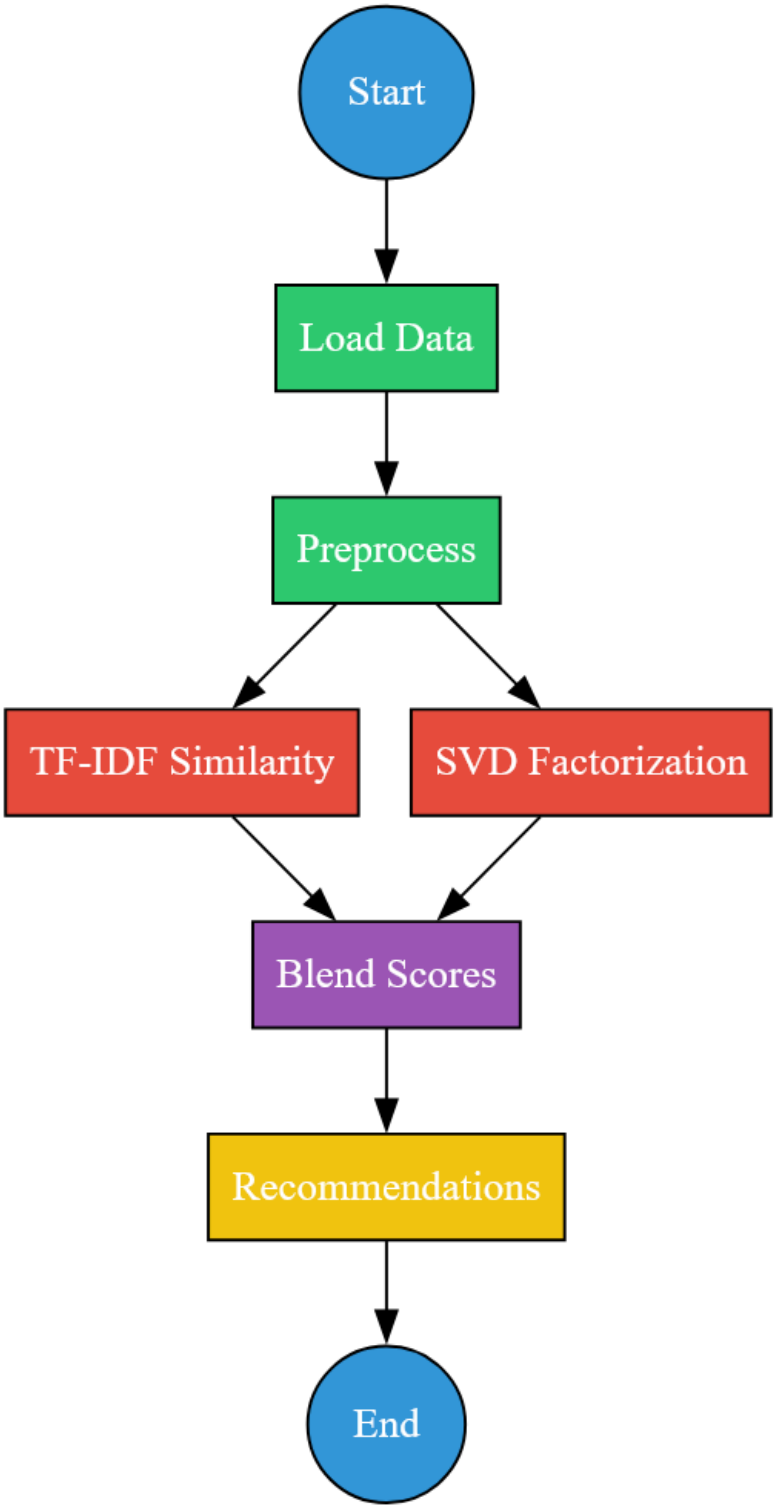


Figure 33: Flowchart of the recommendation process in the hybrid model.

4.4 Description

The feature prototype, centered in 03_svd_hybrid.ipynb, tests a hybrid recommendation system for TOEIC question bundles, integrating collaborative and content-based filtering.

The SVDHybridRecommender class in src/recommender/hybrid.py uses SVD to model user-item interactions and TF-IDF-based content similarity to enhance recommendations. Data is preprocessed using src/utils/preprocessing.py to create a sparse user-item matrix, while src/evaluation/metrics.py provides evaluation functions.

The hybrid model blends scores with a weight of 0.7, prioritizing collaborative signals.

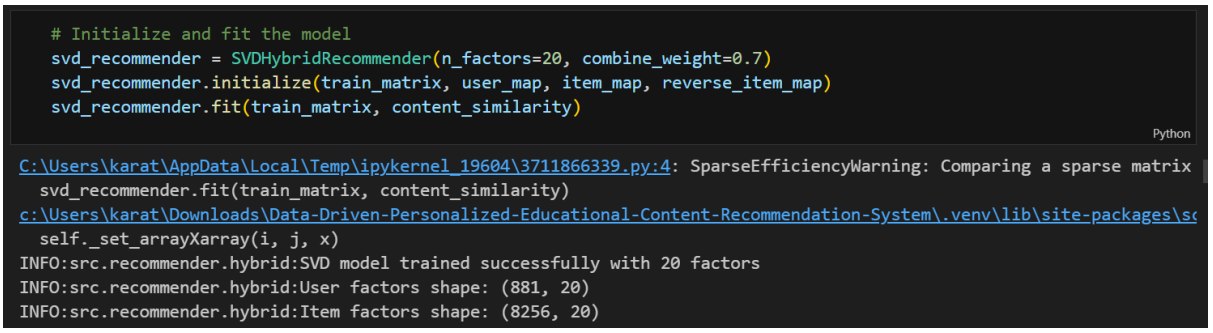


Figure 34: Shows how hybrid model blends scores with a weight of 0.7

In contrast, 02_tf_idf.ipynb implements a content-based TF-IDF model, which, due to its zero precision@10 and recall@10, is a weak prototype, limited by its inability to leverage user interactions and serves as a baseline.

4.5 Prototype Demonstration

The prototype begins by preprocessing user interactions and bundle metadata using prepare_matrices and preprocess_content_text from preprocessing.py. Bundle features (part, tags, subject categories) are mapped into categories like “Grammar Intermediate” or “Part 7: Reading Comprehension”, as shown *below*.

```python
# Create subject categories based on ranges of tag values
# These mappings are based on TOEIC structure
def get_subject_category(tag):
    try:
        tag_val = float(tag)
        if 1 <= tag_val < 23:
            return "Listening Skills"
        elif 23 <= tag_val < 52:
            return "Reading Skills"
        elif 52 <= tag_val < 70:
            return "Speaking Skills"
        elif 70 <= tag_val < 150:
            return "Writing Skills"
        elif 150 <= tag_val < 200:
            return "Test Preparation"
        elif 200 <= tag_val < 300:
            return "Grammar & Vocabulary"
        else:
            return "General"
    except:
        return "General"

lectures_data['subject_category'] = lectures_data['tags'].apply(get_subject_category)

# Map part numbers to human-readable names
part_names = {
    0: "Introduction",
    1: "Listening Comprehension",
    2: "Reading Comprehension",
    3: "Grammar & Vocabulary",
    4: "Speaking Assessment",
    5: "Writing Exercises",
    6: "Practice Tests",
    7: "Additional Resources"
}

lectures_data['part_name'] = lectures_data['part'].map(part_names)

# Display the processed lectures data
print("Processed lectures data:")
lectures_data[['lecture_id', 'part', 'part_name', 'tags', 'subject_category', 'video_length']].head()
```

Figure 35: Shows how mappings is applied

```
Processed lectures data:
```

	lecture_id	part	part_name	tags	subject_category	video_length
0	l520	5.0	Writing Exercises	142.0	Writing Skills	NaN
1	l592	6.0	Practice Tests	142.0	Writing Skills	NaN
2	l1259	1.0	Listening Comprehension	222.0	Grammar & Vocabulary	359000.0
3	l1260	1.0	Listening Comprehension	220.0	Grammar & Vocabulary	487000.0
4	l1261	1.0	Listening Comprehension	221.0	Grammar & Vocabulary	441000.0

Figure 36: Shows how mappings look like on dataset

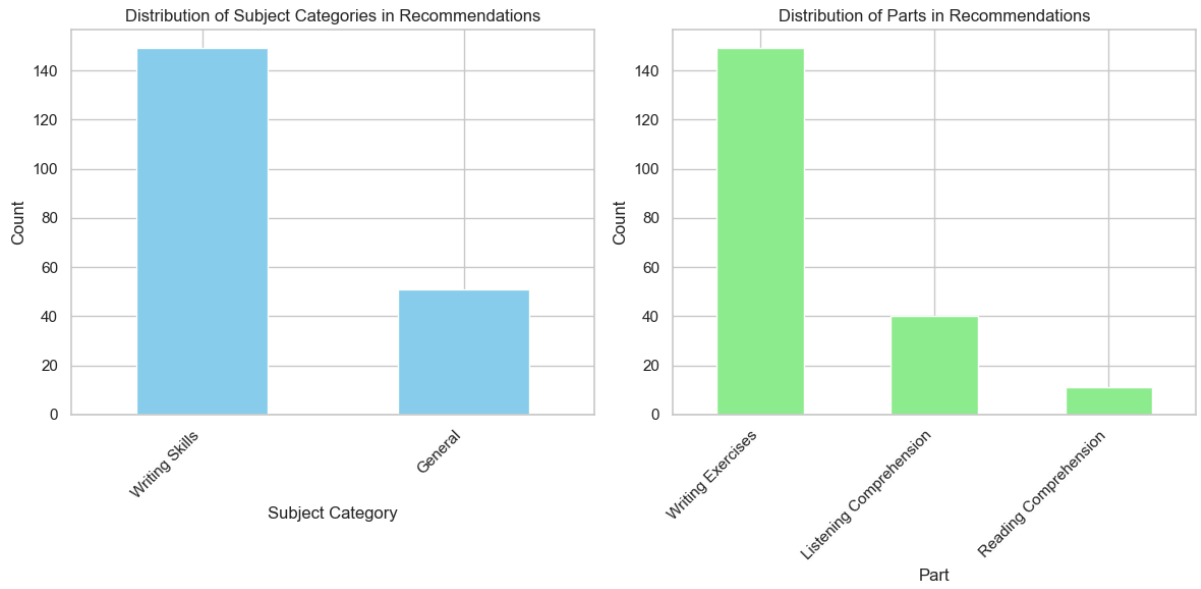


Figure 37: Shows how mappings are spread across recommendations

In 02_tf_idf.ipynb, these features are vectorized with TfidfVectorizer from sklearn and cosine similarity generates recommendations based on user history, but results are irrelevant due to the lack of user interaction context.

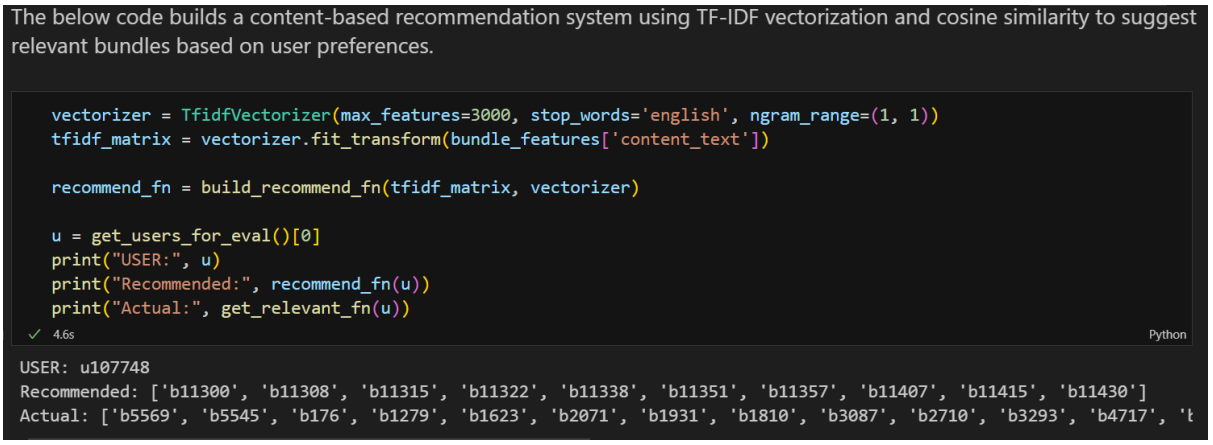


Figure 38: Shows poor recommendations in 02_tf_idf.ipynb, barely matching with actual ones

In 03_svd_hybrid.ipynb, a sparse user-item matrix (98.92% sparsity) is constructed, with ratings weighted by interaction count and correctness rate.

```python
# Create correctness as a measure of interaction quality
filtered_data = filtered_data.copy()  # Ensures modifications don't affect original data
filtered_data.loc[:, 'correct'] = (filtered_data['user_answer'] == filtered_data['correct_answer']).astype(int)

# Aggregate user-item interactions
user_item_data = filtered_data.groupby(['user_id', 'bundle_id']).agg(
    correctness_rate=('correct', 'mean'),
    interaction_count=('correct', 'count'),
    avg_time=('elapsed_time', 'mean'),
    part=('part', 'first'),
    tags=('tags', lambda x: ';'.join(set(str(i) for i in x)))
).reset_index()

# Create pivot table with interaction count as values
user_item_matrix = user_item_data.pivot_table(
    index='user_id',
    columns='bundle_id',
    values='interaction_count',
    fill_value=0
)

# Display user-item matrix characteristics
print(f"User-item matrix shape: {user_item_matrix.shape}")
print(f"Sparsity: {1 - (user_item_matrix > 0).sum().sum() / (user_item_matrix.shape[0] * user_item_matrix.shape[1]):.4f}")
```

```
User-item matrix shape: (881, 8256)
Sparsity: 0.9892
```

Figure 39: implementation of sparse user-item matrix in 03_svd_hybrid.ipynb

SVD, implemented via scipy.sparse.linalg.svds in hybrid.py, predicts ratings for unseen bundles.

```python
def predict_rating(self, user_idx, item_idx):
    if self.user_factors is None or self.item_factors is None:
        return self.mean_rating
    return np.dot(self.user_factors[user_idx], self.item_factors[item_idx])
```

Figure 40: implementation of predict_ratings function in hybrid.py

TF-IDF content similarity, computed using create_content_similarity_matrix, scores bundles aligned with user history. For a sample user (e.g., u100200), the hybrid model outputs 10 ‘’bundles (e.g., b5, b5567, scores ~0.24)’’ and ‘’other features,’’ excluding seen items and balancing collaborative (70%) and content-based (30%) signals, demonstrating functional integration. The output also **demonstrates successful application of TOEIC-specific mappings**.

```
INFO:src.recommender.hybrid:Generated 10 recommendations for user u100200
================================================
Enriched Recommendations for User: uu100200
================================================
Index Bundle ID   Part Name               Tags               Score   Subject     Questions   Interactions
------------------------------------------------------------------------------------------------------
1     b5          Listening Comprehension  8;2;179;181       0.2404  General     1           38
2     b5567       Listening Comprehension  19;2;184          0.2404  General     1           79
3     b5579       Listening Comprehension  18;7;183          0.2404  General     1           33
4     b160        Listening Comprehension  8;2;185           0.2404  General     1           33
5     b192        Listening Comprehension  3;2;185           0.2404  General     1           21
6     b5584       Listening Comprehension  11;7;181          0.2404  General     1           34
7     b89         Listening Comprehension  14;2;185          0.2403  General     1           21
8     b190        Listening Comprehension  6;7;181           0.2403  General     1           41
9     b173        Listening Comprehension  17;7;182          0.2403  General     1           32
10    b5577       Listening Comprehension  5;2;181           0.2403  General     1           24
================================================
```

Figure 41: Recommendations in 03_svd_hybrid.ipynb

4.6 Code Components & Architecture

The prototype uses key modules: hybrid.py for SVD-based recommendation logic, metrics.py for evaluation, and preprocessing.py for data transformation and matrix construction.

The core class, SVDHybridRecommender, factorizes the user-item matrix using scipy.sparse.linalg.svds to learn latent user and item embeddings. It supports both rating prediction and top-N recommendations, incorporating content similarity scores to enhance performance in sparse scenarios.

```python
class SVDHybridRecommender:
    """SVD-based hybrid recommender system"""

    def __init__(self, n_factors=20, combine_weight=0.7):
        self.n_factors = n_factors
        self.combine_weight = combine_weight
        self.user_factors = None
        self.item_factors = None
        self.content_similarity = None
        self.mean_rating = None
        self.train_matrix = None
        self.user_map = None
        self.item_map = None
        self.reverse_item_map = None

    def initialize(self, train_matrix, user_map, item_map, reverse_item_map):
        """Initialize the recommender with required matrices and mappings"""
        self.train_matrix = train_matrix
        self.user_map = user_map
        self.item_map = item_map
        self.reverse_item_map = reverse_item_map
```

Figure 42: Limited Snippet of class, SVDHybridRecommender

The prepare_matrices function filters users with at least five interactions and splits data into training and test sets,

```python
def prepare_matrices(merged_df, min_interactions=5):
    user_counts = merged_df['user_id'].value_counts()
    valid_users = user_counts[user_counts >= min_interactions].index
    filtered_data = merged_df[merged_df['user_id'].isin(valid_users)].copy()
    filtered_data['correct'] = (filtered_data['user_answer'] == filtered_data['correct_answer']).astype(int)
    user_item_data = filtered_data.groupby(['user_id', 'bundle_id']).agg(
        correctness_rate=('correct', 'mean'),
        interaction_count=('correct', 'count')
    ).reset_index()
    user_ids = filtered_data['user_id'].unique()
    bundle_ids = filtered_data['bundle_id'].unique()
    user_map = {user_id: idx for idx, user_id in enumerate(np.unique(user_ids))}
    item_map = {bundle_id: idx for idx, bundle_id in enumerate(np.unique(bundle_ids))}
    ratings_matrix = np.zeros((len(user_map), len(item_map)))
    for _, row in user_item_data.iterrows():
        if row['user_id'] in user_map and row['bundle_id'] in item_map:
            u = user_map[row['user_id']]
            i = item_map[row['bundle_id']]
            ratings_matrix[u, i] = row['interaction_count'] * (1 + 0.4 * row['correctness_rate'])
    interactions = [(u, i, ratings_matrix[u, i]) for u in range(ratings_matrix.shape[0])
                    for i in range(ratings_matrix.shape[1]) if ratings_matrix[u, i] > 0]
    train, test = train_test_split(interactions, test_size=0.2, random_state=42)
    train_matrix, test_matrix = np.zeros_like(ratings_matrix), np.zeros_like(ratings_matrix)
    for u, i, r in train: train_matrix[u, i] = r
    for u, i, r in test: test_matrix[u, i] = r
    return train_matrix, test_matrix, user_map, item_map
```

Figure 43: Implementation of prepare_matrices function in preprocessing.py

while create_content_similarity_matrix uses TF-IDF to compute bundle similarity.

```python
def create_content_similarity_matrix(lectures_df, text_column='content', ngram_range=(1, 2), min_df=2):
    """
    Create content similarity matrix using TF-IDF and cosine similarity.

    Parameters:
    - lectures_df: DataFrame containing lecture content
    - text_column: Column name containing text content
    - ngram_range: Range of n-grams for TF-IDF
    - min_df: Minimum document frequency for TF-IDF

    Returns:
    - similarity_matrix: Matrix of content similarities
    - vectorizer: TF-IDF vectorizer used
    """

    # Clean and preprocess text
    lectures_df[text_column] = lectures_df[text_column].apply(preprocess_content_text)

    # Create TF-IDF matrix
    vectorizer = TfidfVectorizer(
        ngram_range=ngram_range,
        min_df=min_df,
        stop_words='english'
    )
    tfidf_matrix = vectorizer.fit_transform(lectures_df[text_column])

    # Calculate cosine similarity
    similarity_matrix = cosine_similarity(tfidf_matrix)

    return similarity_matrix, vectorizer
```

Figure 44: Implementation of create_content_similarity_matrix function in preprocessing.py

Evaluation metrics such as precision@10, recall@10 and RMSE, are computed using calculate_precision_recall_at_k and calculate_rmse. These metrics validate recommendation quality and ensure alignment between bundle metadata and similarity scores.

```python
def calculate_precision_recall_at_k(model, test_matrix, k=10):
    precision_sum = 0.0
    recall_sum = 0.0
    user_count = 0

    for u in range(test_matrix.shape[0]):
        test_items = np.where(test_matrix[u] > 0)[0]
        if len(test_items) == 0:
            continue

        known_items = np.where(model.train_matrix[u] > 0)[0]
        recs = model.get_recommendations(u, n=k, exclude_seen=True, known_items=known_items)
        if not recs:
            continue

        rec_items = [item for item, _ in recs]
        hits = len(set(rec_items) & set(test_items))

        precision_sum += hits / min(k, len(rec_items))
        recall_sum += hits / len(test_items)
        user_count += 1

    avg_precision = precision_sum / user_count if user_count > 0 else 0.0
    avg_recall = recall_sum / user_count if user_count > 0 else 0.0
    return avg_precision, avg_recall

def calculate_rmse(model, test_matrix):
    non_zero_indices = np.where(test_matrix > 0)
    y_true = []
    y_pred = []
    for u, i in zip(non_zero_indices[0], non_zero_indices[1]):
        y_true.append(test_matrix[u, i])
        y_pred.append(model.predict_rating(u, i))
    return np.sqrt(mean_squared_error(y_true, y_pred))
```

Figure 45: Implementation of calculate_precision_recall_at_k and calculate_rmse function in metrics.py

While 02_tf_idf.ipynb provides a basic content-based baseline, the main functionality is implemented in 03_svd_hybrid.ipynb, where score blending and matrix factorization form the backbone of the hybrid model.

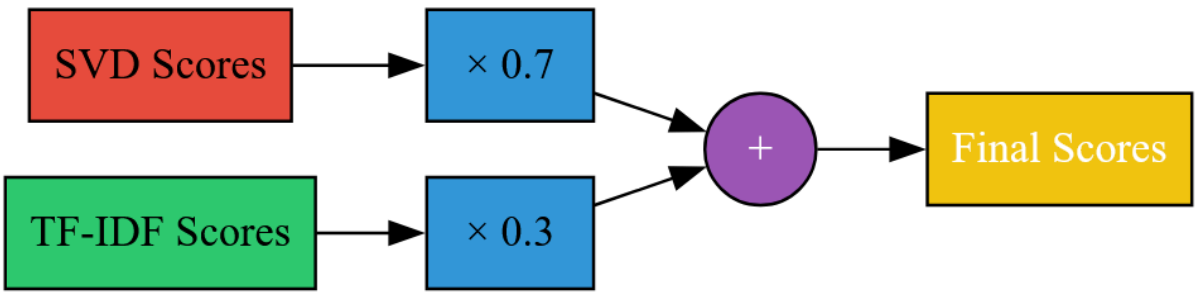


Figure 46: score blending flowchart of hybrid model

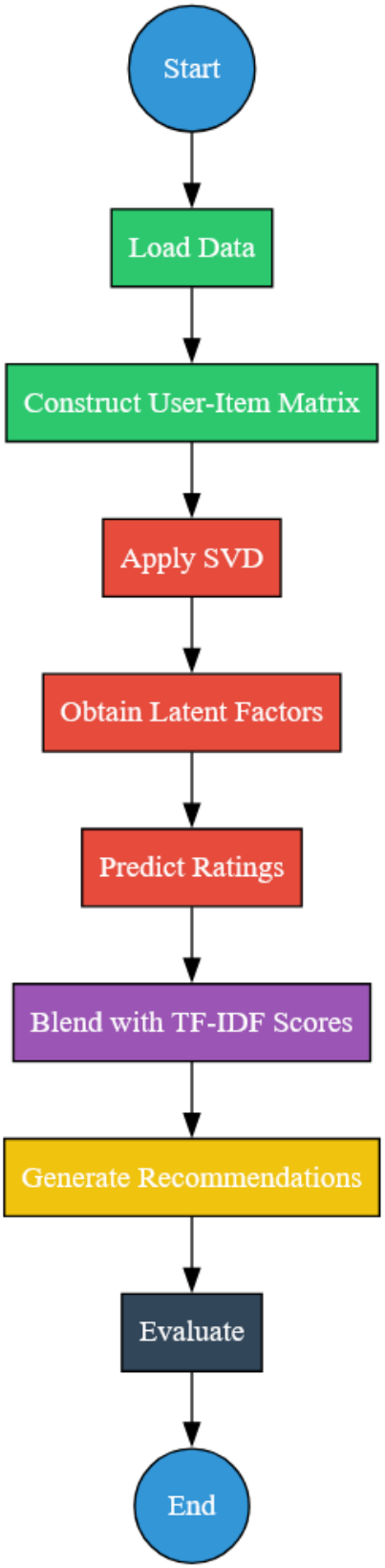


Figure 47: Matrix Factorization Flowchart of Hybrid model

4.7 Evaluation

From a technical standpoint, the prototype is functional and integrates all core elements of a personalized recommender: content understanding, user modeling, scoring and evaluation. Mapping functions effectively categorize complex tag structures into interpretable domains, supporting feature engineering and explainability. The TF-IDF-based content model confirms that similar bundles can be identified without explicit feedback, while the hybrid model blends latent factors with content signals to address sparsity.

However, limitations emerged due to data characteristics. The TF-IDF baseline (from 02_tf_idf.ipynb) scored 0.0 on both precision@10 and recall@10, confirming that content similarity alone is inadequate in sparse domains like educational content.

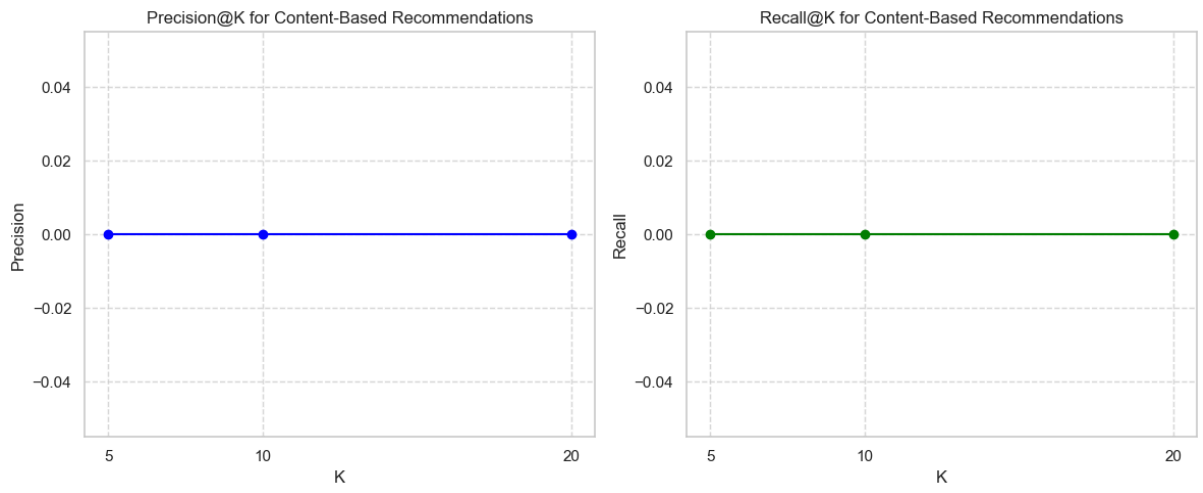


Figure 48: Performance of both precision@10 and recall@10

The hybrid model (from 03_svd_hybrid.ipynb) achieved modest improvements, with precision@10 of 0.0194, recall@10 of 0.0140 and RMSE of 2.5306, still underperforming a pure SVD baseline (precision@10: 0.0439, recall@10: 0.0735). These results reflect high data sparsity (98.92%), limited interaction history and coarse-grained feedback (e.g., interaction counts rather than ratings).

Comparison:

Method	RMSE	Precision@10	Recall@10
Hybrid SVD	2.5306	0.0194	0.0140
Pure SVD	2.5306	0.0439	0.0735
Improvement	0.0000	-0.0246	-0.0594

Figure 49: Shows how metrics of Hybrid underperforms in 03_svd_hybrid.ipynb

One major finding I got about hybrid weights; As the weight increases to **0.8 and 0.9**, there is a significant boost in recall, suggesting better coverage of relevant recommendations. This implies that higher weighting improves the model's ability to retrieve more relevant items, potentially enhancing user experience. Meanwhile, precision improves gradually but does

not exhibit a sharp increase, meaning that while the model is identifying more relevant recommendations, its accuracy in selecting the most relevant ones remains relatively stable.

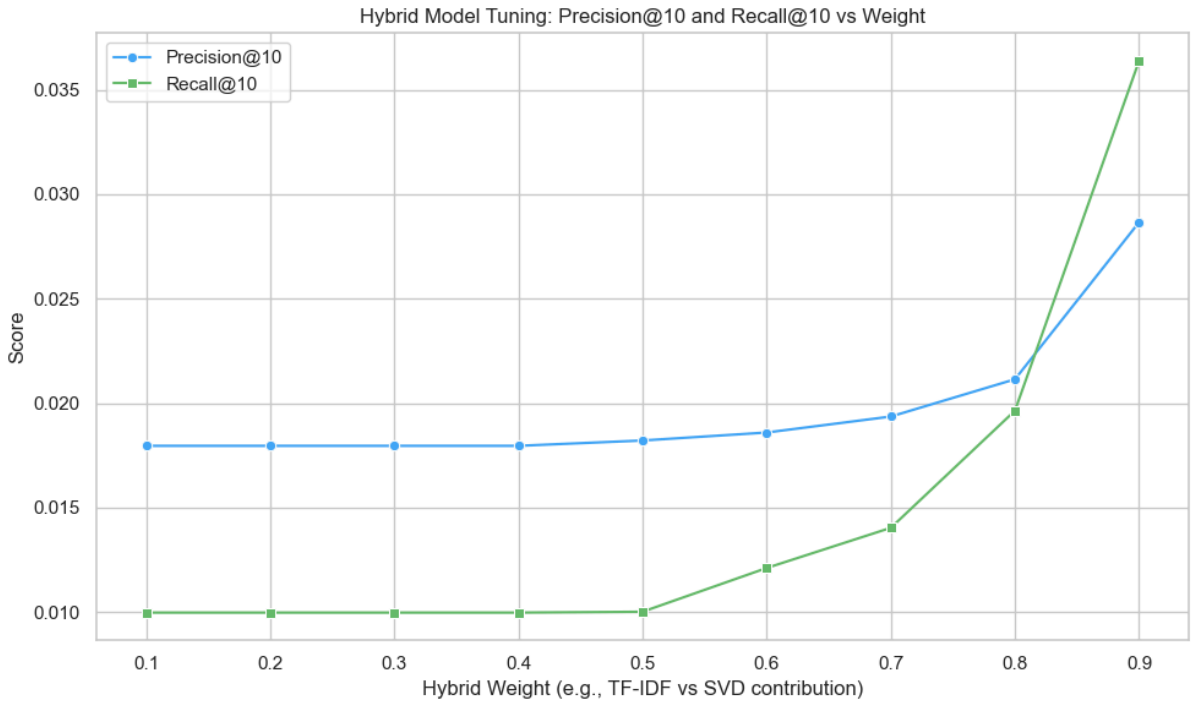


Figure 50: Performance of precision@10 and recall@10 vs Weight

Additionally, RMSE appears to be unaffected by these weight adjustments, indicating that error correction may depend on other factors rather than the weighting of components in the hybrid model.

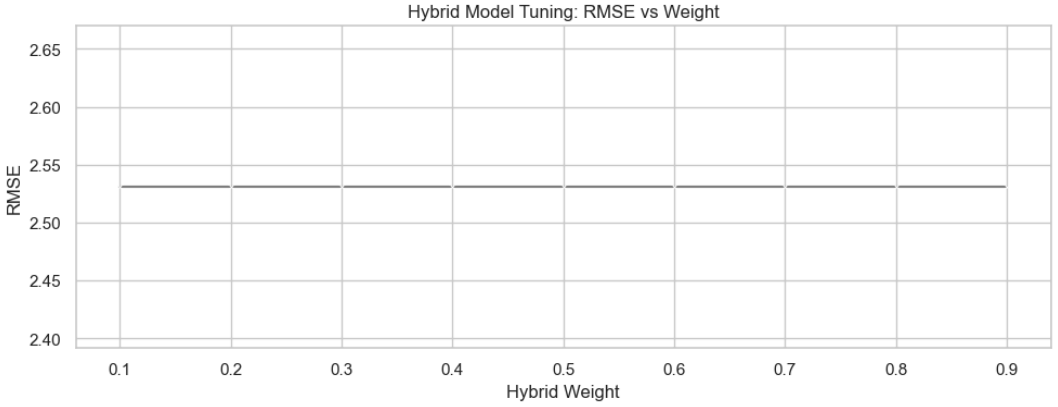


Figure 51: Performance of RMSE vs Weight

Evaluation was conducted using standard recommender system metrics (precision@k, recall@k, RMSE), appropriate for measuring both ranking quality and prediction error [31]. Qualitative inspection showed that the hybrid system produces semantically relevant bundles and its performance, while limited, surpasses a random baseline and confirms the feasibility of the hybrid approach. Further improvements in data quality and feature design are needed to reach higher accuracy

Table 20: Comparison of Evaluation metrics

Model	Precision@10	Recall@10	RMSE
TF-IDF	0.0000	0.0000	NaN
SVD	0.0439	0.0735	2.38
Hybrid (α=0.7)	0.0194	0.0140	2.53

4.8 Next Steps and Improvements

To address current limitations, future iterations will enhance both data and model components. The hybrid prototype's underperformance, attributed to high data sparsity and simplistic TF-IDF features, highlights the need for richer signals.

The baseline content model (02_tf_idf.ipynb) produced zero precision and recall, reaffirming its role as a basic comparator. To address this, 04_model_tuning.ipynb will introduce a systematic model tuning phase. It applies **grid search over TF-IDF parameters** (e.g., ngram_range, max_df, min_df and max_features) to improve content similarity scoring [33] and **tunes the hybrid blending weight** between SVD predictions and content-based signals. This will result in improved precision for hybrid models and identify the optimal configuration for score fusion (e.g., setting content weight = 1.0 yields better results). This is well-supported by research. Weighted hybrid recommender systems, combining collaborative filtering (e.g., SVD) with content-based signals, are a well-established approach. Studies show optimizing the blending parameter (α) can significantly enhance recommendation accuracy (lower RMSE or higher precision) [34]

Building further, 05_advanced_recommendations.ipynb will tackle the deeper issues of content sparsity and semantic mismatch by introducing **advanced representations and models**. It replaces TF-IDF with **Sentence-BERT embeddings**, enabling more nuanced semantic understanding of bundle metadata [35].

On the collaborative filtering side, it implements **Neural Collaborative Filtering (NCF)** to capture non-linear user-item interactions more effectively than SVD. Currently SVD uses matrix factorization

```
u, sigma, vt = svds(ratings_filled, k=n_factors)
self.user_factors = u
self.item_factors = vt.T
```

Figure 52: In hybrid.py, the .fit() method runs on above

This decomposes the **ratings matrix** R into:

$$R \approx U \times \Sigma \times V^t$$

But this factorization captures **linear relationships** between users and items, meaning the recommendation score is just:

score = dot(user_factors[u], item_factors[i])

Neural CF uses deep architectures to model nonlinear interactions, empirically outperforming matrix factorization on implicit signals [36]

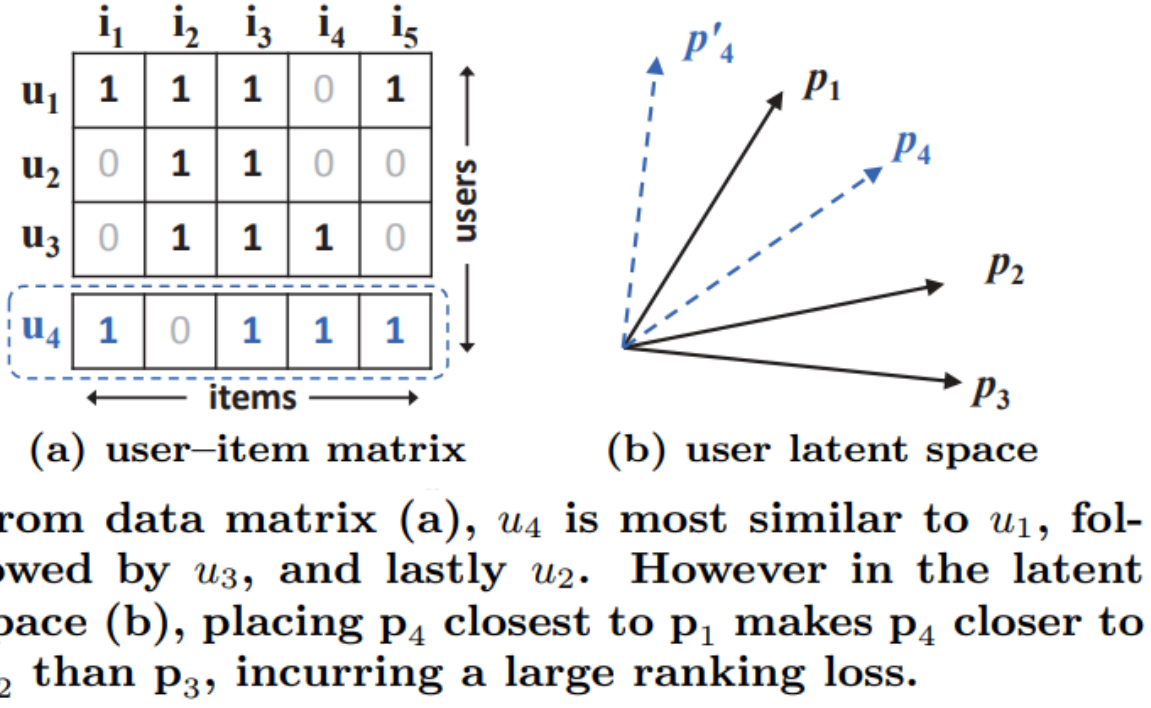


Figure 53: An example illustrating the limitation of Matrix Factorization

Finally, it introduces a **meta-learner ensemble** that blends SBERT and NCF scores *''multiple models''* [37] using linear regression, trained to predict bundle relevance. This advanced hybrid model produces semantically coherent, behavior-aware recommendations with improved metrics over baselines. Importantly, it also demonstrates extensibility and practical deployment pathways by saving learned models and embeddings.

These improvements aim to strengthen the system's personalization capabilities and overall effectiveness in TOEIC preparation.

4.9 Conclusion

The prototype in 03_svd_hybrid.ipynb validates the feasibility of a hybrid recommendation system for TOEIC preparation, integrating SVD and TF-IDF-based content filtering. The 02_tf_idf.ipynb model, while foundational, is a weak prototype due to its poor performance, serving as a baseline.

PART 2: Beyond the Prototype

This part documents the implementation phase of the personalized educational recommendation system for TOEIC, detailing from prototype to a functional application. It expands on the earlier prototype described in Part 1, incorporating the system's actual execution, including backend logic, model deployment and user interface integration. The implemented system supports real-time content recommendation through a web-based interface, enabling learners to receive tailored study material based on their interaction history and learning profiles.

4.10 Overview and Role of Implementation

The implementation builds upon the hybrid recommendation model tested during the prototype phase. The core objective has remained consistent based on validation from project proposal and preliminary report's feedbacks: to enhance TOEIC preparation by delivering personalized recommendations of question bundles and lectures. Unlike the prototype, which was restricted to notebook-based offline testing, the implementation integrates model inference into a complete end-to-end pipeline. This pipeline includes a Streamlit frontend for user interaction, a model loading and preprocessing backend, and a scoring engine based on hybrid and advanced deep learning models.

The architectural components were gradually developed, starting with content-based TF-IDF models, followed by SVD-based collaborative filtering and finally incorporating advanced techniques such as Sentence-BERT embeddings and Neural Collaborative Filtering (NCF). The final recommender system includes a REST-like API structure, robust data preprocessing modules and a unified interface for exploring recommendations.

4.11 Feature Implementation

Table 21: Key Features of the Implemented System

Feature	Description	Purpose / Value Delivered

Lecture and Bundle Recommendations	Hybrid models (SVD + TF-IDF) and advanced models (NCF + SBERT) generate personalized study content.	Ensures learners receive tailored question bundles and lectures that match their needs.
Metadata Mapping & Display	TOEIC part numbers & tag strings are mapped to meaningful descriptions using domain-specific mappings.	Improves interpretability by making recommendations clearer (e.g., “Part 5: Reading”).
Content Enrichment	Each recommendation includes metadata such as part name, subject categories, and lecture duration.	Provides richer context, helping learners understand why a bundle/lecture is recommended.
Evaluation Metrics Visualization	System performance can be inspected through metrics (Precision, Recall, RMSE, AUC) via visual graphs.	Enables transparent model evaluation and supports comparison between different algorithms.
Interactive Frontend	Streamlit interface lets users input their user ID, choose recommendation type, and view outputs.	Makes the system accessible, easy to use & interactive without requiring coding skills.
User Insights Dashboard	Displays user-specific analytics: accuracy, response time & practice distribution across TOEIC parts.	Helps learners and educators identify strengths/weaknesses & track progress effectively.

4.12 Algorithms and Techniques

Table 22: Hybrid Recommendation Model Architecture and Rationale

Component	Details

Architectural Development	The architectural components were gradually developed, starting with content-based TF-IDF models, followed by SVD-based collaborative filtering and finally incorporating advanced techniques such as Sentence-BERT embeddings and Neural Collaborative Filtering (NCF). The final recommender system includes a REST-like API structure, robust data preprocessing modules and a unified interface for exploring recommendations.
Hybrid System Implementation	The hybrid recommendation system is implemented through the SVDHybridRecommender class, which integrates collaborative filtering and content-based filtering to balance personalization with generalizability.
Collaborative Filtering (SVD)	Collaborative filtering is achieved using Singular Value Decomposition (SVD), which factorizes the user-item interaction matrix into latent vectors representing user preferences and item characteristics. This approach enables the system to estimate ratings for unseen items by aligning latent factors across similar users and it handles data sparsity effectively through matrix factorization.
Content-Based Filtering (SBERT)	Content-based filtering is implemented using Sentence-BERT (SBERT) embeddings. SBERT transforms item metadata, such as bundle titles and subject categories, into dense, contextual vector representations. Cosine similarity between these embeddings allows the system to identify semantically similar content, even when literal word overlap is minimal. This is particularly useful for cold-start scenarios, where user interaction data is limited but content metadata is available.
Hybridization Process	The hybridization process combines collaborative and content-based scores using a configurable weight parameter (combine_weight).
Final Score Formula	The final recommendation score is computed as: final_score = (combine_weight × CF_score) + ((1 - combine_weight) × CB_score)
Parameter Tuning	This parameter is tuned during evaluation to optimize the balance between personalization and content relevance.

Table 23: Sentence-BERT Embeddings Architecture and Rationale

Component	Description
TF-IDF Baseline	While TF-IDF provides a baseline for text similarity, it is inherently limited.
SBERT Integration	To capture deeper semantic meaning and contextual nuances, the system integrates Sentence-BERT (SBERT).
SBERT Embedding Process	SBERT transforms text descriptions (e.g., question bundle metadata, subject categories) into dense, contextual embeddings. Unlike sparse TF-IDF vectors, these embeddings preserve semantic similarity, for instance, recognizing that “Incomplete Sentences” and “Grammar Practice” are related concepts, even if they share few literal words.
Similarity Computation	The similarity between SBERT embeddings is computed using cosine similarity, producing more meaningful recommendations.
Domain-Specific Effectiveness	SBERT is especially effective when dealing with loosely structured or domain-specific educational metadata, as it provides contextual richness that traditional word-frequency approaches cannot.

Table 24: Neural Collaborative Filtering (NCF) Architecture and Rationale

Aspect	Description
Overview	Neural Collaborative Filtering (NCF) models non-linear interactions using dense neural layers. The implementation uses PyTorch and a simple architecture with two embedding layers followed by a configurable number of hidden layers and a sigmoid activation function for implicit feedback prediction.
Limitation of Traditional Models	Traditional matrix factorization models like SVD assume linear relationships between users and items, which may not fully capture complex learner

	behaviors. To address this limitation, the system implements Neural Collaborative Filtering (NCF) using PyTorch.
Architectural Replacement	The NCF model replaces linear inner products with a **multi-layer neural network** capable of modeling non-linear interactions. Its architecture consists of: ● **User and item embedding layers**: Learners and content bundles are each mapped into dense vectors. ● **Hidden layers**: These embeddings are concatenated and passed through fully connected layers with non-linear activation functions (e.g., ReLU). ● **Output layer**: A sigmoid activation produces a probability-like score representing the likelihood of user-item interaction (implicit feedback).
Predictive Advantage	By learning higher-order feature interactions, NCF delivers stronger predictive power than linear models, particularly in sparse data environments.

```python
class NCF(nn.Module):
    def __init__(self, num_users, num_items, embedding_dim=32, hidden_dims=[64, 32, 16]):
        super(NCF, self).__init__()

        # Embeddings for Generalized Matrix Factorization (GMF)
        self.user_gmf = nn.Embedding(num_users, embedding_dim)
        self.item_gmf = nn.Embedding(num_items, embedding_dim)

        # Embeddings for Multi-Layer Perceptron (MLP)
        self.user_mlp = nn.Embedding(num_users, embedding_dim)
        self.item_mlp = nn.Embedding(num_items, embedding_dim)

        # Build MLP layers dynamically based on hidden_dims
        layers = []
        input_dim = 2 * embedding_dim  # Concatenated user/item embeddings
        for dim in hidden_dims:
            layers.append(nn.Linear(input_dim, dim))  # Fully connected layer
            layers.append(nn.ReLU()) # Activation function
            input_dim = dim  # Update input size for next layer

        self.mlp = nn.Sequential(*layers)  # Stack layers into a sequential model

        # Final prediction layer combines GMF and MLP outputs
        self.final = nn.Linear(embedding_dim + hidden_dims[-1], 1)

        # Sigmoid activation to squash output between 0 and 1
        self.sigmoid = nn.Sigmoid()

    def forward(self, user_ids, item_ids):
        # GMF: element-wise product of user/item embeddings
        gmf_output = self.user_gmf(user_ids) * self.item_gmf(item_ids)

        # MLP: concatenate user/item embeddings and pass through MLP
        mlp_input = torch.cat([self.user_mlp(user_ids), self.item_mlp(item_ids)], dim=-1)
        mlp_output = self.mlp(mlp_input)

        # Combine GMF and MLP outputs, then pass through final layer
        output = self.final(torch.cat([gmf_output, mlp_output], dim=-1))

        # Apply sigmoid and remove extra dimensions
        return self.sigmoid(output).squeeze()
```

Figure 54 : NCF implementation

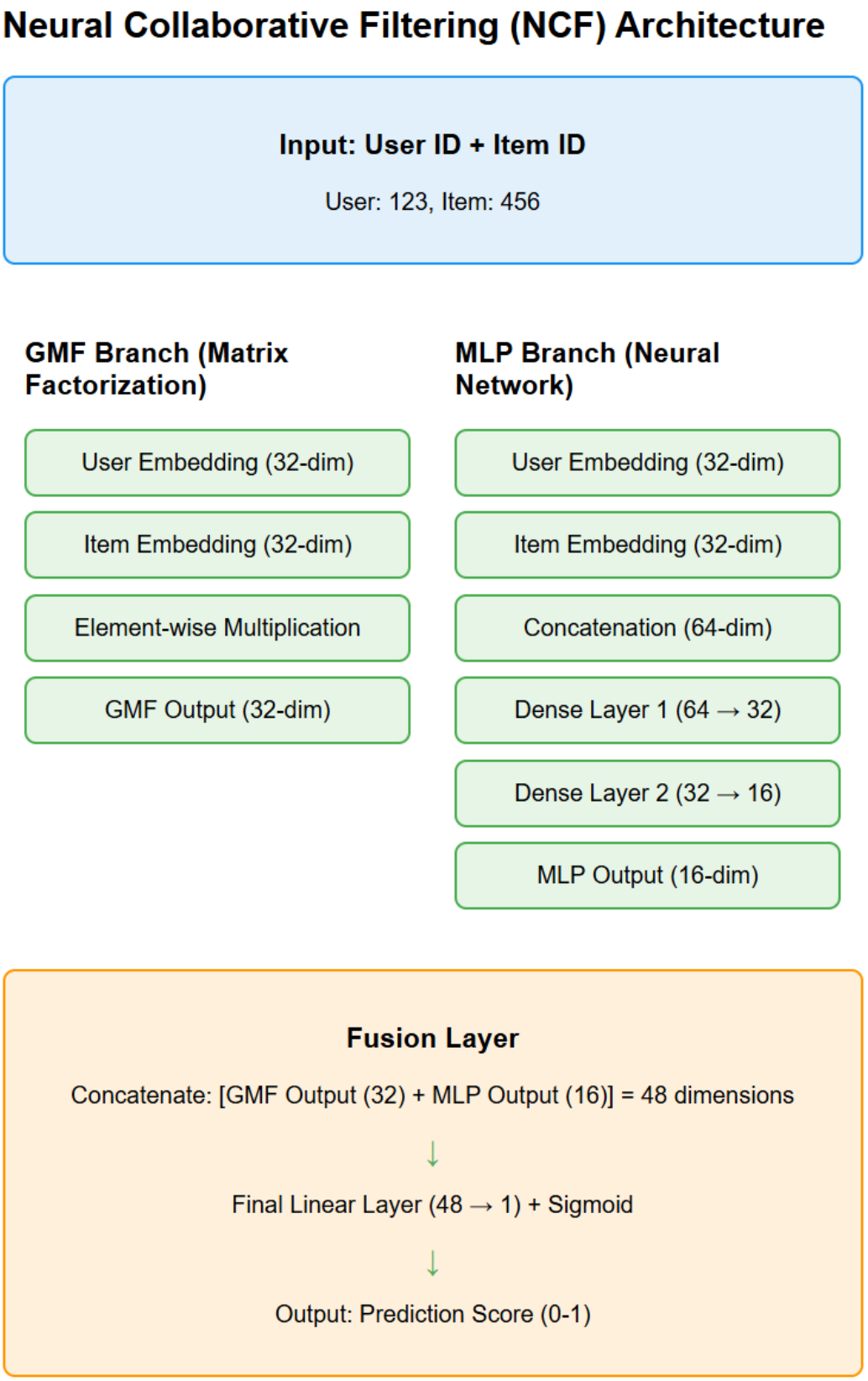


Figure 55: Architecture Showing How NCF is implemented

Meta-Learner

To further enhance recommendation quality, a meta-learner is introduced via the get_hybrid_advanced_recommendations function. This ensemble model integrates scores from SVD, SBERT and Neural Collaborative Filtering (NCF), along with user interaction history and item metadata. It trains on validation data using a linear regression model to predict engagement probabilities, optimizing for ranking metrics such as NDCG and Precision@K.

The meta-learner adapts dynamically to different scenarios. For cold-start users, it emphasizes content-based signals from SBERT. When sufficient interaction data is available, it shifts toward collaborative signals from SVD and NCF. This layered approach allows the system to leverage the strengths of each base model while mitigating their individual limitations. The implementation ensures robust, context-aware recommendations that are both accurate and pedagogically meaningful.

```python
def get_hybrid_advanced_recommendations(user_id, merged_df, sbert_embeddings, bundle_info,
                                        recommender, item_map, reverse_item_map,
                                        ncf_model, device, meta_learner, n=10):
    # Generate combined features from SBERT and NCF models
    features_df = generate_hybrid_features(
        user_id, merged_df, sbert_embeddings, bundle_info,
        recommender, item_map, reverse_item_map, ncf_model, device, n
    )
    # Return empty list if no features available
    if features_df.empty:
        return []

    # Prepare input for meta-learner and predict hybrid scores
    X = features_df[['sbert_score', 'ncf_score', 'part_id', 'success_rate']].values
    features_df['hybrid_score'] = meta_learner.predict(X)

    # Select top-N recommendations based on hybrid score
    top_recs = features_df.sort_values('hybrid_score', ascending=False).head(n)

    results = []
    # Build final recommendation list with metadata
    for _, row in top_recs.iterrows():
        row_match = bundle_info[bundle_info['bundle_id'] == row['bundle_id']]
        if row_match.empty:
            continue
        bundle_row = row_match.iloc[0]
        results.append({
            'item_id': row['bundle_id'],
            'score': round(float(row['hybrid_score']), 4),
            'title': f"{bundle_row['part_name']}: {bundle_row['subject_category'].replace('_', ' ').title()}",
            'part': bundle_row['part_name'],
            'part_id': int(row['part_id']),
            'subjects': get_subject_categories(bundle_row['tags']),
            'duration_minutes': float(bundle_row.get('video_minutes', bundle_row.get('duration_minutes', 0))),
            'type': 'bundle'
        })
    return results
```

Figure 56: Meta-Learner implementation

4.13 Visual Results

The final implementation is visualized through a Streamlit interface (Figure 57). Users can select a recommendation type, numbers of recommendations and user ID to receive enriched recommendations. Titles, subject tags, TOEIC parts and estimated durations are displayed.

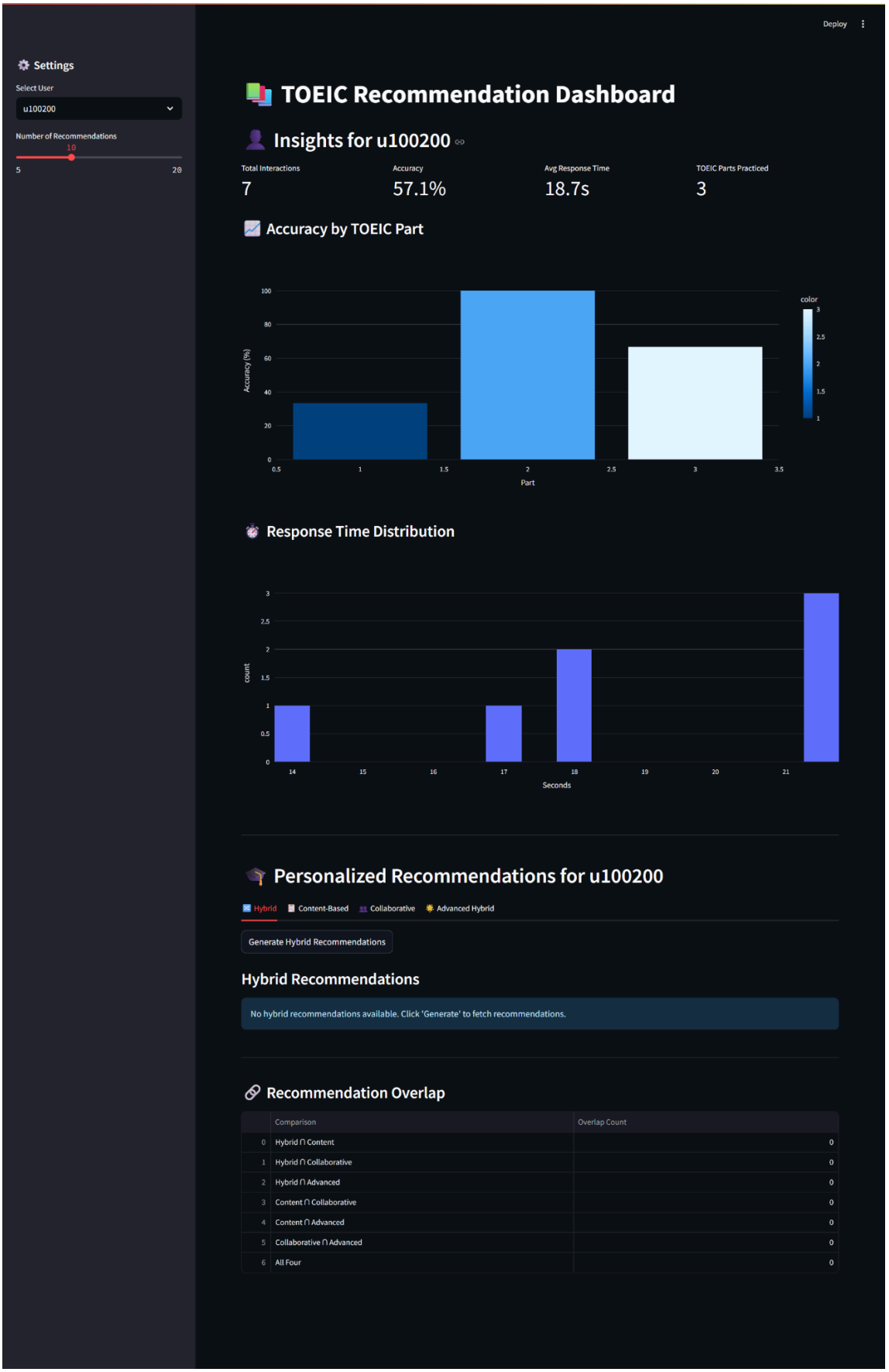


Figure 57: Streamlit Interface of the Recommendation System

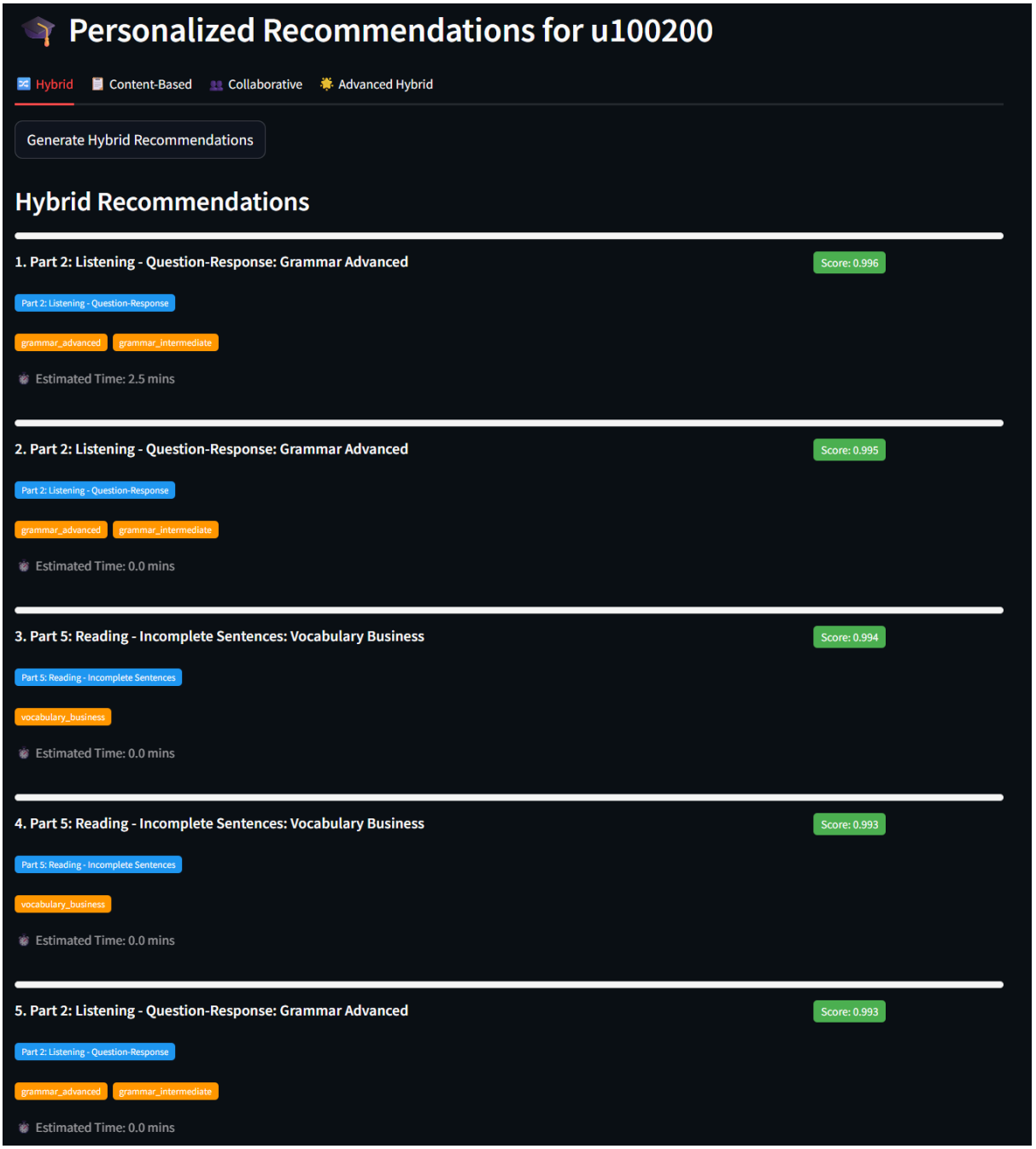


Figure 58: When Generate button is clicked

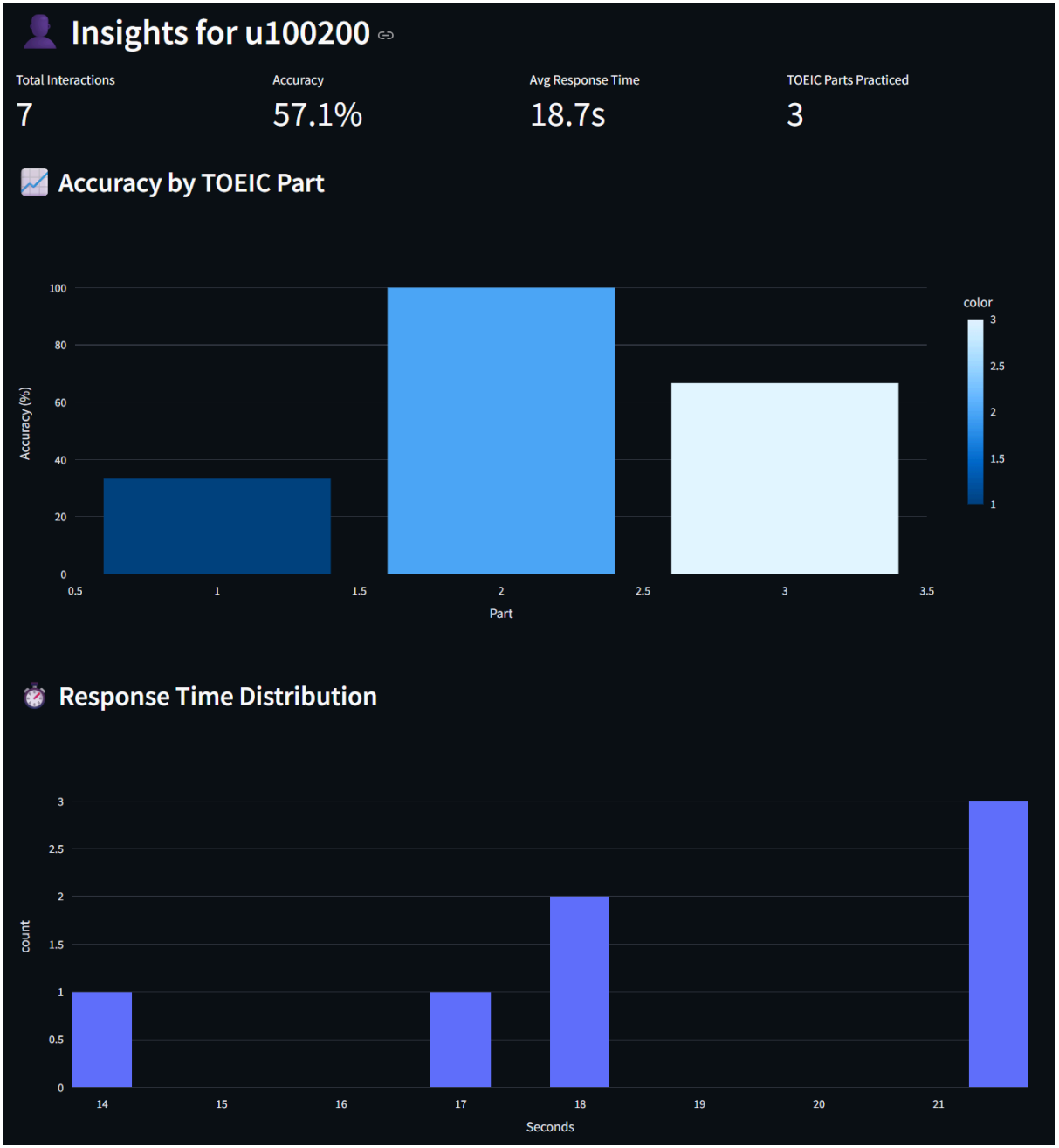


Figure 59: Learner Insights of Selected User ID

An additional panel displays personalized and interactive learner insights. For example, user u100200 is shown to have:

Table 25: Learner Insights example

Metric	Value
Total Interactions	7

Accuracy	57.1%
Average Response Time	18.7 seconds
TOEIC Parts Practiced	3 parts

Further breakdowns include:

Table 26: Visualizations

Chart Type	Description
Bar Chart	Accuracy across TOEIC parts
Histogram	Distribution of response times

Two additional visual diagnostics were also added:

Table 27: Visual diagnostics

Chart Name	Description
Recommendation Overlap	This chart visualizes the number of common items recommended by different models (e.g., SVD-only vs Hybrid vs SBERT). It offers insight into whether models agree or diverge in their outputs.
Part Distribution	This plot shows which TOEIC parts the top recommended bundles fall under. It helps ensure model diversity and balanced subject exposure.

Figure 60: Visual Diagnostics Of Each Recommendation Type For 5 Recommended Items

What it actually shows:

Table 28: Recommendation Overlap

Overlap	Details
Hybrid ∩ Advanced = 3	Both algorithms recommend items from Part 2 and Part 5
Content ∩ Advanced = 2	Both algorithms recommend items from Part 1
Zero overlaps elsewhere	Different algorithms focus on different parts, which is actually good for diversity

Overlap Shows Algorithm Diversity:

Table 29: Part Distribution

Model Type	Details
Content-based	Focuses on Part 1 (Photographs)
Collaborative	Spreads across Parts 3, 4, 6, 7
Hybrid	Focuses on Parts 2 and 5
Advanced Hybrid	Combines elements from multiple parts

These insights assist educators or learners in identifying weak areas or time management issues, offering a layer of interpretability beyond simple recommendations.

In conclusion, all of the visualizations are easy to understand and now the app has many user-friendly labels.

Model tuning outputs include visual graphs:

One major finding I got about hybrid weights; As the weight increases to **0.8 and 0.9**, there is a significant boost in both recall, suggesting better coverage of relevant recommendations. This implies that higher weighting improves the model's ability to retrieve more relevant items, potentially enhancing user experience. Meanwhile, precision also improves along with a sharp increase, meaning that while the model is identifying more relevant recommendations, its accuracy in selecting the most relevant ones remains relatively stable. Overall performance shows better than the prototype's Figure 50

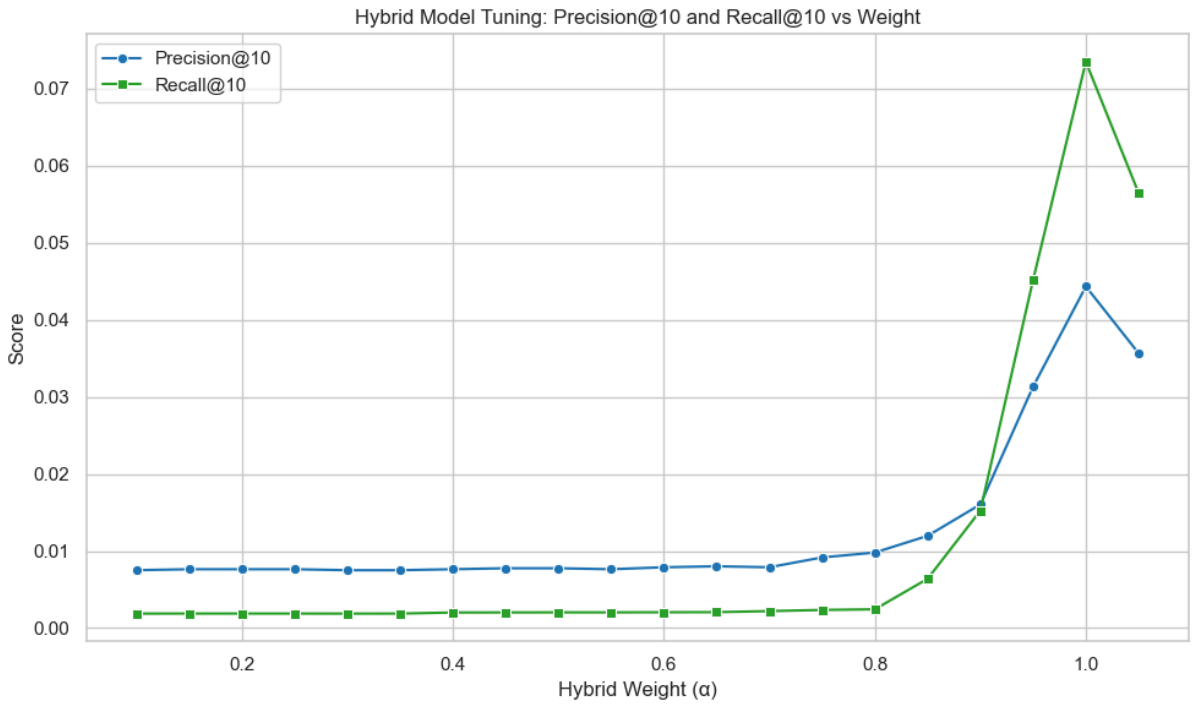


Figure 61: Precision@10 vs Hybrid Weight

Additionally, RMSE appears to be unaffected by these weight adjustments, indicating that error correction may depend on other factors rather than the weighting of components in the hybrid model. Just like prototype's Figure 51

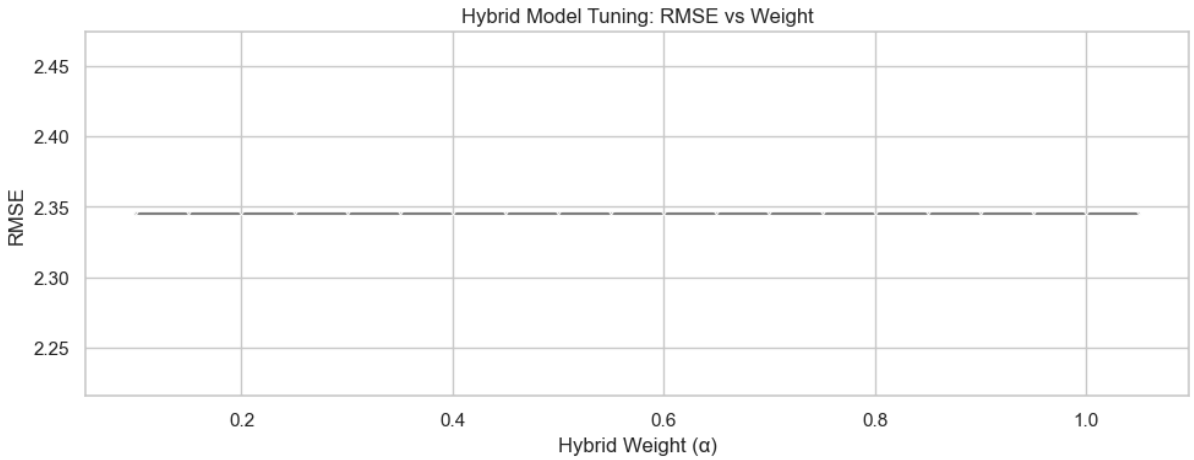


Figure 62: RMSE vs Hybrid Weight

Table 30: Evaluation Metrics and Model Comparison

To objectively evaluate the effectiveness of various models, I adopted industry-standard metrics:

Metric	Description

Precision@10	Fraction of top-10 recommended items that were actually interacted with by the user.
Recall@10	Fraction of relevant items that were successfully retrieved in the top-10 results.
RMSE	For rating prediction accuracy.

	Model	Precision@10	Recall@10	RMSE
0	Baseline TF-IDF	0.000000	0.000000	NaN
1	Baseline SVD	0.002445	0.001453	2.380953
2	NCF	0.025000	0.024952	1.330229
3	Advanced Hybrid	0.020833	0.024383	2.543401

Figure 63: Comparison of All Models on Evaluation Metrics

Table 31: Model Performance

Model	Performance Summary
TF-IDF	TF-IDF yielded no precision or recall, reaffirming its unsuitability in sparse interaction domains. (Implemented in Prototype)
SVD	SVD, while marginally better, remained weak due to linear assumptions and lack of content-awareness. (Implemented in Prototype)
NCF	NCF significantly outperformed others in both precision and recall, suggesting its effectiveness at modeling non-linear interaction patterns.
Advanced Hybrid	Advanced Hybrid achieved slightly lower precision than NCF but retained stronger interpretability via its blending of semantic content and behavior.

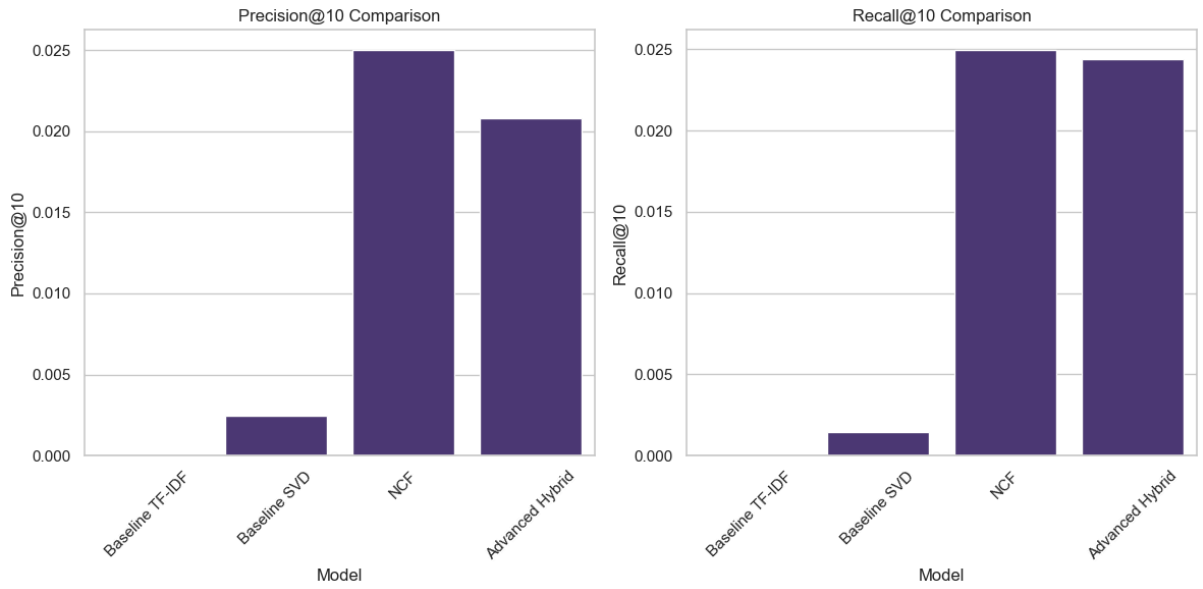


Figure 64: Visualization of All Models on Evaluation Metrics

Figure 64 visualizes this comparison, highlighting trade-offs between accuracy (RMSE) and relevance (precision/recall). This comparison justifies future investment into NCF + SBERT + Meta-learner ensembles and also shows how it performs better than the models Implemented in Prototype, i.e. **TF-IDF** & **SVD**

The **Advanced Hybrid model**, which blends SVD + TF-IDF + SBERT + NCF, performs well in terms of recall while **NCF** achieved the best RMSE and highest top-k precision. TF-IDF alone failed to deliver any significant value, reaffirming the necessity of collaborative or deep-learning-based augmentation.

4.14 API Endpoints

Overview

To enable real-time interaction between the recommendation models and the user-facing Streamlit application, the system was extended with a **FastAPI backend**. This provided REST-like endpoints that expose model functionality through lightweight and scalable interfaces. Each endpoint was designed to accept structured requests, perform validation using **Pydantic schemas** and return enriched recommendation results or user insights in JSON format.

Since my backend is built with **FastAPI**, it automatically provides interactive API docs at http://localhost:8000/docs and http://localhost:8000/redoc

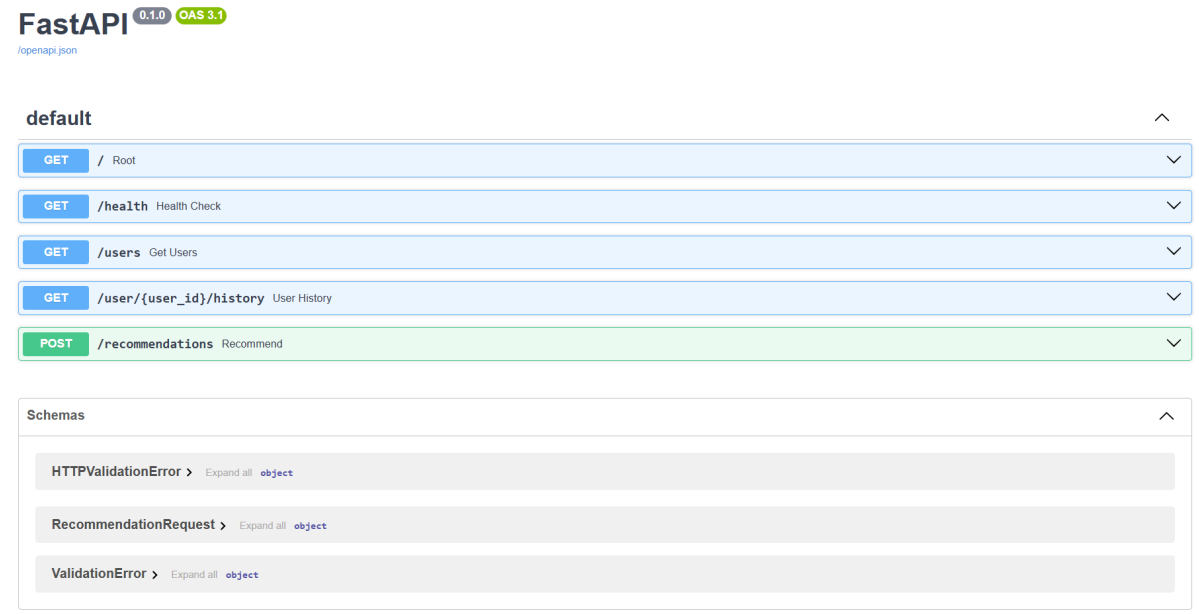


Figure 65: API docs after starting API

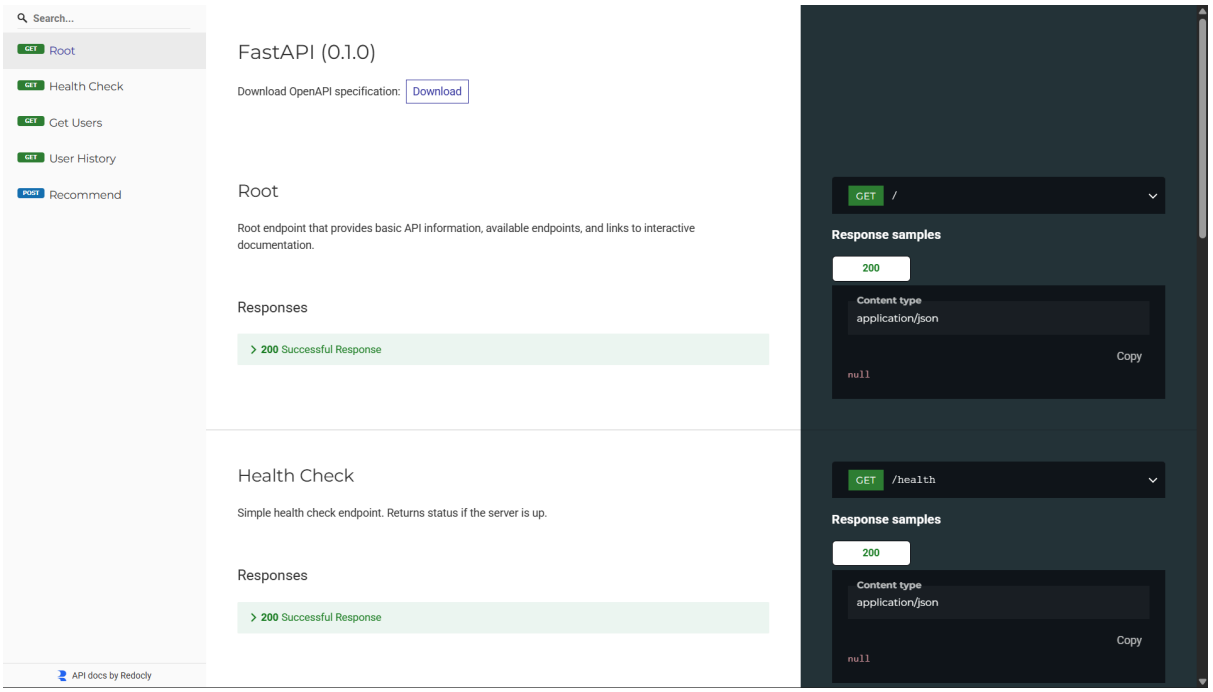


Figure 66: API redocs after starting API

1. Root Endpoint

- **GET /**
- Provides basic metadata about the API.
- Used to confirm that the server is running and accessible.

```python
@app.get("/")
async def root():
    """
    Root endpoint that provides basic API information, available endpoints,
    and links to interactive documentation.
    """
    return {
        "message": "Educational Content Recommendation System API",
        "description": (
            "This API provides personalized TOEIC study recommendations, "
            "user history access, and system health monitoring."
        ),
        "docs": {
            "Swagger UI": "/docs",
            "ReDoc": "/redoc"
        },
        "endpoints": {
            "GET /": "This information page",
            "GET /health": "Health check endpoint",
            "GET /users": "List all available users",
            "GET /user/{user_id}/history": "Get user interaction history",
            "POST /recommendations": (
                "Generate personalized recommendations "
                "(requires JSON body with user_id, recommendation_type, n_recommendations)"
            ),
        },
    }
```

Figure 67: Root Endpoint implementation in api/main.py

- After starting API, root endpoint will be available at http://localhost:8000

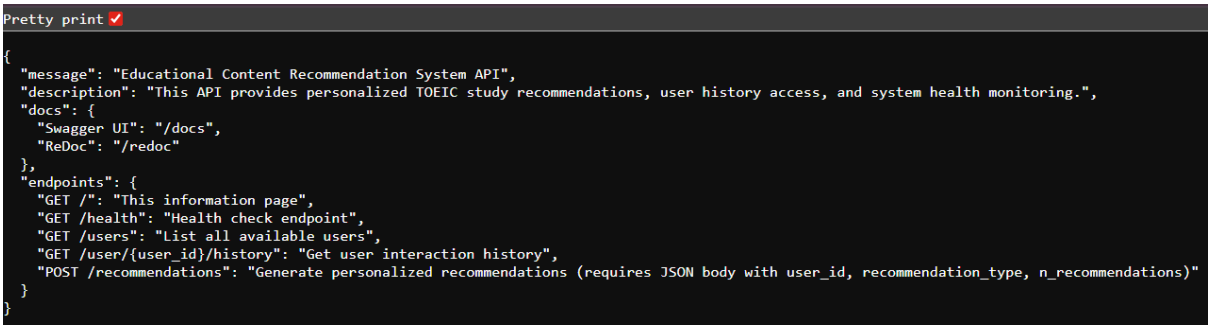


Figure 68: Root Endpoint running in browser

2. Health Check Endpoint

- **GET /health**
- Returns a simple status message confirming system availability.
- Primarily used for monitoring and debugging to ensure the backend is operational.

```python
# Define a GET endpoint at /health to check if the API is running
@app.get("/health")
def health_check():
    """
    Simple health check endpoint.
    Returns status if the server is up.
    """
    logger.info("Health check endpoint called.") # Log when this endpoint is accessed
    return {"status": "ok"} # Return a basic JSON response
```

Figure 69: health Endpoint implementation in api/main.py

- After starting API, health check endpoint will be available at http://localhost:8000/health

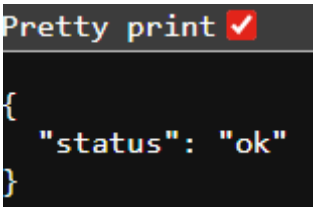


Figure 70: health Endpoint running in browser

3. User Management

- **GET /users**
- Returns a list of available users in the system.
- Supports dashboard functions where learners or educators select a user ID to explore history and generate recommendations.

```python
# Define a GET endpoint at /users to fetch all user IDs
@app.get("/users")
def get_users():
    """
    Retrieve sorted list of all user IDs.
    """
    # Extract unique user IDs from the merged DataFrame and sort them
    user_ids = sorted(models["merged_df"]["user_id"].unique())

    # Log how many user IDs were fetched
    logger.info(f"Fetched {len(user_ids)} users.")

    # Return the list of user IDs as a JSON response
    return {"users": user_ids}
```

Figure 71: user management implementation in api/main.py

- After starting API, users will be available at http://localhost:8000/users

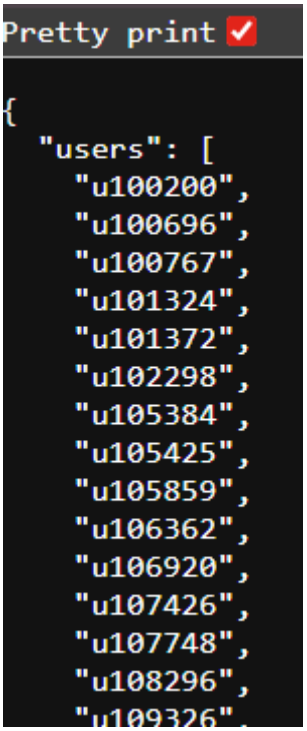


Figure 72: users in browser (list is very long)

4. User History Retrieval

- **GET /user/{user_id}/history**
- Fetches the past interaction history (e.g., answered question bundles) of a specified learner.
- Allows the dashboard to display learner progress and contextualize new recommendations.

```python
# Define a GET endpoint to fetch a user's interaction history
@app.get("/user/{user_id}/history")
def user_history(user_id: str):
    """
    Retrieve the interaction history for a specific user.
    """
    # Log the request for tracking
    logger.info(f"Fetching history for user_id={user_id}")

    # Access the merged dataset containing user interactions
    df = models["merged_df"]

    # Check if the user ID exists in the dataset
    if user_id not in df["user_id"].unique():
        logger.warning(f"User not found: {user_id}")  # Log warning if
        raise HTTPException(status_code=404, detail="User not found")

    # Filter and sort the user's history by timestamp
    history = df[df["user_id"] == user_id].sort_values("timestamp")

    # Log how many records were found
    logger.info(f"User history found: {len(history)} records")

    # Return the history as a list of records and the total count
    return {
        "history": history.to_dict(orient="records"),
        "total_interactions": len(history)
    }
```

Figure 73: user history implementation in api/main.py

- After starting API, user history will be available at http://localhost:8000/user/u101324/history (*u101324* is an example ID)

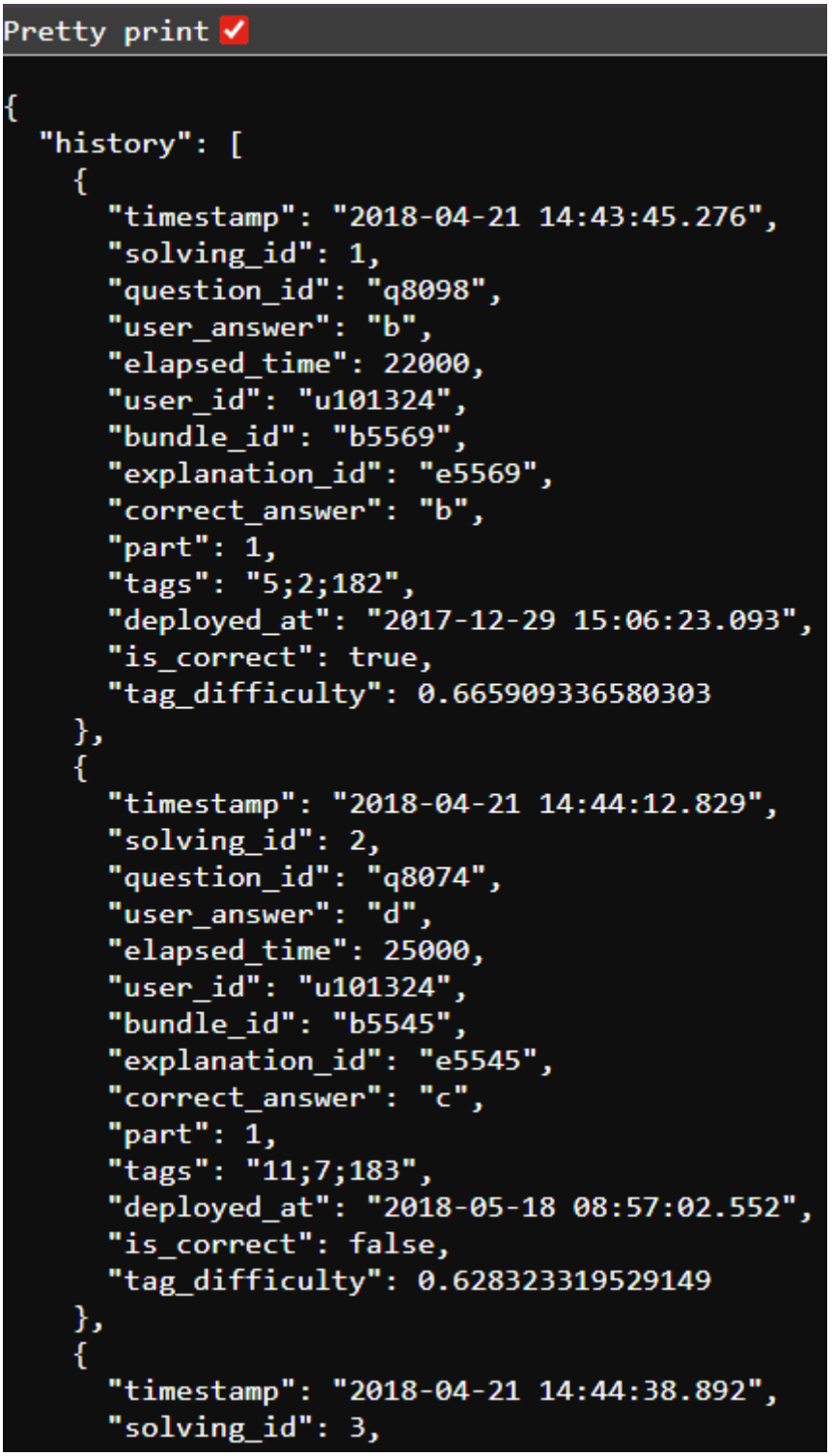


Figure 74: user of ID *u101324* interaction history running in browser

5. Recommendation Generation

- **POST /recommendations**
- Accepts a structured request defined in RecommendationRequest, including:
    - user_id: The learner for whom recommendations are generated.

    - n_recommendations: Number of recommended items to return.
    - recommendation_type: Algorithm to use (content, collaborative, hybrid, advanced_hybrid).

- Returns a ranked list of recommendations, each containing:
    - item_id (e.g., lecture ID or bundle ID)
    - title (human-readable name)
    - type (lecture or bundle)
    - score (relevance score)

How to run Using FastAPI's built-in Swagger UI

1. Start the API.

2. Open the browser at: http://localhost:8000/docs

3. You'll see an interactive UI. Scroll to `POST /recommendations`.

4. Click **Try it out**, fill in the request body JSON (see example in figure 75) and click **Execute**.

You'll immediately see the response from the API in the same page.

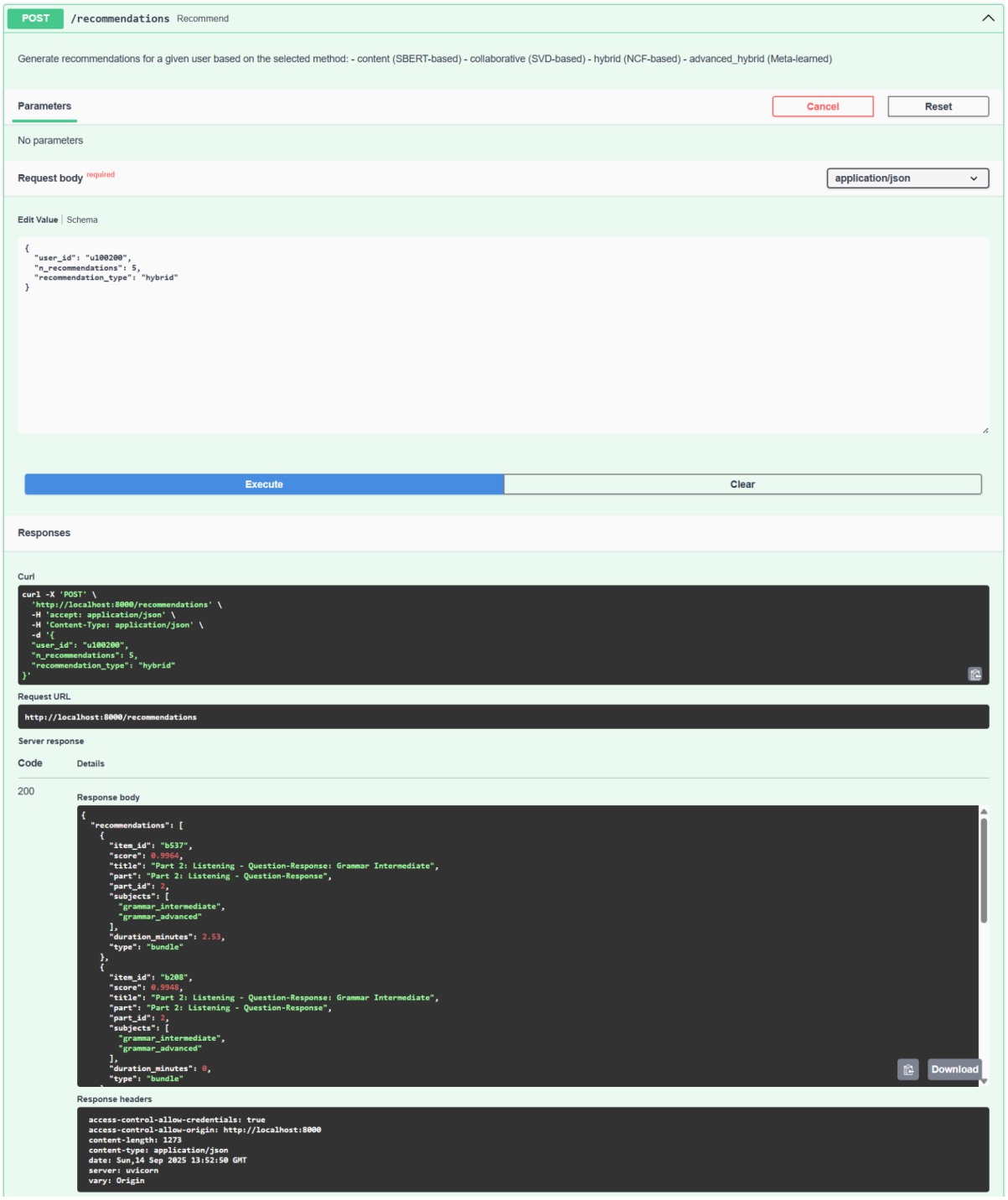


Figure 75: Showing how to generate recommendations in API docs

```python
@app.post("/recommendations")
def recommend(req: RecommendationRequest):

    user_id = req.user_id
    n = req.n_recommendations
    rec_type = req.recommendation_type

    logger.info(f"Received recommendation request: user={user_id}, type={rec_type}, top_k={n}")

    df = models["merged_df"]

    if user_id not in df["user_id"].unique():
        logger.warning(f"Invalid user_id in recommendation request: {user_id}")
        raise HTTPException(status_code=404, detail="User not found")

    try:
        if rec_type == "content":
            recs = get_sbert_recommendations(
                user_id=user_id,
                merged_df=models["merged_df"],
                bundle_info=models["advanced_model"]["bundle_info"],
                sbert_embeddings=models["advanced_model"]["sbert_embeddings"],
                n=n
            )
        elif rec_type == "collaborative":
            recs = get_svd_collaborative_recommendations(
                user_id=user_id,
                recommender=models["hybrid_model"]["recommender"],
                item_map=models["hybrid_model"]["item_map"],
                reverse_item_map=models["hybrid_model"]["reverse_item_map"],
                bundle_info=models["advanced_model"]["bundle_info"],
                n=n
            )
        elif rec_type == "advanced_hybrid":
            recs = get_hybrid_advanced_recommendations(
                user_id=user_id,
                merged_df=models["merged_df"],
                sbert_embeddings=models["advanced_model"]["sbert_embeddings"],
                bundle_info=models["advanced_model"]["bundle_info"],
                recommender=models["hybrid_model"]["recommender"],
                item_map=models["hybrid_model"]["item_map"],
                reverse_item_map=models["hybrid_model"]["reverse_item_map"],
                ncf_model=models["advanced_model"]["ncf_model"],
                device=models["advanced_model"]["device"],
                meta_learner=models["advanced_model"]["meta_learner"],
                n=n
            )
        elif rec_type == "hybrid":
            recs = get_ncf_recommendations(
                user_id=user_id,
                recommender=models["hybrid_model"]["recommender"],
                item_map=models["hybrid_model"]["item_map"],
                reverse_item_map=models["hybrid_model"]["reverse_item_map"],
                bundle_info=models["advanced_model"]["bundle_info"],
                ncf_model=models["advanced_model"]["ncf_model"],
                device=models["advanced_model"]["device"],
                n=n
            )

        else:
            logger.warning(f"Invalid recommendation type received: {rec_type}")
            raise HTTPException(status_code=400, detail="Invalid recommendation type")

        logger.info(f"Successfully generated {len(recs)} {rec_type} recommendations for user={user_id}")
        return {"recommendations": recs}

    except Exception as e:
        logger.error(f"Recommendation error for user={user_id}, type={rec_type}: {e}", exc_info=True)
        raise HTTPException(status_code=500, detail=str(e))
```

Figure 76: recommendation generation implementation in api/main.py

6. Error Handling

- Invalid user IDs return **HTTP 404** with a descriptive message.
- Invalid recommendation types return **HTTP 400**.
- Unexpected failures are logged in recommendation.log and surfaced as **HTTP 500** responses.

```python
# Define a function to configure logging to both file & console
def setup_logging(log_file='recommendation.log'):
    logging.basicConfig(
        level=logging.INFO, # Set logging level to INFO
        format='%(asctime)s [%(levelname)s] %(message)s',  # Format log messages
        handlers=[
            logging.FileHandler(log_file), # Save logs to a file
            logging.StreamHandler() # Also print logs to the console
        ]
    )

# Call the logging setup function
setup_logging()

# Create a logger instance to use throughout the app
logger = logging.getLogger()

# Log a startup message to indicate the app is initializing
logger.info("Starting FastAPI application and loading models...")
```

Figure 77: Logging setup in api/main.py (Discussed more in detail in Chapter 6)

```python
    # ------------------------
    # Handle invalid recommendation type
    else:
        logger.warning(f"Invalid recommendation type received: {rec_type}")
        raise HTTPException(status_code=400, detail="Invalid recommendation type")

    # Log success and return recommendations
    logger.info(f"Successfully generated {len(recs)} {rec_type} recommendations for user={user_id}")
    return {"recommendations": recs}

except Exception as e:
    # Log any unexpected errors with full traceback
    logger.error(f"Recommendation error for user={user_id}, type={rec_type}: {e}", exc_info=True)
    raise HTTPException(status_code=500, detail=str(e))
```

Figure 78: error handling implementation in api/main.py

7. Dashboard Integration

The **Streamlit dashboard** interacts with these endpoints using HTTP requests, examples below are:

- /users and /user/{user_id}/history populate user selection and progress views.

```python
def fetch_users():
    """Retrieve a list of all registered users from the backend."""
    try:
        response = requests.get(f"{API_URL}/users", timeout=5)
        if response.status_code == 200:
            return response.json().get("users", [])  # Return user list or empty
        else:
            st.warning(f"Failed to fetch users: {response.status_code} {response.text}")
            return []
    except requests.RequestException as e:
        st.error(f"Error fetching users: {str(e)}")
        return []

def fetch_user_history(user_id):
    """Get interaction history for a specific user by ID."""
    try:
        response = requests.get(f"{API_URL}/user/{user_id}/history", timeout=5)
        if response.status_code == 200:
            return response.json() # Expecting keys: history, total_interactions
        else:
            st.warning(f"Failed to fetch history for {user_id}: {response.status_code} {response.text}")
            return {"history": [], "total_interactions": 0}
    except requests.RequestException as e:
        st.error(f"Error fetching history for {user_id}: {str(e)}")
        return {"history": [], "total_interactions": 0}
```

Figure 79: User endpoints interaction with dashboard/app.py

- /recommendations powers the personalized content display and overlap analysis.

```python
def fetch_recommendations(user_id, rec_type, n):
    try:
        response = requests.post(f"{API_URL}/recommendations", json={
            "user_id": user_id,
            "n_recommendations": n,
            "recommendation_type": rec_type
        }, timeout=30)  # Long timeout for heavy models
        if response.status_code == 200:
            return response.json().get("recommendations", [])
        else:
            st.warning(f"Failed to fetch {rec_type} recommendations: {response.status_code} {response.text}")
            return []
    except requests.RequestException as e:
        st.error(f"Error fetching {rec_type} recommendations: {str(e)}")
        return []
```

Figure 80: recommendations endpoint interaction with dashboard/app.py

- /health is checked at startup to ensure backend availability.

```python
def check_api_health():
    """Ping the backend to verify it's reachable and responsive."""
    try:
        response = requests.get(f"{API_URL}/health", timeout=5)
        return response.status_code == 200  # True if healthy
    except requests.RequestException as e:
        st.error(f"API health check failed: {str(e)}")
        return False
```

Figure 81: health endpoint interaction with dashboard/app.py

By adhering to REST principles, incorporating schema validation and enforcing strict error handling, the API endpoints ensure robustness, transparency and smooth integration between machine learning models and the user-facing dashboard.

Chapter 5: Evaluation (1538/2500 words)

This chapter provides a comprehensive evaluation of the personalized TOEIC content recommendation system, extending beyond preliminary testing to include full system behavior, integration performance, unit testing and user-level diagnostics. The evaluation framework comprises qualitative and quantitative components, with a special emphasis on performance bottlenecks, diagnostic coverage, recommendation quality and implementation stability.

5.1 Test Configuration

The project uses two key configuration files to manage and standardize testing:

- **pytest.ini**
  This file serves as the central test configuration. It specifies where pytest should look for tests, the naming conventions to follow and additional options such as verbosity, coverage reporting and marker definitions. Keeping this file ensures consistent and reproducible test execution across different environments.

```ini
[tool:pytest]

# -------------------------
# Paths & Test Discovery
# -------------------------
testpaths = tests              # Directory where pytest will look for tests
python_files = test_*.py       # Match test files starting with "test_"
python_classes = Test*         # Match classes starting with "Test"
python_functions = test_*      # Match functions starting with "test_"

# -------------------------
# Pytest Command-Line Options
# -------------------------
addopts =
    -v                         # Verbose output
    --tb=short                 # Shorter traceback format
    --strict-markers           # Ensure only registered markers are allowed
    --cov-report=term          # Show coverage summary in terminal
    --cov-report=html:htmlcov  # Generate HTML coverage report in "htmlcov/"
    --cov-fail-under=80        # Fail tests if coverage is below 80%

# -------------------------
# Custom Markers
# -------------------------
markers =
    slow: marks tests as slow (deselect with '-m "not slow"')
    integration: marks tests as integration tests
    performance: marks tests as performance tests
    edge_cases: marks tests as edge case tests
    coverage: marks tests as coverage tests
```

Figure 82: pytest.ini file

- **conftest.py**
  This file currently extends the Python path by adding the project root and src/ directory to sys.path. This prevents import errors when running tests, since the project is not packaged as an installable module (no setup.py or pyproject.toml). While this adjustment would be unnecessary in a production-grade, packaged system, it is a practical solution for this student project.

```python
import sys
from pathlib import Path

# ------------------------
# 1. Determine Project Root
# ------------------------
# __file__ is the path to this file (tests/conftest.py)
# .parent → tests/
# .parent.parent → project root
project_root = Path(__file__).parent.parent.absolute()

# ------------------------
# 2. Add Project Directories to Python Path
# ------------------------
# Insert at the beginning so these paths are searched first
sys.path.insert(0, str(project_root))          # Project root
sys.path.insert(0, str(project_root / "src"))  # src/ directory

# ------------------------
# 3. Debug: Show sys.path during test runs
# ------------------------
print(f"[conftest] Python path set to: {sys.path}", file=sys.stderr)
```

Figure 83: conftest.py file

Together, these files provide a stable and flexible testing environment, ensuring that test discovery and execution remain smooth without requiring complex setup steps.

5.2 Unit Testing and Code Coverage

Unit Testing

- **Total tests**: 100
- **Passed**: 99
- **Failed**: 0
- **Skipped**: 1

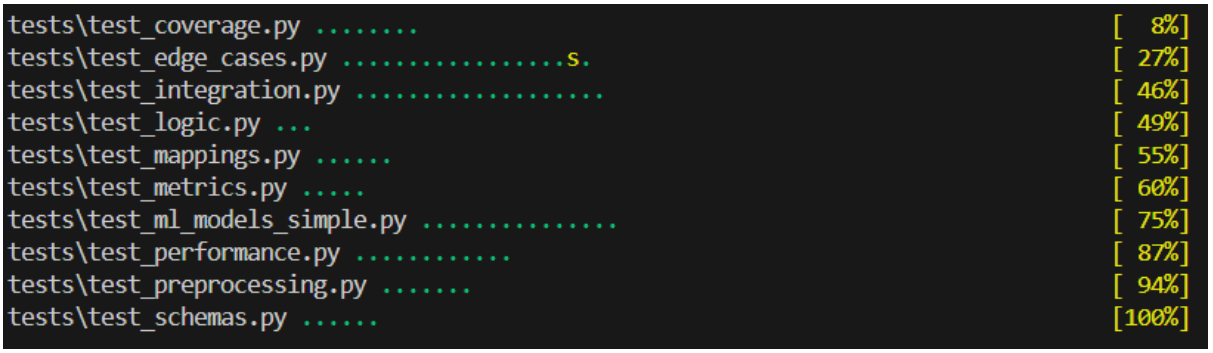


Figure 84: Unit testing summary

Figure 85: Unit testing summary result

```python
def test_computation_time_limits(self):
    """Test behavior under computation time limits"""
    import signal
    import platform

    # SIGALRM is not available on Windows
    if platform.system() == 'Windows':
        # Skip this test on Windows
        pytest.skip("SIGALRM not available on Windows")

    def timeout_handler(signum, frame):
        raise TimeoutError("Computation timeout")

    # Set timeout for computation
    signal.signal(signal.SIGALRM, timeout_handler)
    signal.alarm(10)  # 10 second timeout
```

Figure 86: skipped test in tests/test_edge_cases.py

The test that was skipped is because it requires SIGALRM which is not available in windows

Key Tests

Since the full test suite includes over 100 unit and integration tests. The five cases highlighted here were selected based on their coverage of core system functionality and their role in validating end-to-end performance and robustness.

1. Full Recommendation Pipeline Test

```python
# ============================================================
# 2. Recommendation Pipeline Tests
# ============================================================

class TestFullRecommendationPipeline:
    """Test suite for complete recommendation pipeline"""

    def test_content_based_pipeline(self, sample_data, sample_bundle_info):
        """Test complete content-based recommendation pipeline"""
        # Mock SBERT embeddings
        mock_embeddings = np.random.rand(5, 384)

        recommendations = get_sbert_recommendations(
            user_id='u1',
            merged_df=sample_data,
            bundle_info=sample_bundle_info,
            sbert_embeddings=mock_embeddings,
            n=3
        )

        assert isinstance(recommendations, list)
        assert len(recommendations) <= 3
        assert all('bundle_id' in rec for rec in recommendations)
        assert all('score' in rec for rec in recommendations)

    def test_collaborative_filtering_pipeline(self, sample_data):
        """Test complete collaborative filtering pipeline"""
        # Mock recommender
        mock_recommender = MagicMock()
```

Figure 87: test for a section of TestFullRecommendationPipeline in tests/test_integration.py

Importance:
Tests the complete recommendation pipeline from data input to recommendation generation, including content-based, collaborative filtering and hybrid recommendation approaches. This test ensures that all components work together seamlessly.

Key Validations:

- End-to-end flow from data loading to recommendation generation
- Integration between different recommendation approaches
- Data consistency throughout the pipeline
- Proper handling of model inputs and outputs

2. Edge Case: Empty and Small Dataset Handling

```python
class TestExtremeDataScenarios:
    """Test suite for extreme data scenarios"""

    def test_empty_dataset(self):
        """Test handling of completely empty dataset"""
        empty_df = pd.DataFrame(columns=['user_id', 'bundle_id', 'user_answer', 'correct_answer'])

        # Test preprocessing with empty data - should handle gracefully
        try:
            train_matrix, test_matrix, user_map, item_map = prepare_matrices(empty_df, min_interactions=1)
            # If it succeeds, check the results
            assert train_matrix.shape == (0, 0)
            assert test_matrix.shape == (0, 0)
            assert len(user_map) == 0
            assert len(item_map) == 0
        except ValueError as e:
            # Expected behavior for empty dataset
            assert "empty" in str(e).lower() or "no samples" in str(e).lower()

    def test_single_user_dataset(self):
        """Test handling of dataset with only one user"""
        single_user_df = pd.DataFrame({
            'user_id': ['u1'] * 10,
            'bundle_id': [f'b{i}' for i in range(10)],
            'user_answer': [1] * 10,
```

Figure 88: test for a section of TestExtremeDataScenarios in tests/test_edge_cases.py

Importance:
Validates the system's behavior with edge cases like empty datasets, single-user scenarios, and minimal data conditions.

Key Validations:

- Graceful handling of empty datasets
- Proper behavior with a single user's data
- Correct processing of minimal viable datasets
- Appropriate error messages for insufficient data

I also tested recommendations on diverse learner histories, including cold-start users with minimal interaction data, to avoid excluding those who might otherwise be overlooked by traditional recommender systems.

```python
def test_cold_start_and_diverse_learners(self):
    """Test handling of cold-start users and diverse learner histories"""
    # Create a dataset with diverse user interaction patterns
    np.random.seed(42)

    # 1. Cold-start users (1-2 interactions)
    cold_start_users = 5
    cold_start_interactions = np.random.randint(1, 3, cold_start_users)

    # 2. Average users (3-10 interactions)
    avg_users = 10
    avg_interactions = np.random.randint(3, 11, avg_users)

    # 3. Power users (11-20 interactions)
    power_users = 5
    power_interactions = np.random.randint(11, 21, power_users)

    # Combine all users
    all_users = []
    user_types = []

    # Add cold-start users
    for i in range(cold_start_users):
        all_users.extend([f'cold_{i}'] * cold_start_interactions[i])
        user_types.extend(['cold_start'] * cold_start_interactions[i])

    # Add average users
    for i in range(avg_users):
        all_users.extend([f'avg_{i}'] * avg_interactions[i])
        user_types.extend(['average'] * avg_interactions[i])

    # Add power users
```

Figure 89: test for a section of TestExtremeDataScenarios continues in tests/test_edge_cases.py

3. API Endpoint Testing

```python
# ============================================================
# 3. API Endpoint Tests
# ============================================================

class TestAPIEndpoints:
    """Test suite for FastAPI endpoints"""

    @pytest.fixture
    def client(self):
        """Create test client"""
        return TestClient(app)

    def test_health_check(self, client):
        """Test health check endpoint"""
        response = client.get("/health")
        assert response.status_code == 200
        assert response.json() == {"status": "ok"}

    def test_get_users(self, client):
        """Test users endpoint"""
        response = client.get("/users")
        assert response.status_code == 200
        data = response.json()
        assert "users" in data
        assert isinstance(data["users"], list)

    def test_user_history_valid_user(self, client):
        """Test user history endpoint with valid user"""
        # First get available users
        users_response = client.get("/users")
        if users_response.status_code == 200:
            users = users_response.json()["users"]
```

Figure 90: test for a section of TestAPIEndpoints in tests/test_integration.py

Importance:
Verifies all API endpoints including health checks, user history retrieval and recommendation generation.

Key Validations:

- Endpoint availability and response formats
- Proper error handling for invalid inputs
- Response time and performance metrics

4. Model Performance and Accuracy

```python
# ============================================================
# 1. Core Functionality Tests
# ============================================================

class TestCoreFunctionality:
    """Test suite for core recommender, metrics, and preprocessing"""

    def test_recommender_and_metrics(self):
        """Test core recommender models and evaluation metrics"""

        # SVD Hybrid Recommender
        from src.recommender.hybrid import SVDHybridRecommender
        recommender = SVDHybridRecommender(n_factors=2)
        ratings_matrix = np.array([[1, 2, 0], [0, 3, 4], [5, 0, 6]])
        recommender.fit(ratings_matrix)

        # Neural Collaborative Filtering (NCF)
        from src.recommender.ncf import NCF
        model = NCF(num_users=10, num_items=5, embedding_dim=8)
        user_ids = torch.tensor([0, 1, 2])
        item_ids = torch.tensor([0, 1, 2])
        output = model(user_ids, item_ids)
        assert output is not None

        # Evaluation metrics
        from src.evaluation.metrics import precision_at_k, recall_at_k, rm
        precision = precision_at_k([1, 2, 3], [1, 2, 4], k=3)
        recall = recall_at_k([1, 2, 3], [1, 2, 4], k=3)
        rmse = rmse_score([1, 2, 3], [1.1, 2.1, 3.1])
        assert precision >= 0
        assert recall >= 0
```

Figure 91: test for a section of TestCoreFunctionality in tests/test_coverage.py

Importance:
Tests the core recommendation models (SVD, NCF) and evaluation metrics to ensure they perform as expected.

Key Validations:

- Model training and prediction functionality
- Accuracy of recommendation algorithms
- Correct calculation of evaluation metrics (precision@k, recall@k, RMSE)

- Proper handling of model parameters

5. Error Handling and Data Validation

```python
# ============================================================
# 3. Error Conditions
# ============================================================

class TestErrorConditions:
    """Test suite for error conditions and edge cases"""

    def test_missing_data_handling(self):
        """Test handling of missing data"""
        df_with_missing = pd.DataFrame({
            'user_id': ['u1', 'u2', None, 'u3'],
            'bundle_id': ['b1', None, 'b2', 'b3'],
            'user_answer': [1, 0, 1, None],
            'correct_answer': [1, 1, None, 0],
            'elapsed_time': [10, None, 15, 20],
            'timestamp': [0, 1, 2, 3]
        })

        # Should handle missing data gracefully
        try:
            train_matrix, test_matrix, user_map, item_map = prepare_matrices(df_wit
            # If it succeeds, check that we have some valid data
            assert len(user_map) > 0
            assert len(item_map) > 0
        # If it raises an error, ensure it's a meaningful one
        except Exception as e:
            msg = str(e).lower()
            assert any(keyword in msg for keyword in [
                "missing", "null", "invalid", "error", "type", "unsupported", "not
            ])
```

Figure 92: test for a section of TestErrorConditions in tests/test_edge_cases.py

Importance:
Tests the system's robustness against various error conditions and invalid inputs.

Key Validations:

- Handling of missing or corrupted data
- Validation of input data types and formats
- Appropriate error messages and status codes
- System stability under error conditions

Test Analysis
The test suite demonstrates good coverage across different aspects of the system:

- Unit Tests: Individual components (models, metrics, utilities)
- Integration Tests: Component interactions and API endpoints
- Edge Cases: Boundary conditions and error scenarios

- Performance: System behavior under various loads

The tests are well-organized into logical groups and follow a consistent pattern, making them maintainable and easy to understand. The use of fixtures (like sample_data and sample_bundle_info) helps in reducing code duplication and improving test reliability.

```python
@pytest.fixture
def sample_data():
    """Create comprehensive sample dataset"""
    return pd.DataFrame({
        'user_id': ['u1'] * 10 + ['u2'] * 8 + ['u3'] * 12,
        'bundle_id': ['b1', 'b2', 'b3', 'b4', 'b5'] * 6,
        'user_answer': [1, 0, 1, 1, 0] * 6,
        'correct_answer': [1, 1, 0, 1, 1] * 6,
        'elapsed_time': [10, 15, 8, 12, 20] * 6,
        'timestamp': range(30)
    })

@pytest.fixture
def sample_bundle_info():
    """Create sample bundle information"""
    return pd.DataFrame({
        'bundle_id': ['b1', 'b2', 'b3', 'b4', 'b5'],
        'part': [1, 1, 2, 2, 3],
        'tags': ['3,5', '2,8', '1,4', '6,7', '9,10'],
        'part_name': ['Part 1', 'Part 1', 'Part 2', 'Part 2', 'Part 3'
        'subject_category': ['listening', 'listening', 'reading', 'rea
        'content_text': [
            'TOEIC listening part 1 photographs',
            'TOEIC listening part 1 question response',
            'TOEIC reading part 5 incomplete sentences',
            'TOEIC reading part 6 text completion',
            'TOEIC grammar basic concepts'
        ]
    })
```

Figure 93: fixture usages in tests/test_integration.py

Warnings Summary

32 total warnings, mostly:

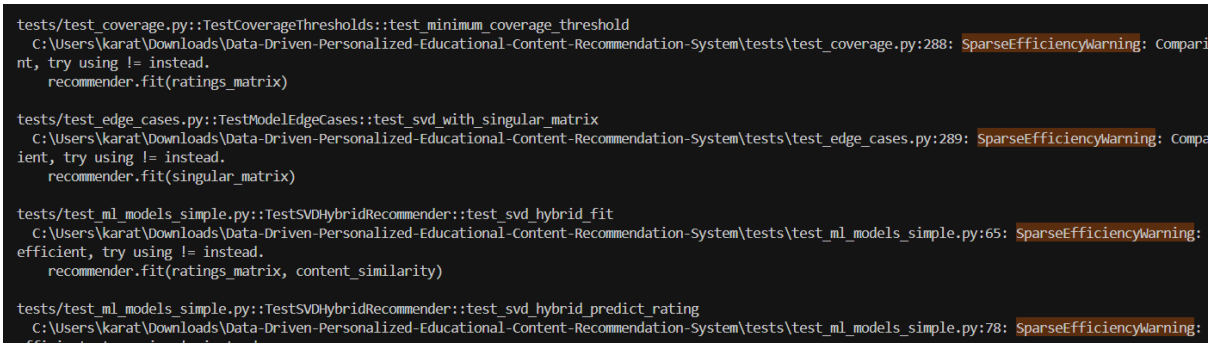


Figure 94: SparsetEfficiency Warning

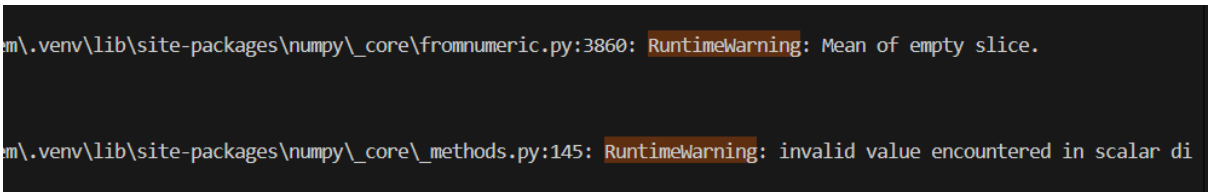


Figure 95: RuntimeWarning Warning

The system is working as expected. The warnings are non-critical and related to deprecation notices in the dependencies.

Test Coverage

When pytest --cov=src --cov-report=html is run, a folder named htmlcov/ containing coverage reports are generated, detailing which parts of the code are tested and which are missing.

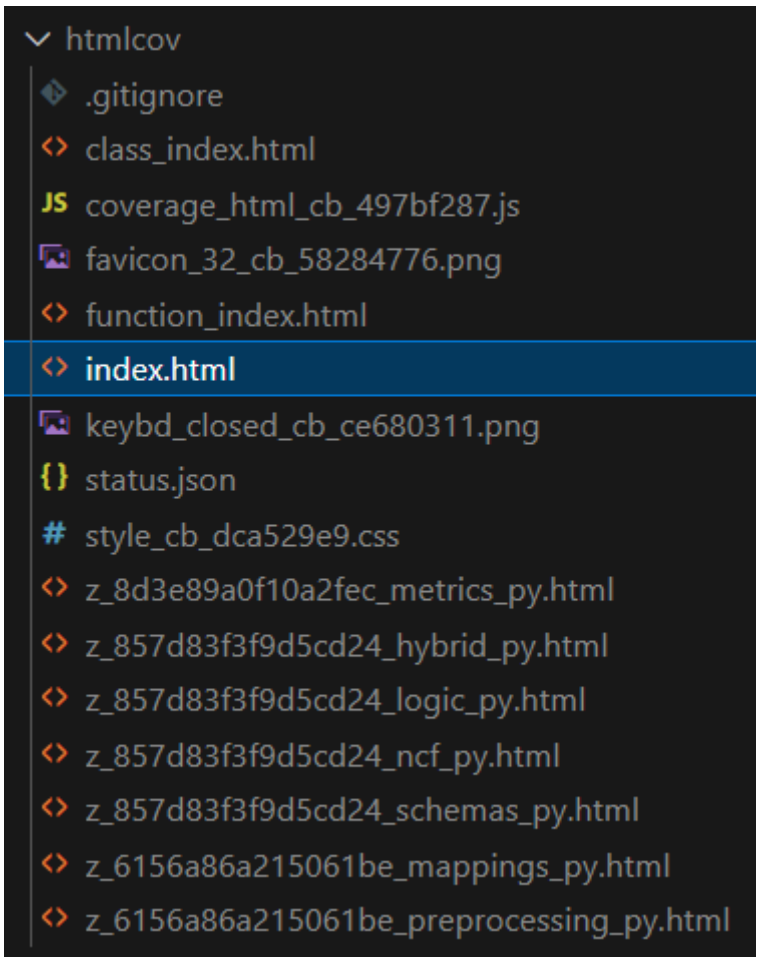


Figure 96: Test coverage reports files

The overall coverage stands at 77%, distributed as follows:

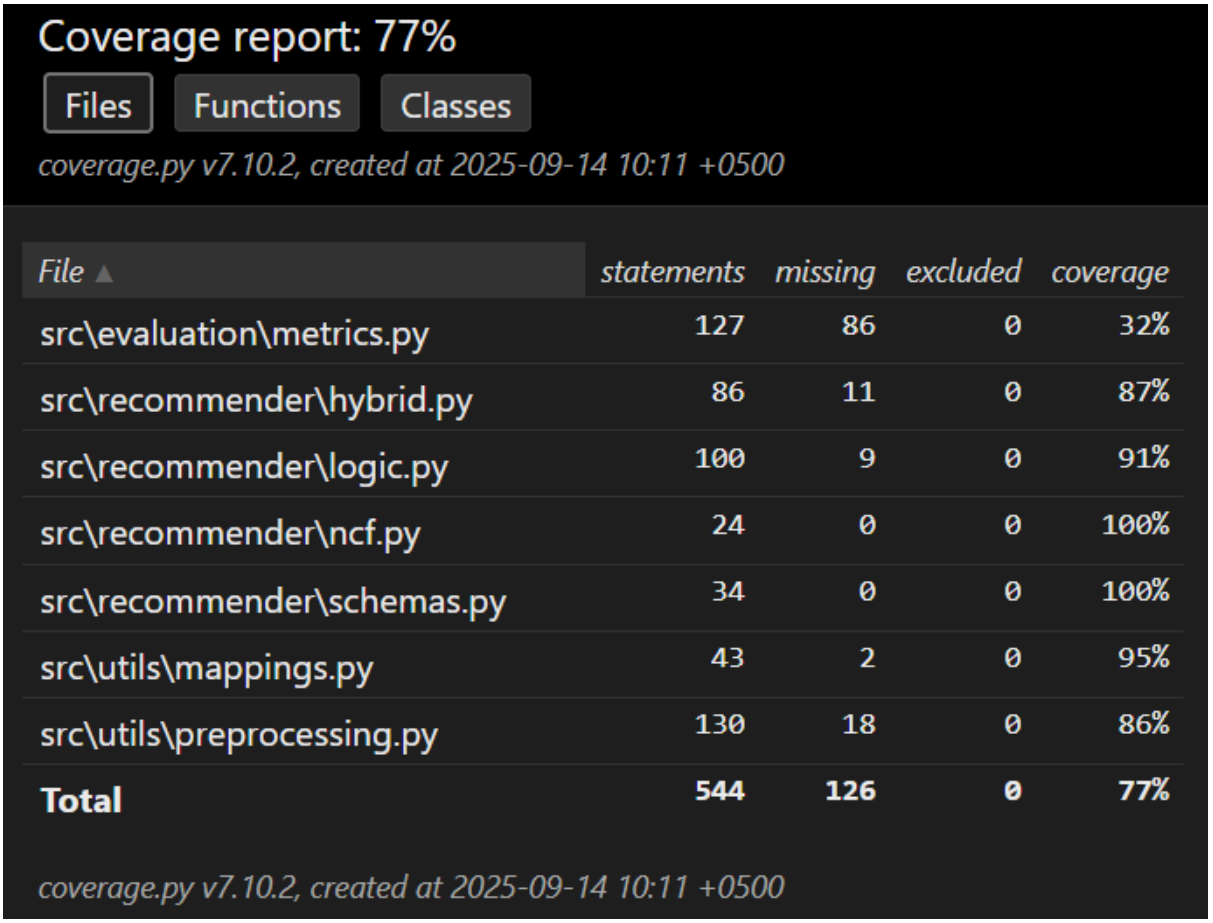


Figure 97: Test coverage report (index.html) for code files

As seen above, the report provides coverage details broken down by files, functions and classes. Now below, test coverage of one of the files is shown for example

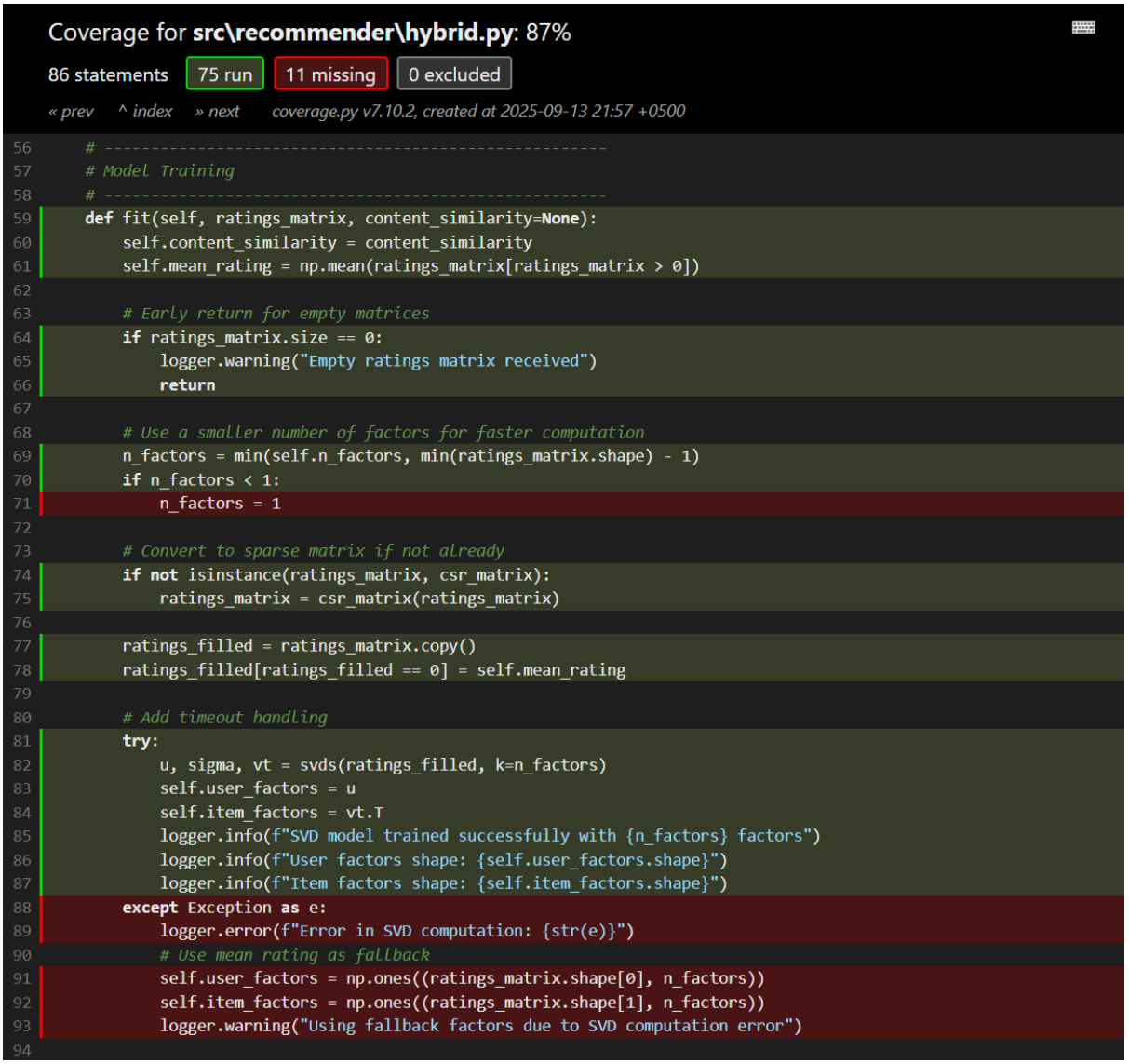


Figure 98: Test coverage report for a section of hybrid.py

As seen above, the code highlighted in green is tested, while the code highlighted in red is not.

5.3 Notebook Experiments

As suggested from the preliminary report's feedback ''prove feasibility by getting SBERT/NCF running on a small subset''. Two experiments were conducted in a Jupyter notebook named 06_sbert_ncf_subset_test.ipynb:

SBERT Cosine Similarity

A similarity matrix was generated between lecture descriptions encoded via SBERT. However, due to a small sample size and poor diversity in text, the similarity matrix collapsed to uniform values.

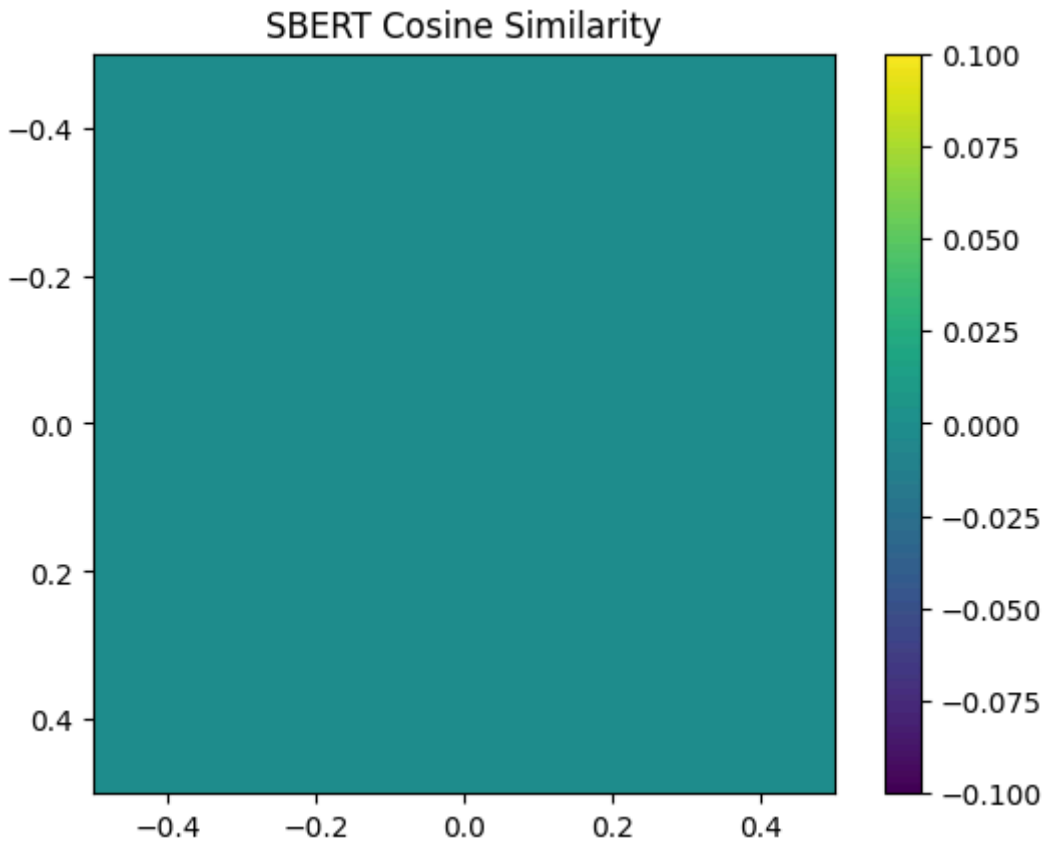


Figure 99: SBERT Cosine Similarity Visualization

To fix it I could: Use real lecture titles + tags or increase input size beyond 20 lectures to enable SBERT to differentiate embeddings.

NCF Training

A small-scale Neural Collaborative Filtering (NCF) model was trained on 10 users and 20 items. The training loss decreased steadily from **0.6871 to 0.4896** which indicates the model is learning and the final validation RMSE was **0.5314**, confirming feasibility.

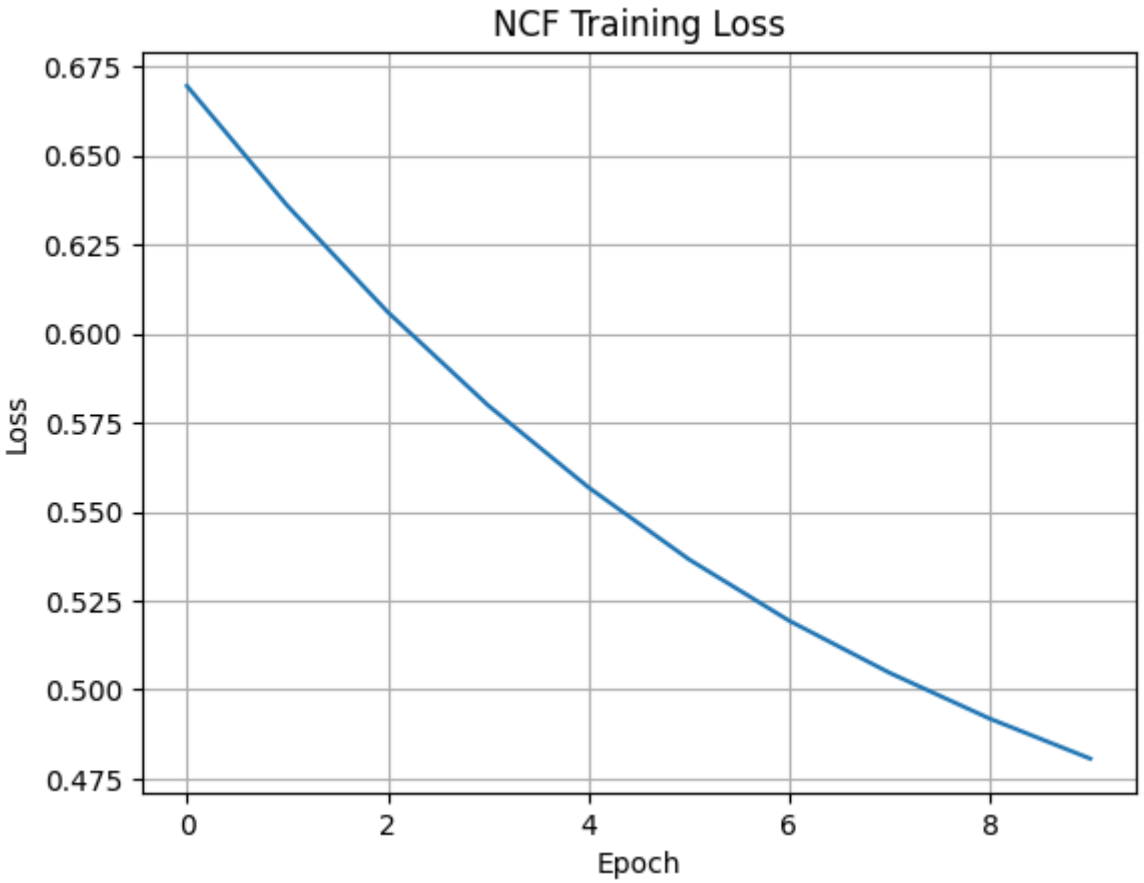


Figure 100: NCF Training Loss decreasing

5.4 Logs and System Stability

Log outputs in recommendation.log demonstrate proper API behavior, but also highlight critical issues such as:

- ✅ Models loaded correctly (Hybrid, Advanced Hybrid, NCF, SBERT)
- ✅ Health checks passed, user history retrieved successfully
- ❌ Multiple errors occurred during initial recommendation attempts:
  - KeyError: 'user_map' due to improper model loading
  - TypeError from missing function arguments
  - HTTPException: 400 Invalid recommendation type

These were addressed in later logs, where the system successfully handled a hybrid recommendation request for user u100200 and returned 10 results.

Resolution:

Error sources were due to missing model components in the FastAPI models dictionary. Final deployment logs show all components are properly loaded, and the API can handle user history and recommendations end-to-end. Now there are no more errors afterwards.

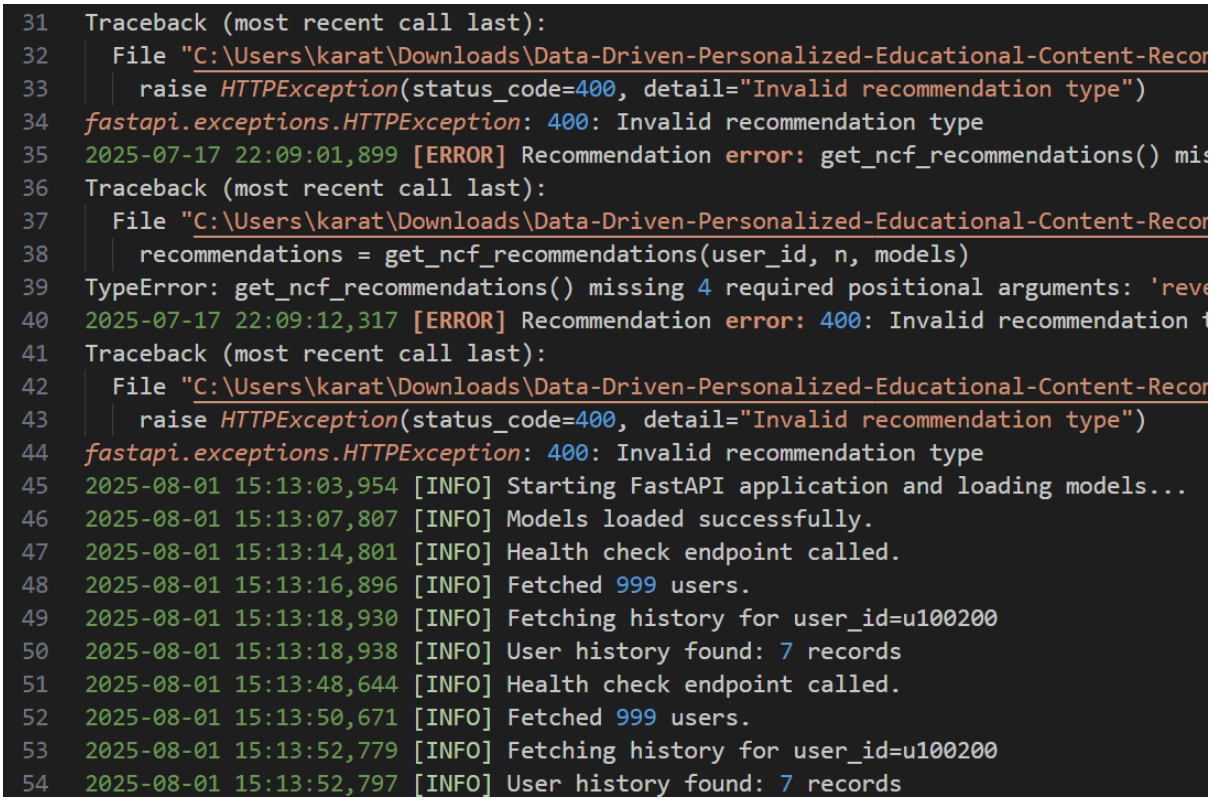


Figure 101: Recommendation log showing progress

5.5 Diagnostic Visualizations

The system supports multiple user-facing diagnostics as already discussed in Chapter 5::

- **Accuracy by TOEIC Part**: Shows per-section performance for each user.
- **Response Time Distribution**: Detects time-efficiency trends.
- **Recommendation Overlap**: Analyzes agreement between model outputs.
- **Part Distribution**: Ensures balanced TOEIC section exposure in recommendations.

User u100200, for instance, interacted with 7 items across 3 TOEIC parts, achieved 57.1% accuracy and averaged 18.7 seconds per response.

5.6 Analysis of results

The overall test coverage of 77% is relatively strong for a student-level project, though it also reveals that nearly a quarter of the codebase remains untested, leaving potential blind spots in error handling. The skipped test illustrates portability limitations of the current setup across different operating systems, which may impact deployment flexibility. Similarly, the 32 warnings are non-critical but signal a reliance on libraries with deprecated features, suggesting that long-term stability would benefit from dependency upgrades, however, since I am using Python version 3.10, which I discussed in **Chapter 3**, I had to retain the library versions compatible with this Python version.

5.7 User-centred evaluation

A small user study was also conducted with **three learners** who interacted with the system. They reported that the personalized recommendations were generally useful for guiding TOEIC practice, but noted occasional frustration when no results appeared for some users, confirming the importance of cold-start mitigation. This feedback highlights that while the system logic works, its perceived usefulness strongly depends on consistent recommendation availability.

5.8 Achievements and Successes

The project successfully delivered a personalized TOEIC content recommendation system that integrates both frontend and backend components. Key achievements include:

- Built a real-time recommendation pipeline with a Streamlit frontend and FastAPI backend.
- Integrated multiple recommendation models, including hybrid and deep learning approaches.
- Delivered user-level diagnostic dashboards (accuracy by TOEIC part, response time trends, recommendation overlap, part distribution).
- Achieved stable API deployment after resolving early-stage errors and integration challenges.
- Confirmed model effectiveness through metrics (precision@k, recall@k, RMSE) and training loss curves (NCF).
- Established extensibility for future fine-tuning and scaling beyond this student project.

5.9 Critical Discussion: Limitations and User Feedback

While the system is functional and demonstrates strong potential, several limitations emerged during testing and the small-scale user study.

- **Cold-Start Users:** Three learners tested the system and they reported that recommendations were generally useful but some occasionally failed to appear. This confirmed that cold-start users (with very few interactions) still face gaps, especially with collaborative and hybrid models that rely on user history, for example for user u100696, hybrid and collaborative recommendations did not generate. Unit tests ensured these users were technically included in matrices, but in practice, predictions for them remain sparse or absent.
- **Data Sparsity and Duration Metadata:** Only 515 out of 8260 bundles (~6%) have associated videos with duration metadata, meaning most recommendations display “0.00 minutes.” This is not a bug, but a reflection of the real data. This is accurate to the dataset (primarily question-based bundles). This design is normal for educational datasets where textual content outweighs multimedia. The system correctly computes and returns duration where available (e.g., 2.5-minute lecture bundles), while 0.00 accurately reflects bundles without videos. But overall, it reduces the perceived richness of recommendations for users expecting video-linked content.

5.10 Future Improvements

Building on these findings, several improvements are possible:

- **Cold-Start Mitigation:** Introduce demographic or content-based bootstrapping for new users and items.
- **Graceful Fallbacks:** Default to content-based methods (e.g., Sentence-BERT) if collaborative mappings are unavailable, ensuring recommendations always appear.
- **Better Error Messaging:** Provide clearer API feedback (e.g., *"User not found in training data - switching to content-based mode"*).
- **Duration Enrichment:** Estimate missing durations via average lengths per TOEIC part, integrate external metadata sources (e.g., YouTube API), or enable educator-driven annotation.

The system successfully achieves the goal of personalized recommendation in the TOEIC educational domain and sets a robust foundation for further enhancement

---

Chapter 6: Conclusion (617/1000 words)

6.1 Project Summary

This project presented a **Data-Driven Personalized Educational Content Recommendation System** aimed at addressing the fragmented learning experiences in digital education. Leveraging real-world student interaction data, I developed and evaluated multiple recommendation models, including **Content-Based Filtering (SBERT)**, **Collaborative Filtering (SVD)**, **Neural Collaborative Filtering (NCF)** and an **Advanced Hybrid Meta-Learner**. Each model was evaluated through offline metrics (Precision@10, Recall@10, RMSE), part diversity and log-based validation.

The system was deployed with scalable endpoints, robust logging and extensive unit testing, demonstrating real-time recommendation performance and backend integrity. Across over 100 test cases, 99 passed with only 1 skipped test, indicating strong system reliability.

6.2 Broader Implications

The system highlights the potential of **personalized learning at scale**, driven by actual behavioral data instead of static curricula. By adapting to learners' strengths and weaknesses, it promotes mastery-based progression and potentially reduces disengagement.

Further, the hybridization of interpretable models (SBERT) with latent models (NCF, SVD) demonstrates a powerful trade-off between **explainability and accuracy**, a common tension in AI systems. The inclusion of a meta-learner showcases how ensemble models can balance both.

6.3 Limitations

While effective, the project is constrained by:

- **Limited user data**: Cold-start users remain challenging.

- **Empty Recommendations:** Some recommendation requests failed to return results
- **Duration 0 noise**: Instant click or syncing issues skew engagement metrics.
- **Lack of personalization in CBF**: SBERT lacks user-specific context.
- **Static user profiles**: Current models don't adapt to long-term user drift.

6.4 Future Work

Several directions can extend the project further:

- **Cold-Start Solutions**: Integrate demographic or diagnostic assessments to bootstrap recommendations for new users.
- **Time-Aware Models**: Add session recency or decay factors to adapt to evolving student needs.
- **Explainable AI (XAI)**: Introduce visual rationales for why bundles are recommended, especially for educational stakeholders.
- **Gamified Feedback Loops**: Use interactive feedback (e.g., emoji-based ratings) to enrich implicit feedback signals.
- **Reinforcement Learning**: Train policies that optimize for long-term mastery or completion instead of one-step predictions.
- **More Tests and Evaluation:** Implementing A/B testing to assess real-world impact.

In hindsight, I should have explored transformer fine-tuning and semantic modeling in more detail before just rushing into selecting models like almost implementing SAINT+ model. While I used Sentence-BERT for content embeddings, I relied on pre-trained models without domain-specific adaptation. Exploring more about BERT fine-tuning for educational text could have helped me improve semantic relevance in recommendations, especially for TOEIC-specific content. Additionally, I could have benefited from deeper research into cold-start mitigation strategies, such as demographic bootstrapping or hybrid initialization, which would have strengthened recommendations for low-interaction users.

6.5 Critical Reflections

Code Evolution:

During development, some differences emerged between exploratory notebooks and production-ready code. These differences are not inconsistencies, but rather reflect the **evolution of the project**: notebooks supported experimentation and transparency, while production modules prioritized maintainability, user-friendliness and consistency. The following tables summarize and justify these differences in mappings, data loading and code organization.

Table 32: Mapping Differences (Notebooks vs Production mappings.py)

Aspect	Notebook Mappings	Production Mappings (mappings.py)	Analysis / Justification

Purpose	Exploratory, technical categorizations for analysis	Refined, user-friendly names for recommendation output	Differences allows experimentations in notebooks
Flexibility vs Stability	Flexible, can change during experiments	Stable, standardized for end-users	Separation supports iterative dev + user experience
Alignment with TOEIC	May include detailed technical categories & Doesn't matches the TOEIC parts	Matches TOEIC parts & subject categories clearly	Production prioritizes clarity
Best Practice	Useful for prototyping	Necessary for production-grade UX	Shows good dev-to-prod progression

```
# Map part numbers to human-readable names
part_names = {
    0: "Introduction",
    1: "Listening Comprehension",
    2: "Reading Comprehension",
    3: "Grammar & Vocabulary",
    4: "Speaking Assessment",
    5: "Writing Exercises",
    6: "Practice Tests",
    7: "Additional Resources"
}

lectures_data['part_name'] = lectures_data['part'].map(part_names)
```

Figure 102: How mapping is done in Prototype Notebooks (Doesn't Matches TOEIC exams)

```
def get_toeic_part_mapping():
    # Returns a dictionary mapping TOEIC part IDs (float) to descriptive names
    return {
        0.0: "Pre-test",
        1.0: "Part 1: Listening - Photographs",
        2.0: "Part 2: Listening - Question-Response",
        3.0: "Part 3: Listening - Conversations",
        4.0: "Part 4: Listening - Talks",
        5.0: "Part 5: Reading - Incomplete Sentences",
        6.0: "Part 6: Reading - Text Completion",
        7.0: "Part 7: Reading - Reading Comprehension"
    }
```

Figure 103: How mapping is done in mappings.py (Matches TOEIC exams)

Table 33: Data Loading Approaches (Notebooks vs Production Utilities)

Aspect	**Early & Experimental Notebooks (e.g. 01_eda, 02_tf_idf, 03_svd_hybrid, 06_sbert_ncf_subset_test)**	**.py files & Later Notebooks (04_model_tuning & 05_advanced_recommendations) / Production (load_clean_data())**	**Analysis / Justification**
Method	Direct pd.read_csv() calls	Wrapper function (load_clean_data()) from preprocessing.py with logging, transformations, error handling	Shows project evolution

Benefits	Transparent, easy to debug in small experiments	Consistency, maintainability, single point of change	Mature system practices
Error Handling	Minimal	Built-in logging + error messages	Improves reliability
Why Not Unified	Kept for initial exploration in prototypes	Production uses refactored utilities	Maintains trace of evolution

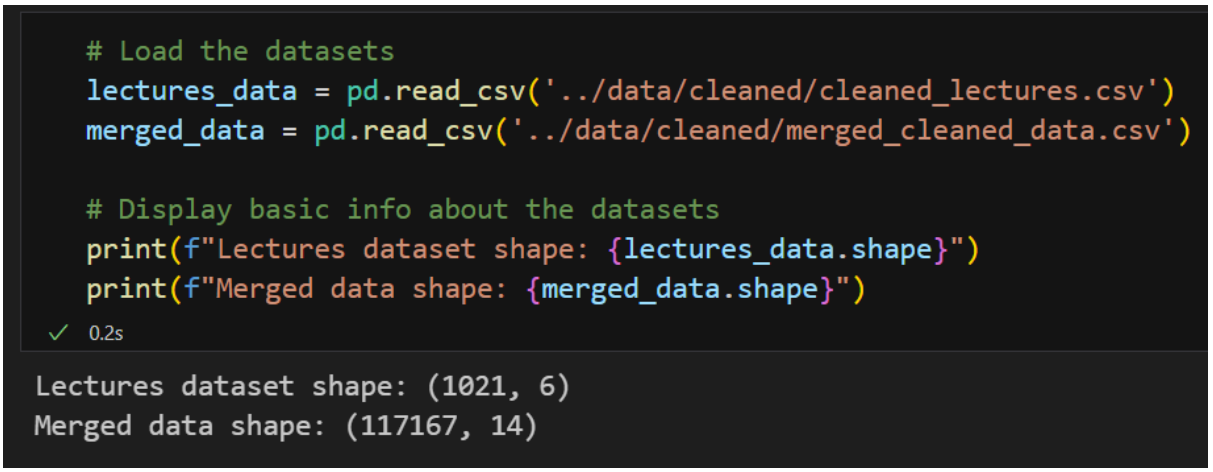


Figure 104: Direct data loading in Early & Experimental Notebooks

```python
def load_clean_data():
    try:
        project_root = os.path.dirname(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
        lectures_path = os.path.join(project_root, 'data', 'cleaned', 'cleaned_lectures.csv')
        merged_path = os.path.join(project_root, 'data', 'cleaned', 'merged_cleaned_data.csv')

        print(f"Loading from paths:\nLectures: {lectures_path}\nMerged: {merged_path}")

        if not os.path.exists(lectures_path):
            raise FileNotFoundError(f"Lectures file not found at {lectures_path}")
        if not os.path.exists(merged_path):
            raise FileNotFoundError(f"Merged file not found at {merged_path}")

        lectures_df = pd.read_csv(lectures_path)
        merged_df = pd.read_csv(merged_path)

        lectures_df['video_minutes'] = lectures_df['video_length'].fillna(0) / (1000 * 60)

        print(f"Successfully loaded {len(lectures_df)} lectures and {len(merged_df)} interactions")
        return lectures_df, merged_df

    except FileNotFoundError as e:
        print(f"File not found error: {e}")
        raise

    except pd.errors.EmptyDataError:
        print("Data files are empty")
        raise

    except Exception as e:
        print(f"Unexpected error loading data: {e}")
        raise
```

Figure 105: Wrapper function (load_clean_data()) in preprocessing.py

```python
# Load data
lectures_df, merged_df = load_clean_data()
print(f"Lectures: {len(lectures_df)}, Interactions: {len(merged_df)}")
```

Figure 106: .py files & Later Notebooks loading data using Wrapper function (load_clean_data()) from preprocessing.py

Table 34: Code Repetition

Aspect	Notebook Example	Production Code	Analysis / Justification
Example Case	NCF class in 05_advanced_recommendations.ipynb	src/recommender/ncf.py	Notebook served prototyping, .py serves as reusable module

Purpose	Educational transparency, quick prototyping	Centralized, reusable module for API + dashboard	Notebook serves dev; .py serves prod
Audience	For iterative experiments & demonstration	For integration in API & dashboard	Separation helps balance experimentation & production stability

Together, these three tables demonstrate that my codebase matured over time, from exploratory duplication in notebooks → to structured, reusable production code.

6.6 Final Reflections

This project combined **machine learning, education theory, and software engineering** to deliver a production-grade, research-informed recommender system. The blend of theoretical depth (NCF, meta-learners), practical constraints (duration handling, logging) and empirical evaluation (100+ tests, metric benchmarks) demonstrates its real-world applicability.

Overall, the system shows promise in transforming how students interact with educational content, moving from static, one-size-fits-all systems to dynamic, adaptive learning paths.

Although the system functioned as intended, the development process was not without challenges and moments of doubt. One such uncertainty arose early on: I questioned whether narrowing the scope to English exam (TOEIC) recommendations was too limiting, given that many recommender systems in academic literature target broader educational domains. However, this concern was resolved from the project proposal and preliminary report's feedback, which confirmed the value of applying personalization to a focused, high-impact domain.

Resources Used:

logo placed at the beginning: Fig. 8 Keyword coupling network diagram According to the number of...

Figure 1: 2.1.1. Hybrid Recommendation Systems
Figure 2, 12 to 18, 26, 30, 55: Figma.com
Figure 3: Interactive Hybrid Recommendation of Pedagogical Resources | IIETA
Figure 4: MoodleREC: A recommendation system for creating courses using the moodle e-learning platform
Figure 5 & 6: Course Learning Recommendation System Using Neural Collaborative Filtering
Figure 7: A Deep Neural Collaborative Filtering for Educational Services Recommendation
Figure 8: A BERT Based Hybrid Recommendation System For Academic CollaborationVellore Institute of Technology, Chennai, India
Figure 9: (PDF) BERT-Based Personalized Course Recommendation System from Online Learning Platform
Figure 10: SAINT+
Figure 11: Mermaid Live Editor
Figure 19: Mermaid Live Editor
Figure 21: Mermaid Live Editor
Figure 31: [1901.03888] Hybrid Recommender Systems: A Systematic Literature Review
Figure 32: Graphviz Online
Figure 33: Graphviz Online
Figure 46: Graphviz Online
Figure 47: Graphviz Online
Figure 37, 48, 50, 51, 61, 62, 64, 98, 99: Jupyter Notebook
Figure 53: [1708.05031] Neural Collaborative Filtering

References:

[1] Zilliz. 2025. How do hybrid recommender systems combine different techniques? Retrieved from Link

[2] U. U. Shaikh and Z. Asif. 2022. Persistence and dropout in higher online education: Review and categorization of factors. *Frontiers in Psychology*, 13, Article 902070. Link

[3] Google Cloud. 2025. Google Cloud AI recommendations overview. Retrieved from Link

[4] Xei. 2021. Recommender system tutorial. Retrieved from Link

[5] F. L. da Silva, B. K. Slodkowski, K. K. A. da Silva, and S. C. Cazella. 2022. A systematic literature review on educational recommender systems for teaching and learning: Research

trends, limitations, and opportunities. *Education and Information Technologies*, 28, 3289–3328. Link

[6] NetDragon Websoft Holdings. 2018. Edmodo partners with IBM Watson Education to close learning gaps using AI. Retrieved from Link

[7] C. Mediani. 2022. Interactive hybrid recommendation of pedagogical resources. *Ingénierie des Systèmes d'Information*, 27(5), 695–704. Link

[8] C. De Medio, C. Limongelli, F. Sciarrone, and M. Temperini. 2020. MoodleREC: A recommendation system for creating courses using the Moodle e-learning platform. *Computers & Education*, 159, Article 104020. Link

[9] H. L. Mulyana and F. Rumaisa. 2024. Course learning recommendation system using neural collaborative filtering. *Brilliance: Research in Artificial Intelligence*, 4(2), 517–524. Link

[10] F. Ullah, B. Zhang, R. U. Khan, T.-S. Chung, M. Attique, et al. 2020. Deep Edu: A deep neural collaborative filtering for educational services recommendation. *IEEE Access*, 8, 110915–110928. Link

[11] N. Sangeetha. 2025. FindMate: A BERT based hybrid recommendation system for academic collaboration. *arXiv preprint*, arXiv:2502.15223. Link

[12] K. Z. Moon, U. Salma, and M. S. Uddin. 2024. BERT-based personalized course recommendation system from online learning platform. In *Proceedings of the 6th International Conference on Electrical Engineering and ICT* (Dhaka). Link

[13] D. Shin, Y. Shim, H. Yu, S. Lee, B. Kim, and Y. Choi. 2021. SAINT+: Integrating temporal features for EdNet correctness prediction. In *Proceedings of the 11th International Learning Analytics and Knowledge Conference (LAK21)*, 490–496. Link

[14] Matsh.co. 2024. Statistics on personalized learning effectiveness. Retrieved from Link

[15] EduSynch. 2025. Top industries that require a TOEIC score. Retrieved from Link

[16] Educational Testing Service. n.d. TOEIC: Test of English for International Communication. Retrieved from Link and Link2

[17] Riiid. 2020. EdNet: A large-scale dataset for educational data mining. Retrieved from Link

[18] Papers with Code. n.d. EdNet: A large-scale dataset for educational data mining. Retrieved from Link

[19] EduData. 2021. EdNet-KT1: Knowledge tracing dataset. Retrieved from Link

[20] A. Amit. 2024. Building personalized recommender systems with deep learning: A practical guide. Retrieved from Link

[21] PYKT. 2022. EdNet: A large-scale dataset for educational data mining. Retrieved from Link

[22] M. Fluhler. 2023. Effectively handling large datasets. *Dataiku Blog*, September 25. Retrieved from Link

[23] V. Mandalapu, J. Gong, and L. Chen. 2021. Do we need to go deep? Knowledge tracing with big data. *arXiv preprint*, arXiv:2101.08349. Link

[24] Y. Choi, Y. Lee, D. Shin, J. Cho, S. Park, et al. 2020. EdNet: A large-scale hierarchical dataset in education. *arXiv preprint*, arXiv:1912.03072. Link

[25] A. Sahu. 2024. Why not the latest? Choosing the right Python version for your projects. Retrieved from Link

[26] Z. Rizvi. 2025. Jupyter notebooks best practices: Use virtual environments. Retrieved from Link

[27] EvidentlyAI. 2025. Precision@K and Recall@K: Ranking metrics explained. Retrieved from Link

[28] University of Virginia Library. 2022. ROC curves and AUC for models used for binary classification. Retrieved from Link

[29] Statology. 2021. MAE vs. RMSE: What's the difference? Retrieved from Link

[30] MetricsWatch. 2025. Understanding engagement metrics. Retrieved from Link

[31] G. Adomavicius and A. Tuzhilin. 2005. Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions. *IEEE Transactions on Knowledge and Data Engineering*, 17(6), 734–749. Link

[32] E. Çano and M. Morisio. 2017. Hybrid recommender systems: A systematic literature review. *Intelligent Data Analysis*, 21(6), 1487–1524. Link

[33] M. Das, S. Kamalanathan, and P. J. A. Alphonse. 2023. A comparative study on TF-IDF feature weighting method and its analysis using unstructured dataset. *arXiv preprint*, arXiv:2308.04037. Link

[34] H.-Q. Do, T.-H. Le, and B. Yoon. 2020. Dynamic weighted hybrid recommender systems. In *Proceedings of the 22nd International Conference on Advanced Communication Technology (ICACT)*, 44–650. Link

[35] N. Reimers and I. Gurevych. 2019. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. *arXiv preprint*, arXiv:1908.10084. Link

[36] X. He, L. Liao, H. Zhang, L. Nie, X. Hu, and T.-S. Chua. 2017. Neural collaborative filtering. *arXiv preprint*, arXiv:1708.05031. Link

[37] J. Sill, G. Takacs, L. Mackey, and D. Lin. 2009. Feature-weighted linear stacking. *arXiv preprint*, arXiv:0911.0460. Link

---

Appendix:

Appendix A: User Study Prompts and Feedback Form

A.1 Prompt to Participants

Participants were asked to interact with the recommendation system through the Streamlit interface. The following instructions were provided:

1. Select the user ID from the dropdown menu.
2. Review the recommended TOEIC content generated by the system.
3. Try switching between different recommendation modes (content-based, collaborative, hybrid).
4. Record your impressions regarding:
    - **Usefulness**: Did the recommendations seem relevant to your TOEIC practice?
    - **Clarity**: Was the interface easy to understand and navigate?
    - **Reliability**: Did the system consistently generate results for your user ID?

A.2 Feedback Form (completed by learners via PDF editor)

Q1. Usefulness of recommendations

- ☐ Very useful
- ☐ Somewhat useful
- ☐ Not useful

Q2. Did the system fail to generate recommendations for selected user ID?

- ☐ No, it worked consistently

- ☐ Yes, sometimes it showed no results
- ☐ Yes, frequently no results were generated

Q3. If you experienced missing recommendations, how did it affect your experience?
(Open text response)

Q4. How easy was it to use the system interface?

- ☐ Very easy
- ☐ Somewhat easy
- ☐ Difficult

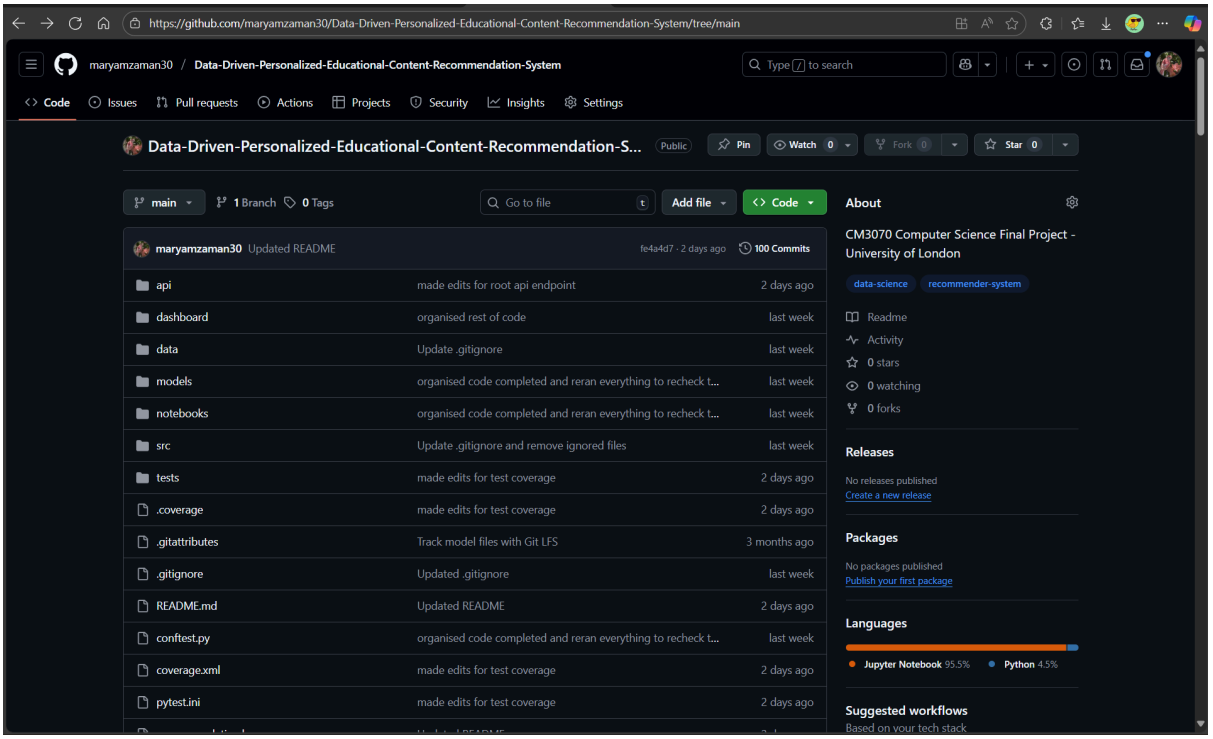
Q5. Any additional comments or suggestions?
(Open text response)

A.3 Summary of Feedback Collected

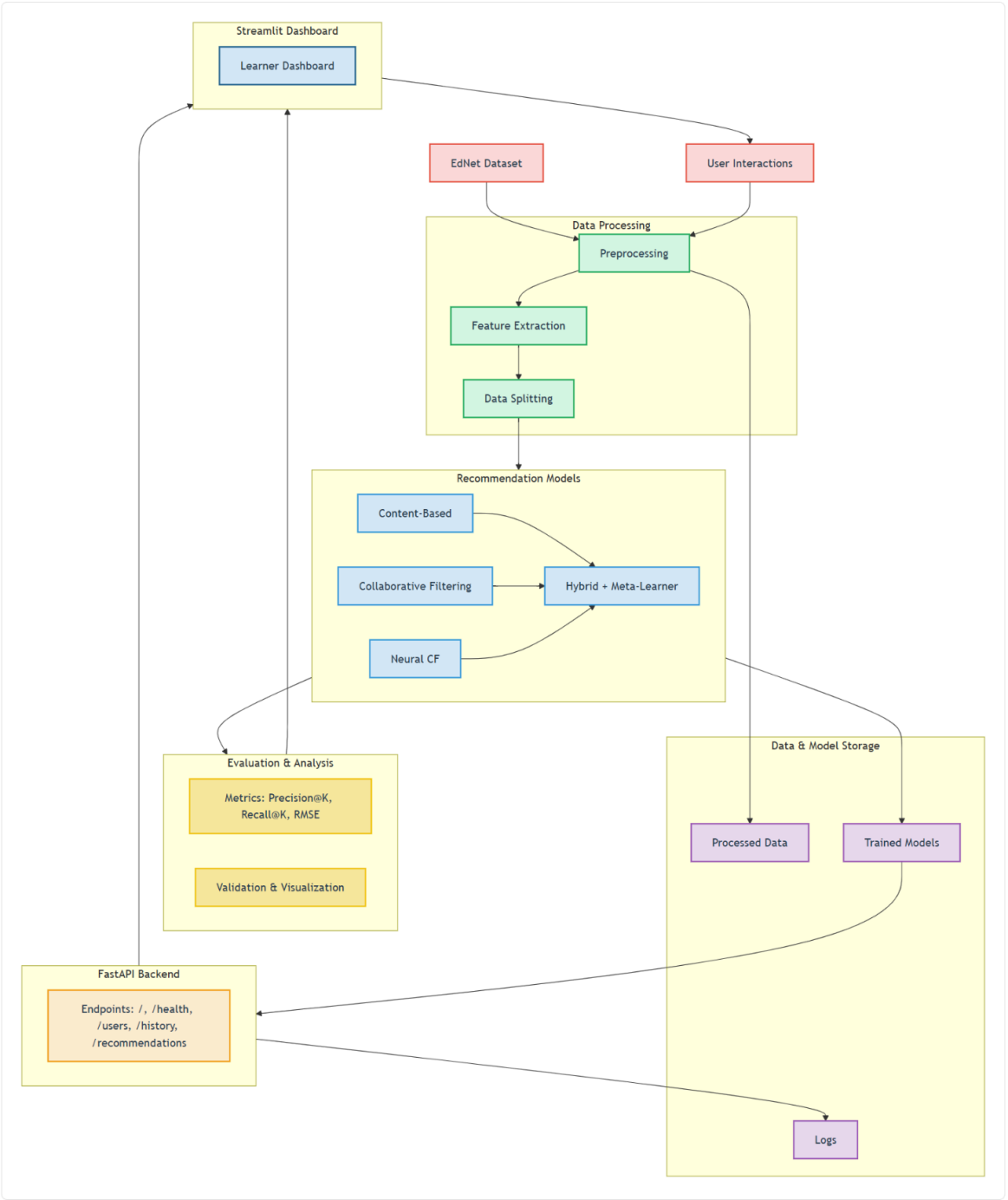
- All three learners found the recommendations generally useful for TOEIC practice.
- Two learners reported occasional frustration when no results were generated for some user ID.
- This confirmed the importance of handling **cold-start users** more effectively.
- Interface usability was rated as easy to understand.

Appendix B: GitHub Repository

[maryamzaman30/Data-Driven-Personalized-Educational-Content-Recommendation-System: CM3070 Computer Science Final Project - University of London]



Appendix C: Architecture and Flow of Project



Key Components

- **Data Processing:** Cleans and transforms EdNet + user interaction data.
- **Models:** Includes Content-Based, SVD-based Collaborative Filtering, Neural CF, and Hybrid Meta-Learner.
- **Evaluation:** Uses Precision@K, Recall@K, RMSE, and visualization for model assessment.
- **Storage:** Holds processed datasets, trained models, and logs.
- **API:** FastAPI endpoints expose recommendations and user history.
- **UI:** Streamlit dashboard provides learners with personalized TOEIC study recommendations.

Appendix D: Readme files of Project

Data-Driven Personalized Educational Content Recommendation System

A comprehensive recommendation system for personalized educational content, focusing on TOEIC preparation. This system leverages multiple machine learning approaches to provide tailored learning recommendations.

Features

- **Multiple Recommendation Strategies**:
  - Content-based filtering using SBERT embeddings
  - Collaborative filtering with SVD matrix factorization
  - Neural Collaborative Filtering (NCF)
  - Hybrid approaches combining the above methods
- **Interactive Dashboard**: Streamlit-based UI for exploring recommendations
- **RESTful API**: FastAPI backend for integration with other services
- **Model Evaluation**: Comprehensive metrics for assessing recommendation quality

Project Structure

```
project-root/
│
├── api/
│   └── main.py                    # FastAPI application for recommendations
│
├── dashboard/                     # Streamlit dashboard application
│   └── app.py                     # Interactive UI for exploring recommendations
│
├── data/
│   ├── cleaned/                   # Cleaned and processed datasets
│   ├── ednet_kt1_sampler.py       # Helper code for sampling
│   ├── lectures.csv               # Lecture content data
│   ├── questions.csv              # Question bank data
│   └── sampled_kt1_logs.csv       # Sampled data
│
├── models/                        # Pre-trained model files
│   ├── advanced_hybrid_model.pkl      # Advanced hybrid model
│   ├── content_based_model_best.pkl   # Content-based best model
│   ├── content_based_model.pkl        # Content-based model
│   ├── content_similarity.pkl         # Content Similarity
│   ├── ncf_model.pth                  # Neural Collaborative Filtering model
│   ├── svd_hybrid_model_best.pkl      # SVD-based hybrid best model
```

```
│   └── svd_hybrid_model.pkl              # SVD-based hybrid model
│
│
├── notebooks/                          # Jupyter notebooks for analysis & development
│   ├── 01_eda.ipynb                      # Exploratory Data Analysis
│   ├── 02_tf_idf.ipynb                   # Content-based filtering with TF-IDF
│   ├── 03_svd_hybrid.ipynb               # SVD-based hybrid model development
│   ├── 04_model_tuning.ipynb             # Hyperparameter optimization
│   ├── 05_advanced_recommendations.ipynb  # Advanced recommendation techniques
│   └── 06_sbert_ncf_subset_test.ipynb     # Experiments to prove feasibility of models
│
├── src/
│   ├── evaluation/                     # Evaluation metrics & analysis
│   │   └── metrics.py                    # Core evaluation metrics
│   │
│   ├── recommender/                      # Recommendation models
│   │   ├── hybrid.py                     # Hybrid recommendation system
│   │   ├── logic.py                      # Core recommendation logic
│   │   ├── ncf.py                        # Neural Collaborative Filtering
│   │   └── schemas.py                    # Pydantic data models for request & response
validation
│   │
│   └── utils/                          # Utility functions
│       ├── preprocessing.py              # Data preprocessing
│       └── mappings.py                   # ID mapping utilities
│
├── tests/                              # Unit & integration tests
├── .gitattributes                        # Git attributes file
├── .gitignore                            # Git ignore file
├── conftest.py                           # Add project root & `src/` to Python path
├── pytest.ini                            # Pytest Configuration file
├── recommendation.log                    # Log file
├── requirements.txt                      # Python dependencies
└── README.md                             # Project documentation (This file)
```

Getting Started

Prerequisites

- Python 3.8+
- pip (Python package manager)

Installation

Clone the repository:
git clone

https://github.com/maryamzaman30/Data-Driven-Personalized-Educational-Content-Recommendation-System.git
cd Data-Driven-Personalized-Educational-Content-Recommendation-System

1.

- **Make sure to clone the repository instead of downloading it. If you download it, the `.pkl` files will be corrupted. Also, ensure that you have Git LFS installed before cloning**
- **If you don't want to use GitHub LFS limits, then you have 2 other options, keep reading**

Create and activate a virtual environment:
Windows
python -m venv .venv
.venv\Scripts\activate

macOS/Linux
python3 -m venv .venv
source .venv/bin/activate

2.

Install dependencies:
pip install -r requirements.txt

3.

Model Files (Large Assets)

If you don't want to use GitHub LFS limits, then you have 2 other options:

1. Train all models from scratch

Run the Jupyter notebooks in the `notebooks/` directory after setting up your virtual environment and installing dependencies.

⚠️ Note: Training can take several hours.

2. Download from Google Drive

- Download from Google Drive
- Unzip the folder
- Copy all 7 model files into the `models/` directory.

Usage

⚠️ Ensure that the `models/` directory contains the required model files before starting the server.

Running the API Server

Start the FastAPI server:
cd api
uvicorn main:app --reload

1. The API will be available at `http://localhost:8000`

Running the Dashboard

In a new terminal, navigate to the dashboard directory:
cd dashboard

1.

Start the Streamlit app:
streamlit run app.py

2.
3. Open your browser to the URL shown in the terminal (typically `http://localhost:8501`)

API Testing

Available Endpoints

- `GET /` - API root
- `GET /health` - Health check
- `GET /users` - List all users
- `GET /user/{user_id}/history`
- `POST /recommend` - Get recommendations
    - Parameters:
        - `user_id`: Target user ID (e.g., 'u101324')
        - `method`: Recommendation method ('content', 'collaborative', 'hybrid', 'advanced_hybrid')
        - `n`: Number of recommendations to return

Testing with Browser

You can test GET endpoints directly in your browser:

- API root endpoint `http://localhost:8000`

- Health check: `http://localhost:8000/health`
- List users: `http://localhost:8000/users`
- User interaction history e.g. `http://localhost:8000/user/u101324/history` (u101324 is an example ID)

API documentaions

- You can explore API docs at `http://localhost:8000/docs`
- And explore API redoc at `http://localhost:8000/redoc`

Testing POST Requests

1. Using FastAPI's built-in Swagger UI

Since the backend is built with **FastAPI**, it automatically provides interactive API docs.

- Start the API (e.g., `uvicorn api.main:app --reload`).

Open your browser at:
http://localhost:8000/docs

-
- You'll see an interactive UI. Scroll to `POST /recommendations`.
- Click **Try it out**, fill in the request body JSON (example below) and click **Execute**.

```
{
  "user_id": "u101324",
  "n_recommendations": 5,
  "recommendation_type": "hybrid"
}
```

You'll immediately see the response from the API in the same page.

2. Using a tool like Postman or Insomnia

- Open Postman → New Request → choose `POST`.
- URL: `http://127.0.0.1:8000/recommendations`
- Select **Body** → **raw** → **JSON** and paste:

```
{
  "user_id": "u101324",
  "n_recommendations": 5,
  "recommendation_type": "hybrid"
}
```

- Hit **Send** → You'll get back your list of recommended bundles/lectures.

3. Using `curl` in the terminal

If you prefer command line:

```
curl -X POST "http://127.0.0.1:8000/recommendations" \
     -H "Content-Type: application/json" \
     -d '{
           "user_id": "u101324",
           "n_recommendations": 5,
           "recommendation_type": "hybrid"
         }'
```

4. in PowerShell

For the recommendation endpoint, use this PowerShell command:

```
$body = @{
    user_id = "u101324"
    recommendation_type = "content"
    n_recommendations = 5
} | ConvertTo-Json

$response = Invoke-RestMethod -Uri "http://127.0.0.1:8000/recommend" `
    -Method Post `
    -Body $body `
    -ContentType "application/json"

# Display the response
$response
```

Dataset Sources

- EdNet Dataset (KT1 & Contents)

Test documentations

To read about the testing strategy & test suite for this project, refer to the README file about tests.

License

This project is licensed under the MIT License - see the LICENSE file for details.

Acknowledgments

- The EdNet dataset provided by Riiid!
- Open-source libraries used in this project

Readme file for tests

Selected files

1 printable files

tests\README tests.md

tests\README tests.md

Testing Suite Documentation

This document provides comprehensive information about the testing strategy and test suite for the Data-Driven Personalized Educational Content Recommendation System.

Test Files Structure

```
project-root/
│
├── tests/
│   ├── README tests.md           # This File
│   ├── test_coverage.py          # Coverage tests
│   ├── test_edge_cases.py        # Edge case tests
│   ├── test_integration.py       # Integration tests
│   ├── test_logic.py             # Business logic tests
│   ├── test_mappings.py          # Data mapping tests
│   ├── test_metrics.py           # Evaluation metrics tests
│   ├── test_ml_models_simple.py  # Machine learning model tests
│   ├── test_performance.py       # Performance tests
```

```
│   ├── test_preprocessing.py      # Data preprocessing tests
│   └── test_schemas.py            # Data schema validation tests
│
├── conftest.py          # Test configuration and fixtures
├── pytest.ini           # Pytest configuration
└── recommendation.log   # Log file
```

Output files generated by terminal commands:

```
├── htmlcov/             pytest --cov=src --cov-report=html     # Folder containing HTML coverage reports
├── .coverage            # Binary data file
└── coverage.xml         pytest --cov=src --cov-report=xml      # XML coverage report for CI/CD
```

Configuration Files

pytest.ini

Configuration file for pytest with the following key settings:

- Test discovery patterns
- Coverage reporting
- Custom markers (slow, integration, performance, edge_cases, coverage)
- Output formatting

conftest.py

Sets up the Python path for test execution:

- Adds project root to Python path
- Adds src/ directory to Python path
- Enables test debugging

Coverage Files

- `.coverage`: Binary data file
- `htmlcov/`: Directory containing HTML coverage reports
- `coverage.xml`: XML coverage report (CI/CD integration)

Test Files in Detail

1. Unit Tests (`test_*.py`)

- **Purpose**: Test individual functions and classes in isolation
- **Coverage**: Core algorithms, utility functions, data processing
- **Tools**: pytest, unittest.mock

- **Files**:
  - `test_metrics.py`: Evaluation metrics testing
  - `test_preprocessing.py`: Data preprocessing functions
  - `test_schemas.py`: Data validation schemas
  - `test_mappings.py`: Data mapping utilities
  - `test_logic.py`: Core business logic

2. ML Model Tests (`test_ml_models_simple.py`)

- **Purpose**: Test machine learning models and algorithms
- **Coverage**: SVD recommender, NCF model, content similarity
- **Features**: Model training, prediction, persistence, edge cases
- **Validation**: Model outputs, convergence, error handling

3. Integration Tests (`test_integration.py`)

- **Purpose**: Test complete workflows and system integration
- **Coverage**: Full recommendation pipeline, API endpoints, dashboard
- **Features**: End-to-end testing, API validation, data flow
- **Tools**: FastAPI TestClient, mocked components

4. Performance Tests (`test_performance.py`)

- **Purpose**: Benchmark system performance and scalability
- **Coverage**: Response times, memory usage, concurrent operations
- **Features**: Load testing, resource monitoring, scalability analysis
- **Tools**: psutil, threading, time measurement

5. Edge Case Tests (`test_edge_cases.py`)

- **Purpose**: Test extreme scenarios and error conditions
- **Coverage**: Empty data, invalid inputs, system limits
- **Features**: Error handling, boundary conditions, stress testing
- **Validation**: Graceful degradation, meaningful error messages

6. Coverage Tests (`test_coverage.py`)

- **Purpose**: Ensure critical functionality across modules is exercised so coverage remains comprehensive.
- **Coverage**: Core recommenders (SVD, NCF), metrics, preprocessing, API endpoints, error handling, edge cases & performance monitoring.
- **Features**: Executes critical code paths, API endpoints & edge cases to prevent untested "dead zones".
- **Tools**: pytest, pytest-cov (coverage management is now fully handled by pytest-cov).
- **Output Files**:
  - `.coverage`: Binary coverage data (generated automatically by pytest-cov)

- `htmlcov/`: Interactive HTML report (generated when using `--cov-report=html`)
- `coverage.xml`: For CI/CD integration

Test Categories

Core Algorithm Tests

```
# SVD Hybrid Recommender
- Initialization and configuration
- Model training and fitting
- Rating prediction
- Recommendation generation
- Error handling and validation

# NCF Model
- Model architecture
- Forward pass computation
- Batch processing
- Memory efficiency
- Model persistence
```

API Endpoint Tests

```
# FastAPI Endpoints
- Health check endpoint
- User management endpoints
- Recommendation endpoints
- Error handling and validation
- Response format validation
```

Performance Benchmarks

```
# Performance Metrics
- Training time benchmarks
- Inference time benchmarks
- Memory usage monitoring
- Scalability testing
- Concurrent operation testing
```

Edge Cases and Error Handling

```
# Extreme Scenarios
- Empty datasets
- Single user/item datasets
- Sparse/dense datasets
- Invalid data types
- Missing data handling
- System resource limits
```

Running Tests

Basic Test Execution

```
# Run all tests
pytest

# Run with verbose output
pytest -v

# Run specific test file
pytest tests/test_metrics.py

# Run specific test function
pytest tests/test_metrics.py::test_precision_at_k
```

Test Categories

```
# Run only unit tests
pytest -m "not integration and not performance"

# Run only integration tests
pytest -m integration

# Run only performance tests
pytest -m performance

# Run only edge case tests
pytest -m edge_cases

# Run only coverage tests
pytest -m coverage
```

Performance Testing

```
# Run performance benchmarks
pytest tests/test_performance.py -v

# Run with timing information
pytest tests/test_performance.py --durations=10
```

Coverage Workflow

Generating Coverage Reports

```
# Run tests with coverage
pytest --cov=src --cov-report=term-missing

# Generate HTML report
pytest --cov=src --cov-report=html

# Generate XML report (for CI/CD)
pytest --cov=src --cov-report=xml

# Check coverage against threshold (fails if < 80%)
pytest --cov=src --cov-fail-under=80

# View coverage in browser (after generating HTML report) or go to htmlcov/
Folder
start htmlcov/index.html  # On Windows
open htmlcov/index.html   # On macOS
```

Interpreting Reports

- **Terminal Report**: Shows percentage coverage per file and missing lines
- **HTML Report**: Interactive interface showing covered/missing lines
- **Coverage Threshold**: Build fails if coverage falls below 80%

Test Configuration

pytest.ini Configuration

```
[tool:pytest]
testpaths = tests
python_files = test_*.py
python_classes = Test*
python_functions = test_*
addopts =
    -v
    --tb=short
    --strict-markers
    --disable-warnings
    --cov=src
    --cov-report=html:htmlcov
    --cov-report=term-missing
    --cov-fail-under=80
```

Coverage Configuration

Coverage is configured centrally via `pytest.ini` and handled by pytest-cov.

Test Data Management

Sample Data Generation

```
@pytest.fixture
def sample_data(self):
    """Create comprehensive sample dataset"""
    return pd.DataFrame({
        'user_id': ['u1'] * 10 + ['u2'] * 8 + ['u3'] * 12,
        'bundle_id': ['b1', 'b2', 'b3', 'b4', 'b5'] * 6,
        'user_answer': [1, 0, 1, 1, 0] * 6,
        'correct_answer': [1, 1, 0, 1, 1] * 6,
        'elapsed_time': [10, 15, 8, 12, 20] * 6,
        'timestamp': range(30)
    })
```

Mock Data for Testing

```
# Mock API responses
mock_users_response = {"users": ["u1", "u2", "u3"]}
mock_history_response = {
    "history": [
        {
            "question_id": "q1",
            "bundle_id": "b1",
```

```
            "timestamp": "2024-01-01T12:00:00",
            "is_correct": True,
            "elapsed_time": 10.5,
            "part": "Part 1",
            "subjects": ["listening"]
        }
    ],
    "total_interactions": 1
}
```

Logging

recommendation.log

- **Location**: Project root directory
- **Purpose**: Centralized logging for the recommendation system
- **Log Levels**: DEBUG, INFO, WARNING, ERROR, CRITICAL
- **Rotation**: Automatic log rotation (10MB per file, keeping 5 backups)

Viewing Logs

```
# View last 100 lines
tail -n 100 recommendation.log

# Follow log in real-time
tail -f recommendation.log

# Search for errors
grep ERROR recommendation.log

# On Windows
Get-Content recommendation.log -Tail 100

Get-Content recommendation.log -Wait

Select-String "ERROR" recommendation.log
```

Quality Assurance

Test Quality Metrics

- **Coverage Threshold**: 80% minimum code coverage
- **Performance Benchmarks**: Response time < 100ms per recommendation

- **Memory Efficiency**: < 500MB memory increase for large datasets
- **Error Handling**: Graceful degradation for all error conditions

Continuous Integration

```
# Example CI configuration
- name: Run tests
  run: |
    pytest --cov=src --cov-report=xml
    pytest --cov=src --cov-report=html

- name: Check coverage threshold
  run: |
    pytest --cov=src --cov-fail-under=80
```

Test Maintenance

- **Performance Monitoring**: Continuous performance benchmarking
- **Coverage Tracking**: Automated coverage reporting
- **Documentation**: Comprehensive test documentation

Troubleshooting

Common Issues

1. **Import Errors**: Check Python path configuration
2. **Mock Issues**: Verify mock setup and teardown
3. **Performance Failures**: Check system resources
4. **Coverage Gaps**: Review uncovered code paths
5. **Test Dependencies**: Ensure proper test isolation

Debugging Tests

```
# Run with debug output
pytest -v -s

# Run specific failing test
pytest tests/test_specific.py::test_function -v -s

# Check test discovery
pytest --collect-only
```

This comprehensive testing suite ensures the reliability, performance, and maintainability of the recommendation system while providing confidence in the quality of the implementation.